

ENGINEERING STUDY GUIDE

System Design by Case Study

Fifteen real systems, taken apart. What each one had to solve, the architecture it arrived at, what it gave up to get there — and the patterns that carry over to the next design you are asked to do.

Part I Foundations — the scaling ladder, estimation, and a pattern catalog

Part II Fifteen case studies — Twitter, WhatsApp, YouTube, Uber, Instagram, Airbnb, Redis, Spotify, Tinder, Zoom, Figma, Cloudflare, PayPal, Lyft, Bluesky

Part III Reference — pattern index, stale-facts appendix, numbers, study plan

A note on sourcing. Every scale figure in this guide carries a date and a citation. Where a widely-taught claim turns out to be stale, misquoted, or unsourced, it is flagged in the text and cataloged in Appendix B. Architecture changes; the lesson usually survives it.

How to use this guide

Read the foundations once. Then work the cases in any order — but work them, don't skim them.

There are two ways to get worse at system design. One is to learn no architectures. The other is to memorize fifteen of them.

Memorized architectures fail in the room because the interviewer — or the production incident — never hands you the exact problem you memorized. What transfers is not *"Twitter uses fanout-on-write"*. What transfers is the reasoning that produced it: a read path running at 300,000 requests per second against a write path running at 6,000, and a decision to move work onto the cheap side of that ratio. That reasoning is still correct. The architecture it produced has since been deleted.

So each chapter here is built to teach the reasoning and treat the architecture as evidence. The structure is the same every time:

Section	What it is for
At a glance	The scale facts, each with a date. Numbers are not timeless; a 2014 figure describing a 2026 system is a wrong answer delivered confidently.
Requirements	Functional and non-functional, plus the clarifying questions that should have been asked first. Most bad designs are correct answers to unasked questions.
Back of the envelope	The arithmetic that constrains the design. Do it yourself before reading it.
Architecture	The components and the request flow, with a diagram. This is the part people memorize; it is the least durable part of the chapter.
The hard part	The one problem that makes this system interesting rather than generic. This is the part worth memorizing.
The trade (inline)	A boxed callout at the moment a significant decision is made, rather than after the fact. See the note below — its last line works differently from everything else in this guide.
Trade-offs	The consolidated table at the end of each chapter. A design with no stated cost has not been understood.
What transfers	The portable patterns, stated so they apply to systems that are not this one.
Practice	Questions with no answer key. The answer key is the chapter.

On the “trade” boxes

A design read as a finished artifact always looks inevitable. A design read as a sequence of decisions looks contestable — which is the more useful way to read it, and the only way that helps when you face a problem nobody has written up. So at the significant decision points, a box interrupts:

THE TRADE — HOW TO READ THESE	
CHOSEN	What the system actually does.
OVER	The alternative it was selected against — often something the company tried first, or evaluated and rejected.
COSTS	What the choice gives up. This is the part that tends to go missing, because published write-ups are usually written by the people who made the decision.
REVERSES WHEN	<i>The conditions under which the other answer wins. This line is set in italics because it is inference, not citation — it is my reading of when the trade reverses, not something any of these companies published. Argue with it; that is more useful than accepting it.</i>

There are eleven of these, at the decisions where the cost is least obvious from the surrounding text. They supplement the end-of-chapter trade-off table rather than replacing it — the box catches the decision in the moment, the table is there when you come back to revise.

A suggested path

The fifteen cases are numbered as they were given, so you can map them back to the original reading list. But they are not in teaching order. If you are starting from scratch, this sequence builds each idea on the one before it:

Stage	Read	Because
1. Storage first	Redis (7) → Figma (11) → Cloudflare (12)	Every later chapter assumes you know what a cache, a shard, and a connection pool actually do. These three teach the data tier with almost no distributed-systems noise.
2. The feed problem	Twitter (1) → Instagram (5) → Bluesky (15)	The same problem — show a user content from people they follow — solved by precomputing, by placing data geographically, and by refusing to centralize at all.
3. Real-time delivery	WhatsApp (2) → Zoom (10) → PayPal (13)	Connection-oriented systems. What it costs to hold millions of sockets, and two different concurrency models for doing it.
4. Media at scale	YouTube (3) → Spotify (8)	Chunking, transcoding, adaptive bitrate — and why audio is a structurally easier problem than video.

5. Space and time	Uber ETA (4) → Lyft (14) → Tinder (9)	Geospatial indexing and matching. Three systems, one shared insight about how to shard the surface of the Earth.
6. Correctness	Airbnb (6)	Saved for last on purpose. It is the only chapter where the interesting constraint is <i>being right</i> rather than being fast, and it is the one most candidates handle worst.

ON THE FLAGS IN THIS TEXT

Three markers appear throughout. **Stale** means the fact was true and is now out of date — usually because the system changed and the teaching material didn't. **Un sourced** means a number is repeated everywhere but has no primary source; several of the most-quoted figures in system design fall here. **Conflated** means two real things have been merged into one wrong thing. Appendix B collects all of them in one place, and it is worth reading on its own — knowing which parts of the received wisdom are soft is itself a senior skill.

What this guide is not

It is not a substitute for the primary sources. Every chapter ends with the engineering blog posts, conference talks, and papers it was built from, and the best use of this guide is as an index into those. Where an original source sits behind a paywall, the chapter says so and fills the gap from published talks and papers instead — with the seam visible, not hidden.

It is also not a claim to know how these systems work *today*. Almost none of these companies publish current architecture. What they publish is a snapshot, usually several years old, usually of the part they were proud of. Read accordingly.

Contents

How to use this guide	2
-----------------------	---

PART I • FOUNDATIONS

1	The Scaling Ladder — one server to many, in twelve moves	7
2	Back of the Envelope — the arithmetic that constrains everything	12
3	A Framework for Any Design Question — and RADIO, for the client side	16
4	The Pattern Catalog — fourteen ideas the cases keep reusing	19

PART II • FIFTEEN CASE STUDIES

1	Twitter / X Timeline — precompute, then delete the precomputation	26
2	WhatsApp — what you gain by refusing to store anything	35
3	YouTube — chunk it, and the problem becomes parallel	46
4	Uber ETA — turning a map into a graph you can afford to search	56
5	Instagram — data placement as a latency strategy	63
6	Airbnb — where being right matters more than being fast	69
7	Redis at Scale — one datastore, eight jobs	81
8	Spotify — split the object, prefetch the rest	87
9	Tinder — sharding the surface of the Earth	95
10	Zoom — move the work to the client, route only bytes	101
11	Figma — five ways to scale Postgres, in order	106
12	Cloudflare — queueing theory as a database feature	112
13	PayPal — a billion transactions on eight machines	118
14	Lyft — the sidecar, and why Envoy exists	124
15	Bluesky — what happens when you refuse to own the data	132

PART III • REFERENCE

A	Pattern Index — every pattern, and where to see it working	143
B	Stale Facts and Confident Errors — what the received wisdom gets wrong	147
C	Numbers Worth Knowing — latency, capacity, availability	155
D	A Fifteen-Week Study Plan	159

PART ONE

Foundations

Four chapters that every case study assumes you have read. The ladder that systems climb, the arithmetic that decides which rung you are on, a procedure for attacking an unfamiliar problem, and the fourteen patterns you will then watch fifteen companies reuse.

-
- | | |
|---|--------------------|
| 1 | The Scaling Ladder |
|---|--------------------|
-
- | | |
|---|----------------------|
| 2 | Back of the Envelope |
|---|----------------------|
-
- | | |
|---|-------------------------------------|
| 3 | A Framework for Any Design Question |
|---|-------------------------------------|
-
- | | |
|---|---------------------|
| 4 | The Pattern Catalog |
|---|---------------------|
-

The Scaling Ladder

One server to millions of users, in twelve moves — and the new problem each move creates.

Nobody designs the final architecture on day one. Systems arrive at their shape by hitting a wall, moving one thing, and hitting the next wall. Knowing the sequence is most of the skill, because the sequence is almost always the same.

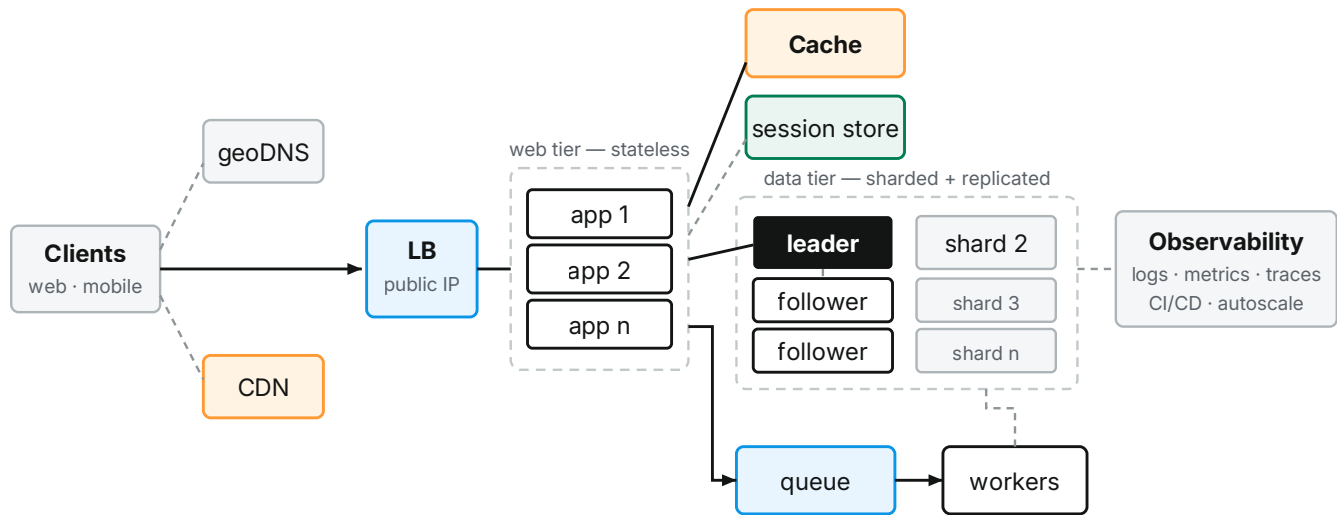
What follows is the canonical progression, drawn from the opening chapter of *System Design Interview*.

¹ Read it as a ladder: each rung solves the problem created by the rung below, and creates the problem solved by the rung above. The thing worth internalizing is not the destination — it is that **every move buys a capability and charges a price**, and being able to name the price is what separates a design from a diagram.

The twelve moves

#	Move	Solves	Introduces
1	Single server	Nothing yet — web app, database and cache on one box. This is the correct architecture for most software that has ever been written.	One failure kills everything. No independent scaling.
2	Split the data tier	Web and database now scale independently, because they exhaust different resources — the web tier runs out of CPU, the data tier runs out of IOPS.	A network hop on every query. Two things to operate.
3	Choose SQL or NoSQL	Relational is the default and has been for forty years. Reach past it for super-low latency, unstructured data, pure serialize/deserialize access, or genuinely massive volume.	Non-relational stores generally give up joins — which pushes work into the application.
4	Load balancer	Removes the web tier as a single point of failure and lets you add capacity by adding servers. Clients now reach a public IP; servers sit on private IPs.	Server-held session state breaks, because request $n+1$ may land elsewhere.
5	Database replication	Reads distribute across followers; the leader takes writes. Most applications read far more than they write, so followers outnumber leaders. Also buys durability and failover.	Replication lag. A read after a write may not see the write — the defect behind a whole class of "impossible" bugs.

6	Cache tier	An in-memory layer in front of the database absorbs the repeated-read traffic that dominates most workloads.	Invalidation, staleness, eviction policy, and a new failure domain. See the callout below.
7	CDN	Static assets — images, CSS, JavaScript, video segments — served from a node near the user instead of from your origin.	Cost per byte, cache-expiry tuning, invalidation, and a fallback path for when the CDN is down.
8	Stateless web tier	Move session state out of the web servers into a shared store. Now any server can serve any request, and autoscaling becomes trivial.	Every request pays a lookup to fetch state that used to be local.
9	Multiple data centers	geoDNS routes users to the nearest region; a regional outage becomes a traffic shift rather than an outage.	Cross-region data synchronization — the hardest problem on this list. Also: testing and deploying consistently everywhere.
10	Message queue	Producers and consumers decouple. Slow work — image processing, transcoding, email — leaves the request path. Each side scales on its own signal.	Eventual consistency becomes user-visible. Queue depth becomes a thing you must monitor and alert on.
11	Shard the data tier	Horizontal partitioning by a shard key. The only move on this list that raises the write ceiling.	Resharding, cross-shard joins, and the celebrity problem — see below.
12	Logging, metrics, automation	Optional at three servers; existential at three hundred. Host metrics, aggregate tier metrics, business metrics, and CI/CD.	Nothing. This is the one move with no downside, which is why skipping it is the most common mistake on the list.



Solid: request path — writes to the leader, reads from cache and followers. Dashed: out-of-band — replication, async work, telemetry.

Figure 1.1 — The top of the ladder. Almost every system in Part II is a specialization of this picture: one tier gets replaced by something exotic, and the rest stays recognizable. Twitter swaps the cache for a purpose-built Redis fork; YouTube swaps the workers for a transcoding ASIC; Bluesky deletes the middle entirely.

Three rungs that deserve more than a table row

Rung 5 — replication lag is a correctness problem, not a latency problem

Adding read replicas is the easiest capacity win on the ladder and the one that most reliably produces confusing bugs. The failure is always the same shape: you write to the leader, immediately read from a follower, and get the pre-write value. In a user interface this looks like "my change didn't save". In a payments system it looks like a duplicated charge.

Airbnb hit exactly this and drew the general rule: strong-consistency state must be read **from the primary only**, because "just a few seconds of replica lag could cause significant financial impact".² The lesson generalizes past money — any read whose result gates a write needs to come from the leader. Chapter 6 works through what that costs.

Rung 6 — the cache is where the interesting failures live

A cache is four decisions, and getting any of them wrong produces a distinct outage:

- **What to cache.** Data read often and written rarely. A cache is volatile memory; anything that must survive a restart belongs in a durable store as well.
- **Expiry.** Too short and you reload constantly, defeating the cache. Too long and you serve stale data. There is no universally right answer, which is why this is a decision and not a default.
- **Consistency.** The store and the cache are updated by separate operations that are not in one transaction, so they will diverge. Facebook's *Scaling Memcache* paper is the canonical treatment.³
- **Eviction.** LRU is the usual choice; LFU and FIFO each suit particular access patterns.

THE THUNDERING HERD

Invalidate a hot key and every request that was being served from cache arrives at the database simultaneously. The database, sized for cache-miss traffic rather than total traffic, falls over — and now *nothing* is cached, so it stays over.

Instagram's answer is a **Memcache lease**: the first request to miss is given a lease and sent to Postgres; concurrent requests for the same key are told to wait, or handed the stale value, which for something like a like-count is entirely acceptable.⁴ Chapter 5 has the full workflow. The general pattern — *one miss does the work, everyone else waits or accepts stale* — is worth knowing by name, because it appears under at least four different names across vendors: request coalescing, single-flight, dogpile prevention, and cache stampede protection.

Rung 11 — sharding, and the celebrity problem

Sharding is the only move that raises the write ceiling, and the choice of shard key is the whole design. A good key distributes data evenly. The three costs are permanent:

- **Resharding.** When one shard fills, or fills faster than its peers, the shard function changes and data must move. Consistent hashing is the standard mitigation.
- **The celebrity (or hotspot) problem.** Uneven *access*, not uneven data. If several very popular entities hash to the same shard, that shard is overwhelmed with reads while its peers idle. It may need a shard of its own, then a partition of that shard.
- **Joins.** Cross-shard joins are impractical, so schemas get denormalized until each query touches one table on one shard.

The celebrity problem recurs so often in Part II that it is worth spotting early. Twitter meets it as accounts with tens of millions of followers (Chapter 1). Tinder meets it as time zones, where peak traffic moves across the world and unbalances shards by hour of day (Chapter 9). Lyft meets it as San Francisco carrying orders of magnitude more traffic than a small market, which is why sharding by city could never balance (Chapter 14). Cloudflare meets it as one tenant issuing an expensive query and starving its neighbors (Chapter 12). Same problem, four disguises.

THE LADDER IN ONE PAGE

1. **Keep the web tier stateless.** Everything else on the ladder gets easier, and autoscaling becomes free.
2. **Build redundancy at every tier.** A tier with one instance is an outage with a schedule.
3. **Cache aggressively — and own the four cache decisions explicitly.**
4. **Put static assets in a CDN.** The cheapest latency win available.
5. **Scale the data tier by sharding,** and choose the key knowing you will live with it for years.
6. **Decouple with queues** so slow work cannot hold the request path hostage.
7. **Support multiple data centers** when availability, not capacity, becomes the binding constraint.
8. **Monitor and automate.** The only move with no trade-off, and therefore the one with no excuse.

PRACTICE

1. You add read replicas and the "edit profile" flow starts intermittently showing users their old name. Name three fixes and state which one you would ship and why.
2. Your cache hit rate is 99%. The database is sized for the 1%. Describe precisely what happens when a deploy flushes the cache, and what you would have built beforehand to survive it.
3. You are sharding a table of user posts. Compare `user_id`, `post_id`, and `created_at` as shard keys. For each, name the query that becomes expensive and the hotspot that becomes possible.
4. Which rung of the ladder is the system you currently work on standing on? Which wall will it hit next?

SOURCES

1. ByteByteGo, *System Design Interview*, ch. 2, "Scale From Zero To Millions Of Users" — bytebytego.com/courses/system-design-interview/scale-from-zero-to-millions-of-users
2. Chew & Khisti, "Avoiding Double Payments in a Distributed Payments System", Airbnb Tech Blog, Apr 2019
3. Nishtala et al., "Scaling Memcache at Facebook", USENIX NSDI '13
4. Neo Kim, "How Instagram Scaled to 2.5 Billion Users", The System Design Newsletter #49, Jun 2024; and Instagram's own "Scaling Instagram Infrastructure" talk

Back of the Envelope

Four calculations, done in ninety seconds, that decide the entire architecture.

The point of estimation is not the number. The point is that the number eliminates most of the design space before you have drawn a single box.

Consider two systems from Part II. Booking.com handles roughly **17 bookings per second** against roughly **1,000 reads per second**.¹ WhatsApp handles roughly **350,000–500,000 messages per second** at peak.² Those two numbers produce entirely different architectures, and you can derive most of the difference before knowing anything else. At 17 writes per second you can afford local ACID transactions, a single relational primary, and pessimistic locking — and Chapter 6 shows that Airbnb's team argued exactly that, rejecting distributed transactions as unjustifiable complexity at that volume. At 500,000 writes per second none of those options exist.

The four calculations

1. QPS — and the peak multiplier

Start from daily active users, multiply by actions per user per day, divide by 86,400 seconds. Then multiply by **3 to 5 for peak**. That last step is the one people skip, and it is the one that determines how much hardware you buy.

```
WhatsApp, worked:
500M DAU x 20 messages/day      = 10B messages/day
10B / 86,400                     = ~115,000 msg/sec average
x 3-5 peak multiplier            = 350,000-500,000 msg/sec peak
```

PEAKS ARE NOT 5×. SOMETIMES THEY ARE 25×.

On 3 August 2013, during a Japanese television broadcast of *Castle in the Sky*, Twitter recorded **143,199 tweets in a single second** — against a steady-state average of about 5,700 per second. That is roughly **25× normal**, and Twitter's write-up notes that users "didn't experience a blip".³

The lesson is not "provision for 25×" — nobody can afford that. It is that your peak multiplier is a property of your product, not a rule of thumb. A social network tied to live broadcast events has a different distribution from an enterprise SaaS tool. Ask what the worst plausible minute looks like, and design the *degradation* for it even if you cannot provision for it.

2. Storage — and the retention question first

Bytes per record × records per day × retention. The third term is the one that matters, and it is a product decision, not an engineering one.

WhatsApp, worked:

1 KB/message × 10B messages/day = 10 TB/day of message data
× 365 days = 3.6 PB/year <-- if you kept it

But messages are deleted on delivery. Actual figure:

30-day retention for undelivered + ~10% keeping history
= 400-500 TB of active storage

An order of magnitude of difference, produced entirely by a retention policy. This is why Chapter 2 argues that *choosing not to store* is a scalability strategy rather than an omission: the architecture "can revolve around throughput, caching, and partitioning" instead of durability, replication factor, compaction, and retention.²

3. Bandwidth

Concurrent connections × bytes per second per connection. Or, for media: streams × bitrate. This is where media systems separate from everything else — Spotify's engineers observed that a 320 kbps audio track is roughly two orders of magnitude below a 4K video bitrate,⁴ which is why a whole music catalog fits in a client-side cache and a video catalog does not.

4. The read/write ratio

The single most consequential number, because it tells you which side of the system to optimize and therefore whether to precompute.

System	Ratio	What it licensed
Twitter, circa 2012	300K reads/s vs 6K writes/s — 50:1	Fanout-on-write. Doing O(followers) work per tweet is rational when reads outnumber writes 50 to 1. (Chapter 1)
YouTube	100:1 views to uploads	Everything optimized for the read path: CDN, adaptive bitrate, pre-generated renditions. (Chapter 3)
Booking.com	1,000 reads/s vs 17 writes/s — ~60:1	Heavy read caching and a search index — but writes stay strongly consistent, because correctness beats throughput on the booking path. (Chapter 6)
WhatsApp	Roughly 1:1	Neither side precomputes. The server is a transit buffer, not a repository. (Chapter 2)

Note the outlier. WhatsApp's near-symmetric ratio is why it is the one system in this guide with no materialized views, no ranking layer, and no cache of derived data — there is no cheap side to move work onto.

Numbers you should know cold

Operation	Order of magnitude
L1 cache reference	~1 ns
Main memory reference	~100 ns

Read 1 MB sequentially from memory	~10 μ s
SSD random read	~100 μ s
Read 1 MB sequentially from SSD	~1 ms
Round trip within the same data center	~0.5 ms
Disk seek (spinning)	~10 ms
Round trip California → Netherlands	~150 ms

The two ratios worth carrying in your head: memory is roughly **1,000× faster** than SSD for random access, and a cross-continent round trip costs roughly **300× a same-datacentre one**. Nearly every caching and data-placement decision in Part II falls out of one of those two facts.

Quantity	Useful approximation
Seconds in a day	86,400 — call it 10^5
1 million QPS sustained	$\approx$ 86 billion requests/day
A UUID	16 bytes binary, 36 chars as text
A timestamp	8 bytes
A tweet-sized text record with metadata	~1 KB
A 3.5-minute audio track at 128 kbps	~3 MB
One minute of 1080p video	~50-100 MB before transcoding
Availability: 99.9% / 99.99% / 99.999%	8.8 h / 53 min / 5.3 min of downtime per year

THE FAN-OUT AVAILABILITY TRAP

Airbnb's property detail page calls **20 downstream services**. If each is independently 99.9% available and nothing mitigates the dependency, the page's availability is $0.999^{20} \approx$ **98.0%** — about **14.5 hours of downtime per month**.⁵

This arithmetic is the entire justification for circuit breakers, bulkheads, request deadlines, and graceful degradation. Run it once and the resilience patterns stop looking like over-engineering.

PRACTICE

1. Estimate storage for a photo-sharing service: 500M DAU, 10% post daily, 2 MB per photo, three renditions each, kept forever. Now recompute with a two-year retention policy for the largest rendition. How much did the product decision save?

2. A service needs 99.99% availability and depends on six services at 99.95% each. Is the target reachable? What would you have to build for it to be?
 3. Your design calls for 50 TB of data served at 200,000 QPS with a 1 ms p99. Can this be served from memory? From SSD? What does the answer force you to change?
 4. Take the last feature you shipped. What was its read/write ratio? Did the design reflect it?
-

SOURCES

1. Neo Kim & Hayk Simonyan, "System Design Interview: Design Airbnb", The System Design Newsletter #113, Jan 2026
2. Neo Kim & Hayk Simonyan, "System Design Interview: Design WhatsApp", The System Design Newsletter #106, Dec 2025; Rick Reed, Erlang Factory 2014
3. "New Tweets per second record, and how!", Twitter Engineering Blog, Aug 2013
4. Kreitz & Niemelä, "Spotify — Large Scale, Low Latency, P2P Music-on-Demand Streaming", IEEE P2P '10
5. He & Guo, "Building Services at Airbnb, Part 3", Airbnb Tech Blog, Dec 2018

A Framework for Any Design Question

Four steps for the server side, five for the client side, and the questions that make both work.

Most people fail system design questions the same way: they start drawing before they know what they are building. The framework's only job is to stop that.

The four-step structure

Step	Phase	What you actually do
1	Understand the problem ~15% of the time	Ask questions until the scope is bounded. Establish functional requirements, non-functional requirements, and scale. Write down what is explicitly <i>out</i> of scope — this is how you avoid being judged on something you were never asked to build.
2	High-level design ~25%	Propose the components and the request flow. Get agreement before going deep. This is also where the API surface and the data model belong, because they constrain everything downstream.
3	Design deep dive ~50%	Pick the one or two genuinely hard parts and go all the way down. Which parts are hard is usually signaled by the interviewer, and is always the part with the interesting trade-off.
4	Wrap up ~10%	Bottlenecks, failure modes, operational concerns, and what you would do with more time. Naming your design's weaknesses is a strength signal, not a weakness one.

The questions to ask in step 1

Here is the shape of a good opening, taken from the YouTube chapter's framing. Notice that every question has a consequence attached — the point of asking is not to seem thorough, it is that the answer changes the design:¹

Question	Answer	What it forces
What scale are we designing for?	1M uploads/day, 100M DAU	Sharding, partitioning and caching strategy
What is the read-to-write ratio?	~100 views per upload	Read-heavy — the whole design tilts toward streaming
Maximum file size?	256 GB	Multipart upload and chunking become mandatory
Which formats and resolutions?	MP4/AVI/MOV, 240p–4K	A transcoding pipeline with many output renditions

Target streaming latency?	First frame < 500 ms	CDN plus adaptive bitrate streaming
Acceptable upload processing time?	10–30 minutes	Processing can be fully asynchronous
Is eventual consistency acceptable for new uploads?	Yes	Availability over immediacy — a CAP decision, made explicitly
Uptime target?	99.9%	Redundancy and failover at every tier
Do we support live streaming?	Not for this exercise	Scope control — the most valuable question on the list

THE ONE QUESTION ALMOST NOBODY ASKS

"How long do we have to keep this data?" It reshapes the architecture more than any other single answer, and Chapter 2 is a 100-billion-messages-per-day system that is architecturally simple almost entirely because the answer is "until delivered".

RADIO — the same discipline for the client side

Front-end system design is a real and increasingly common variant, and it needs a different structure because the constraints are different: rendering efficiency, client state, network utilization, and consistency between client and server — not query latency and replication. Yangshun Tay's **RADIO** framework is the cleanest version:²

	Phase	For a timeline UI, this means
R	Requirements	Is the focus rendering the feed, infinite scroll, or real-time updates? Scope it before designing it.
A	Architecture	The layers: view, store, data access, server. Where rendering happens (server, client, hybrid) and how navigation works (SPA or MPA).
D	Data model	The client-side entities — tweet, user, timeline — and crucially whether they are <i>normalized</i> . See below.
I	Interface	Both the network API and the internal component boundaries. Each with intent, parameters, and response shape.
O	Optimizations	Code splitting, list virtualization, optimistic updates, media handling, pagination strategy.

RADIO is a checklist, not a sequence — architecture, data model and interface are usually discussed together, and some people find the data model easier to fix first. What matters is leading with requirements and ending with optimizations.

CLIENT-SIDE NORMALIZATION — THE IDEA WORTH STEALING

A tweet arriving from the server contains a nested author object. So does a retweet, and so does each reply. Store those naively and one user's display name is duplicated across hundreds of objects; change their avatar and every copy must be found and updated.

Normalizing the client store — users in one collection keyed by id, tweets in another holding only *references* — means a profile change updates exactly one entry and every component referencing it re-renders correctly. It also makes pagination, deduplication and cache merging straightforward, and it is what allows an optimistic like to increment a count once rather than in four places.

This is the same pattern as Chapter 1's server-side decision to store tweet IDs rather than tweet objects. It is one idea appearing at two layers.

PRACTICE

1. Write the ten clarifying questions you would ask for "design a ride-hailing service", each with the design consequence of the answer.
2. For "design Google Docs", decide where rendering happens and justify it against SSR, CSR, and hybrid.
3. You have 45 minutes and the interviewer says "design Instagram". Allocate the time across the four steps, and name in advance which deep dive you intend to take.

SOURCES

1. Neo Kim, "System Design Interview: Design YouTube", The System Design Newsletter #117, Jan 2026
2. Neo Kim & Yangshun Tay, "System Design Interview: Design Twitter/X Timeline — A Frontend Deep Dive", The System Design Newsletter #103, Nov 2025; GreatFrontEnd front-end system design playbook

The Pattern Catalog

Fourteen ideas. Fifteen companies. Almost no new inventions.

Read the fifteen case studies and a suspicion sets in: these companies are not solving fifteen problems. They are solving about fourteen, in different costumes.

This chapter names them. Each entry gives the idea, the mechanism, and where in Part II you will watch it work — including, where it is instructive, a case that chose the opposite. Appendix A inverts this into a full cross-reference.

1. Read/write asymmetry — precompute or compute on read

Measure the ratio of reads to writes, then move work onto the cheap side. **Fanout-on-write** does $O(\text{followers})$ work when content is created so that reads are $O(1)$. **Fanout-on-read** does nothing on write and assembles at query time. They are duals, and the ratio decides.

The transferable insight is not that one is better. It is that **the choice can be made per-entity rather than globally**. Twitter ran fanout-on-write for timelines and fanout-on-read for search — $O(n)$ -write/ $O(1)$ -read in one subsystem, the exact inverse in another, in the same building. And when the economics inverted, the fanout service was deleted outright.

See: Ch. 1 (Twitter, both directions, then the retirement) · Ch. 15 (Bluesky, where indexing replaces fanout entirely) · **Counter-case:** Ch. 2 (WhatsApp, a ~1:1 ratio, so neither side precomputes).

2. Store references, hydrate on read

Do not cache objects. Cache identifiers, and fetch the objects in parallel from services that have their own caches.

Twitter's materialized timelines stored, per entry, a tweet ID, the originating user ID, and four bytes of bit flags — not the tweet. That is what let 800 entries for every active user fit in a few terabytes of RAM; the read path then multi-gets tweet bodies and user records from two independently cached services. The client-side equivalent is store normalization (Chapter 3).

See: Ch. 1 (Twitter) · Ch. 15 (Bluesky's index server builds lists of post IDs, then expands) · Ch. 3 (YouTube's manifest is a list of references).

3. Cache invalidation, leases, and the thundering herd

A cache in front of a database creates three problems: staleness, invalidation, and the stampede when a hot key expires.

Instagram's answer is worth learning as a unit: don't write into the cache from the application — have a separate service watch Postgres for updates and invalidate. Then use a **lease** so that on a miss, one

request goes to the database and the rest either wait or are handed stale data. For a like count, stale is fine. Knowing when stale is fine is the skill.

See: Ch. 5 (Instagram, leases and denormalized counter tables) · Ch. 7 (Redis, the cache-aside mechanics) · Ch. 12 (Cloudflare, the herd as a connection-count problem).

4. Sharding, hot shards, and the celebrity problem

Sharding raises the write ceiling. The recurring failure is not uneven data, it is uneven *access*.

The lesson repeated most often across Part II: **shard on the dimension the load actually varies along, not on the human-legible label**. Lyft sharded driver locations by city and got hot shards, because San Francisco is not Boise. Tinder avoids uniform geographic grids because ocean cells are empty and city cells overflow, then randomly assigns many shards to each physical server to smooth out time-zone-driven peaks.

See: Ch. 1 (accounts with 30M+ followers) · Ch. 9 (Tinder, time zones) · Ch. 11 (Figma, hash-based routing) · Ch. 14 (Lyft, city vs cell) · Ch. 12 (Cloudflare, one tenant starving its neighbors).

THE CELEBRITY PROBLEM'S WORST SYMPTOM IS NOT COST

When a very-high-follower account posts, the expensive part is not the CPU. It is that fanout takes minutes to complete, so **a reply can be delivered to some users before the original post is**. That is a causality violation produced purely by queue latency, and sorting by monotonically increasing IDs does not fix it.

Generalized: **fan-out latency is a consistency property**. Chapter 1 develops this properly; it is the single most under-appreciated idea in the catalog.

5. Geospatial indexing

"Find things near me" is not answerable from raw latitude and longitude at scale. The fix is to map two-dimensional space onto a one-dimensional key that preserves locality, then shard on that key.

Three systems in Part II use the same family of tools — geohash, Google's S2, Uber's H3 — and the shared insight is hierarchy: a cell can be subdivided, so you can choose a granularity that puts a manageable number of entities in each shard. Uber's published argument for hexagons over squares is precise: a square cell has two distinct center-to-neighbor distances (four edge-sharing, four vertex-sharing), a triangle has three, a hexagon has one — which makes a *k*-ring a good approximation of a circle and simplifies analysis over gradients.

See: Ch. 9 (Tinder, S2 and the Hilbert curve) · Ch. 14 (Lyft, cells and Redis sorted sets; Uber's H3) · Ch. 4 (Uber, graph partitioning — the routing cousin of the same idea).

6. Idempotency and exactly-once

Networks retry. Users double-click. Therefore any operation with a side effect needs a client-supplied key that makes the second attempt a no-op rather than a second effect.

Airbnb's Orpheus library is the most completely documented example in this guide, and its structure is the lesson: split every request into **Pre-RPC / RPC / Post-RPC**, with two hard rules — no network calls in the Pre- and Post-RPC phases, and no database interaction during the RPC phase. Take a row-level lock on the idempotency key, granting a *lease* that expires; set the lease expiry longer than the RPC timeout. Classify errors retryable or not, defaulting to not.

See: Ch. 6 (Airbnb, in full) · Ch. 3 (YouTube, resumable multipart upload) · Ch. 13 (PayPal, where a duplicate is money).

7. Concurrency control — four mechanisms, ranked

When two writers race for one resource, you have four real options and one trap.

Mechanism	Guarantees	Fails when
Pessimistic lock SELECT ... FOR UPDATE	Correct at any isolation level $\geq$ READ COMMITTED. Simplest correctness story.	You hold it across a network call. Throughput becomes $1/(\text{lock hold time})$; a 2-second payment call serializes the resource to well under one operation per second.
Optimistic / version column	Prevents lost updates at the granularity of the versioned row.	Contention — retries collide and tail latency degrades superlinearly. Also breaks if you validate the version against a lagging replica.
Database constraint unique index, or exclusion constraint	The strongest option: the invariant cannot be violated by <i>any</i> code path — not a buggy retry, not a backfill, not a second service.	The loser learns by exception rather than by predicate, so you must translate the error. And it cannot span shards.
Distributed lock Redis, ZooKeeper, etcd	Nothing you should rely on for correctness. Leases expire while holders still believe they hold them; single-instance locks lose mutual exclusion on failover.	Used as the invariant. It is legitimate only as admission control <i>in front of</i> a real constraint.

Chapter 6 works all four through the double-booking problem, including the PostgreSQL exclusion constraint that expresses "no overlapping date ranges for this listing" as a single line of DDL.

8. Backpressure, head-of-line blocking, and circuit breaking

Head-of-line blocking is the *default* failure mode of any queueing system, and the fix is always the same: **more queues, keyed by the thing that can fail.**

WhatsApp applied that identical fix at four layers — one queue per destination node, so a slow node only backs up its own traffic; separate read and write queues, because the fast readers pile up behind slow writers; round-robin across async file-IO threads; and FIFO worker dispatch wherever latency is non-deterministic. Paired with it is the opposite discipline: a deliberate refusal to propagate backpressure, because if senders block, "procs that try to send start to get suspended by the scheduler, which causes a cascading failure where everyone is waiting and no work is getting done".

The service-level versions are circuit breakers (stop calling a failing dependency), bulkheads (a separate thread pool per dependency so one cannot exhaust the others), deadline propagation, and adaptive LIFO queueing.

See: Ch. 2 (WhatsApp, the queueing discipline) · Ch. 12 (Cloudflare, TCP Vegas as a throttle and priority queueing by query cost) · Ch. 14 (Lyft, all of it in a sidecar) · Ch. 6 (Airbnb, the 20-dependency page).

9. Chunking for parallelism

Find the boundary at which your work becomes independently processable, and parallelism is suddenly free.

For video, that boundary is the **closed group of pictures** — a chunk starting at a keyframe with no prediction references crossing its edge is independently decodable, so chunks are position-independent units of work that reassemble into a playable stream. The general form: *find the equivalent boundary in your workload* — an idempotent event, a shard key, a date range — and the serial problem becomes an embarrassingly parallel one.

See: Ch. 3 (YouTube, closed GOPs and multiple-output transcoding) · Ch. 8 (Spotify, track segments) · Ch. 4 (Uber, graph partitions).

10. Adaptive delivery — move the decision to the edge

Encode once into a ladder of qualities, cut each into segments, publish a manifest, and let the *client* choose. The manifest is the contract that lets a dumb HTTP cache serve an adaptive stream.

The trade-off is segment duration: shorter means lower live latency and faster reaction to bandwidth change, but more requests, worse compression (more keyframes per second), and larger manifests. Client logic splits into throughput-based (estimate future capacity) and buffer-based — and the best-known result here is that **in steady state, capacity estimation is unnecessary**; mapping buffer occupancy directly to a bitrate is enough, with throughput estimation needed only during startup.

See: Ch. 3 (YouTube, DASH/HLS) · Ch. 8 (Spotify) · Ch. 15 (Bluesky, HLS for short video) · Ch. 10 (Zoom, where SVC layers replace the ladder).

11. Move the work to the client

Sometimes the scalable server is the one that does almost nothing.

Zoom does not composite participant video streams server-side, because compositing is transcoding and transcoding does not scale per-call. Instead the server routes streams and the client decodes and lays them out. Layered encoding means a single stream carries all qualities and each client decodes only the layers it needs. WhatsApp's E2EE forces the same shape for a different reason — the server *cannot* read content, so it cannot transform it.

See: Ch. 10 (Zoom) · Ch. 2 (WhatsApp) · Ch. 14 (Lyft, clients build the road graph locally from downloaded cells).

12. Message passing and isolated state — the actor model

An alternative to shared-memory concurrency: many tiny isolated units, each with private state and a mailbox, each processing one message at a time in FIFO order, communicating only by immutable messages. Threads are assigned to actors only while a message is being processed, so a thread count proportional to CPU cores can serve millions of actors.

The deeper point is that **the concurrency model should match the workload's unit of concurrency**. "N mostly-idle long-lived connections, each holding a little state" maps one-to-one onto a per-process model — and WhatsApp's most effective single optimization was reducing from two processes per connection to one, tightening that mapping rather than loosening it.

See: Ch. 13 (PayPal, Akka) · Ch. 2 (WhatsApp, Erlang/BEAM) · Ch. 12 (Cloudflare, where per-connection OS processes are the *problem*).

13. Decentralization and federation

Instead of one authority, many operators running interoperating servers — the email model. Data lives in per-user repositories that are portable between hosts; a crawler aggregates a firehose from all of them; index servers build the derived views that a centralized system would have materialized directly.

The costs are precise and worth stating: every aggregate — follower counts, like counts — becomes a crawl-and-count problem rather than a column read, and moderation becomes a labeling layer rather than a deletion.

See: Ch. 15 (Bluesky, in full) · **Counter-case:** Ch. 1 (Twitter, the centralized design Bluesky was a reaction against).

14. Two-stage retrieval, then ranking

Modern feeds are not sorted, they are a funnel: cheap recall-oriented candidate generation narrows hundreds of millions of items to a fixed budget, then an expensive precision-oriented model ranks that budget.

Two details make it work. The **candidate budget is the API contract** between the stages. And candidates are **normalized free of their provenance** before ranking — "all candidates are treated equally, without regard for what candidate source it originated from" — which is what makes sources comparable to each other.

See: Ch. 1 (Twitter's For You pipeline) · Ch. 8 (Spotify, retrieval then bandit ranking) · Ch. 6 (Airbnb, embedding retrieval then ranking).

Two meta-patterns

Two ideas sit above the fourteen because they are about how you work rather than what you build.

INSTRUMENT THE RUNTIME, NOT JUST THE APPLICATION

When WhatsApp was pushing connection counts, profiling showed only **16% of CPU time was in the Erlang emulator loop**. Eliminating *all* application execution would have saved 16%. The real wins were a global time-of-day lock, a single timer wheel, the memory allocator's growth policy — and the decisive tool was recompiling the VM with lock counting enabled.

An earlier symptom makes the same point: at 425,000 connections, CPU showed 35–45% utilization while the schedulers showed 95%. **CPU percentage was a misleading health signal**. "Optimize the hot path" is bad advice when the hot path is 16% of the budget.

YOU CANNOT LOAD-TEST A SYSTEM WHOSE HARNESS COSTS MORE THAN THE SYSTEM

Testing one WhatsApp server at a million connections required **30 hosts** just to open enough ports; two million required 60. So they stopped trying, and instead made production observable and safely mutable: tee'd real traffic, hot code loading, runtime knobs, and metrics at one-second resolution because "events that happened in the space of a minute were invisible".

Twitter and Spotify say versions of the same thing — Twitter's Decider decoupled launches from deploys so changes could be enabled for a percentage of traffic; Spotify measured the value of prefetching only because a bug accidentally disabled it for a week. **The methodology is the reusable part**.

PRACTICE

1. Pick the system you work on. Which of the fourteen patterns does it use? Which one does it most obviously need and not have?
 2. For patterns 1 and 7, name the specific conditions under which you would choose each available option. Be numeric.
 3. Pattern 4's callout claims fan-out latency is a consistency property. Construct a second example, from a domain other than social feeds, where slow asynchronous propagation produces an observable causality violation.
-

PART TWO

Fifteen Case Studies

Each chapter takes one system, states the problem it had to solve, walks the architecture it arrived at, and then spends most of its length on the single hard part that makes it worth studying. The numbers carry dates. The claims carry sources.

1	Twitter / X Timeline
---	----------------------

2	WhatsApp
---	----------

3	YouTube
---	---------

4	Uber ETA
---	----------

5	Instagram
---	-----------

6	Airbnb
---	--------

7	Redis at Scale
---	----------------

8	Spotify
---	---------

9	Tinder
---	--------

10	Zoom
----	------

11	Figma
----	-------

12	Cloudflare
----	------------

13	PayPal
----	--------

14	Lyft
----	------

15	Bluesky
----	---------

Twitter / X Timeline

How do you show a user the posts of everyone they follow, 300,000 times a second — and why did the answer stop being the answer?

AT A GLANCE

Timeline reads vs writes	300K QPS vs ~6K req/s (2012)
Timeline deliveries	30 billion/day, ~300K/sec (2012)
Peak tweets in one second	143,199 (3 Aug 2013 JST)
Steady-state average then	>500M tweets/day $\approx$ 5,700 TPS (2013)
Entries cached per timeline	800 maximum (2012)
Posts to rank per request	Hundreds of millions $\rightarrow$ 1,500 candidates (2023)
Ranking pipeline	~5B executions/day, <1.5 s mean, 220 CPU-seconds each (2023)

This is the most-taught architecture in system design, and the version everyone teaches was switched off. That makes it the single most useful case study in this guide — not because the design was wrong, but because watching a correct design become obsolete teaches something no still-current design can.

1 • Requirements

Functional. A user opens the app and sees a list of posts from accounts they follow, newest-ish first. They scroll and more load. They can post, reply, like, and repost. Posts contain text and media.

Non-functional. The read must feel instant — a few hundred milliseconds, end-to-end. The write may be slower, but not so slow that a conversation becomes incoherent. The system must absorb a peak that is *tens of times* its average, because the product is coupled to live events it does not control.

The question that decides everything. What is the read/write ratio? In 2012 the answer was 300,000 timeline reads per second against roughly 6,000 writes.¹ Twitter's own framing: it is "primarily a consumption mechanism, not a production mechanism". That 50:1 asymmetry is the license for everything that follows.

2 • The two possible designs

	Fanout on read (pull)	Fanout on write (push)
On write	Store the post once. Done.	Look up every follower and append the post ID to each of their timelines.
On read	Find everyone this user follows, fetch their recent posts, merge, sort.	Read one list. Done.
Cost	$O(\text{followees})$ per read $\times$ 300K reads/sec	$O(\text{followers})$ per write $\times$ 6K writes/sec
Verdict	Tried and abandoned. Twitter's description: "naive materialization is a massive select statement over all of Twitter. It was tried and died." ¹	Chosen. Write amplification is acceptable when writes are 1/50th of reads.

3 • The write path, as built

A tweet arrives. The fanout pipeline resolves the author's followers from the Social Graph Service — backed by FlockDB on Gizzard-sharded MySQL — and pipelines inserts **4,000 destinations at a time**. A tweet from an account with 20,000 followers becomes roughly 20,000 Redis inserts.¹

Three decisions make this affordable, and all three are worth stealing:

- **Store references, not objects.** Each timeline entry is a tweet ID, the originating user ID, and **four bytes of bit flags** marking retweet, reply and so on. A retweet stores a pointer to the original. Twenty bytes per entry, not a kilobyte.
- **Bound the cache.** Each timeline holds at most **800 entries**. Page back far enough and you hit the wall — "RAM is the limiting resource". Crucially, a bounded cache is only safe because there is a *reconstruction path*: if your timeline falls out of Redis, the system queries the social graph, hits disk for every followee, and re-pushes. The cache is an optimization, never the system of record.
- **Only materialize active users.** "Active" meant logged in within 30 days, a threshold tunable against cache capacity. Inactive users get nothing written, which removes most of the write amplification for free.

The whole thing lived in a Redis cluster holding a couple of terabytes of RAM, each entry replicated three times on three different machines.

4 • The read path is where the design pays off

Reading is **$O(1)$** : find the one machine holding this user's timeline. Because entries are replicated three ways, the system is "effectively running three different hash rings" and returns from whichever replica answers fastest.¹

But the timeline is only IDs, so the assembled array must be **hydrated** — a parallel multi-get against two services, each with its own cache:

- **Tweetypie**, the tweet object service, with memcache holding roughly the last month and a half of tweets.
- **Gizmoduck**, the user service, with memcache holding the *entire* user base.

Compliance filtering happens at read time at the edge — jurisdiction-specific content stripping, for example — rather than being baked into the materialized timeline. That is a deliberate and reusable choice: *policy that can change belongs on the read path, not in the precomputed artefact.*

Measurement (2012)	Latency
Timeline Service, median	5 ms
Timeline Service, p99	100 ms
Timeline Service, p99.9	A few hundred ms — this is where it hits disk
API call, end to end	~400 ms
Cold cache to rendered browser	~2 s
Fanout to 1M followers, p50	3.5 s
Fanout, p99	Up to 5 minutes

5 • The hard part: the celebrity problem is a consistency bug

At the time of the talk, @ladygaga had 31 million followers, @katyperry 28 million, @justinbieber 28 million, @barackobama 23 million. The obvious problem is cost — that is a great many Redis inserts for one tweet. The obvious problem is not the real one.

THE REAL FAILURE MODE: REPLIES ARRIVING BEFORE ORIGINALS

Look again at the fanout latency table. p50 is 3.5 seconds; p99 is five minutes. That spread means **different followers receive the same tweet at very different times**. A user who received it early can reply within seconds — and that reply, fanned out to a smaller follower set, completes fast. For a follower still waiting on the original, the reply arrives first.

Twitter's own description: long fanout queues mean "replies are being seen all the time before the original tweets".¹ Sorting by monotonically increasing Snowflake IDs does not fix it, because the problem is not ordering *within* a timeline — it is skew in delivery time *across* a follower set.

Generalize this. Fan-out latency is a consistency property. Any system that propagates derived state asynchronously to many destinations can produce observable causality violations purely from queue delay, with no bug anywhere in the code.

The proposed fix was a hybrid: stop fanning out high-value accounts, and merge their posts in at read time instead. Twitter estimated it would save "tens of percents of computational resources".¹

FLAG: THE "10,000 FOLLOWER THRESHOLD" HAS NO SOURCE

Almost every treatment of this architecture states a specific follower count above which an account switches to fanout-on-read — usually 10,000 — and several attribute it to Twitter. Neither the original talk nor its widely-cited write-up contains any numeric threshold; the hybrid is described qualitatively ("for people like Taylor Swift") and, more importantly, **as future work rather than shipped state**.

If you use a threshold in an interview, present it as your own design parameter and justify the number. Do not attribute it.

6 • Search is the same problem, inverted

Worth studying alongside the timeline, because it shows the two strategies coexisting in one company. Twitter's real-time search index, **Earlybird**, is a heavily rewritten Lucene held entirely in RAM. An Ingestor tokenizes each incoming tweet and writes it to a *single* Earlybird machine.^{1,2}

	Timeline	Search
Write	O(followers) — expensive	O(1) — one machine
Read	O(1) — one machine	O(shards) — scatter-gather across all of them
Coordinator	Timeline Service	Blender — scatters, gathers, merges, re-ranks by social proof
Latency	5 ms median	~100 ms, and never touches disk

The Earlybird paper is worth reading for its index design, which is deliberately unlike a conventional retrieval engine: each server holds about a dozen index segments but only *one* ingests; postings in that active segment are left **uncompressed**; they are stored in forward chronological order but **traversed backwards** from an end pointer; and concurrency is single-writer/multiple-reader with targeted memory barriers.² Every one of those choices trades space or elegance for the ability to serve a query against data that arrived milliseconds ago.

7 • What actually made the 2013 record survivable

143,199 tweets in one second is roughly 25× the steady-state average, and Twitter's write-up notes users "didn't experience a blip".³ The reason was not capacity planning for that second. It was a rewrite finished years earlier.

After a bad 2010 World Cup, Twitter moved off Ruby on Rails onto the JVM. Rails hosts served **200–300 requests/sec**; the replacement JVM services served **10,000–20,000 requests/sec** per host, and

the machine count needed to serve the site fell by a factor of **5 to 12**.³ Four supporting pieces are named in the same post, and all four are general lessons:

- **Finagle and Futures** — one uniform concurrency, backpressure and connection-pooling model across services, which fixed thundering herds caused by aggressive client retries.
- **Decider** — runtime configuration with binary and percentage-based switches, so *launching a feature is decoupled from deploying code*. This is what makes incremental rollout possible at all.
- **Viz and Zipkin** — per-request distributed tracing, so a latency regression can be attributed to a service rather than guessed at.
- **Snowflake** — globally unique IDs, which existed because sharding tweets across MySQL removed auto-increment. The trade was explicit: give up "increment by one" to get distributed ID generation.

8 • And then the fanout service was deleted

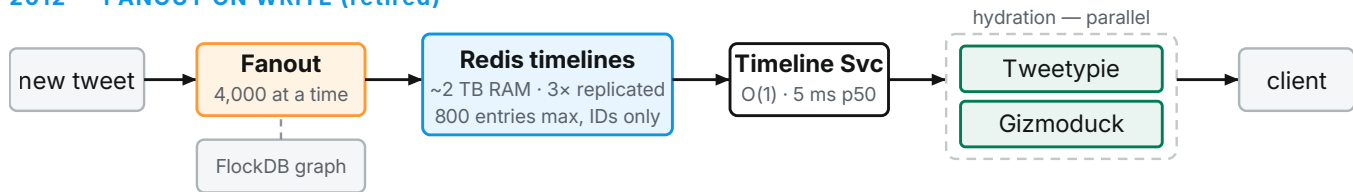
In 2023 Twitter published and open-sourced its recommendation pipeline. Buried in it is the sentence that should reframe this entire chapter:

STALE: FANOUT-ON-WRITE IS NOT HOW X BUILDS YOUR TIMELINE

"We recently stopped using **Fanout Service, a 12-year-old service** that was previously used to provide In-Network Tweets from a cache of Tweets for each user."⁴

In-network posts are now sourced from the **search index** — Earlybird — ranked by a logistic regression model. The read path won. Note what changed to cause it: not the read/write ratio, but the *requirement*. A reverse-chronological list of followed accounts can be precomputed. A personalized ranking over hundreds of millions of candidates cannot.

2012 — FANOUT ON WRITE (retired)

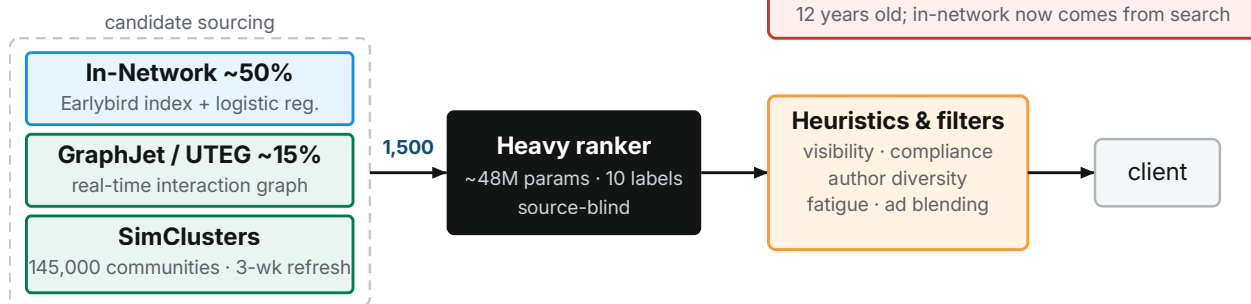


One tweet → O(followers) writes. One read → one machine.

Failure mode: p99 fanout up to 5 minutes → replies can arrive before originals.

2023 — RETRIEVE, THEN RANK

~500M posts/day, hundreds of millions in the pool



Runs ~5 billion times/day. 220 CPU-seconds per execution, delivered in under 1.5 s — roughly 150× parallelism.

The scarce resource is the reader's patience, not CPU. Orchestrated by Home Mixer; inference served by Navi, in Rust.

Figure 1.2 — The same product, ten years apart. Top: precompute the view, read it in O(1). Bottom: precompute nothing, retrieve candidates, rank them. The bottom architecture is not a refinement of the top one — it replaced it, and the component that did the precomputing was removed.

9 • The modern pipeline, briefly

Three stages, orchestrated by **Home Mixer** on a framework called Product Mixer.⁴

Candidate sourcing. "We attempt to extract the best 1,500 Tweets from a pool of hundreds of millions" — roughly half in-network, half out-of-network. In-network comes from Earlybird, with **Real Graph** (a model predicting engagement likelihood between two users) as the most important feature: a higher Real Graph score admits more of that author's posts. Out-of-network arrives two ways — **GraphJet**, a real-time user/post interaction graph answering "what did the people I follow recently engage with?" and serving about 15% of timeline posts; and **SimClusters**, a custom matrix-factorization community detection producing **145,000 communities, refreshed every three weeks**, where a post's community membership derives from its current popularity within each community.

Ranking. A single neural network of roughly **48 million parameters**, continuously trained on interactions, consuming thousands of features and emitting ten labels — each the probability of a distinct engagement. The line that matters architecturally: "all candidates are treated equally, without regard for what candidate source it originated from." Provenance is stripped before ranking, which is what makes the sources comparable.

Heuristics and filters. Visibility filtering for legal compliance, author diversity, content balance, feedback-based fatigue, then blending with ads and follow recommendations.

And the number to remember: the pipeline runs about **5 billion times per day** and completes in **under 1.5 seconds on average**, while a single execution consumes **220 seconds of CPU time** — roughly 150× the latency the user perceives. That is what buying latency with parallelism looks like at the top end.

DON'T MIX THE STRATA

There are three distinct eras of Twitter disclosure and they are a decade apart: the 2012–2014 fanout era, a 2017 infrastructure inventory, and the 2023 recommendation release. Blending numbers across them is the most common error in teaching material. Note in particular that "500 million tweets per day" appears in both the 2013 and 2023 posts and almost certainly counts different things.

Two more conflations to avoid. **Nighthawk** is Twitter's general sharded-Redis cache; the tweet-timeline cache specifically is **Haplo**, a customized Redis implementing a HybridList type, written by both Timeline Service and Fanout Service and read only by Timeline Service.⁵ And **Manhattan** — the eventually consistent, multi-tenant, schemaless key-value store with opt-in strong consistency — backs tweets, DMs and accounts; it did not store timelines, and it launched in 2014, *after* the timeline architecture described above.

10 • A note on the client, because it is half the latency

The original source for this case is actually a *front-end* deep dive, and it is worth not skipping: a timeline client is a distributed system in its own right.⁶

The rendering choice is client-side, in a single-page application. Server-side rendering would help SEO, but timelines are personalized so there is nothing to index; and fetching a timeline already takes under a second, so SSR's cost in infrastructure and complexity is not repaid. The SPA choice pays off on navigation: tapping a post navigates instantly because its text and media are already in the client store, and only replies need fetching.

Four optimizations carry beyond this product:

- **Cursor-based pagination**, not offset. Offsets shift when new posts are inserted, producing duplicates and gaps, and large offsets force the database to skip an increasing number of rows. A cursor — "the next 10 after post ID X" — is stable under insertion and supports bidirectional loading.
- **List virtualization**. Render only what is in the viewport, backed by empty spacer elements so the scrollbar stays honest. Off-screen rows are recycled.
- **Optimistic updates**. Fill the heart and increment the count immediately; reconcile with the server after; roll back on failure. This is only tractable because the store is normalized — there is exactly one copy of the post to update, not one per place it appears.
- **Code splitting by relevance, interaction and navigation**. Load only the code for the post formats actually present in the timeline; defer the emoji picker until someone opens it; defer the detail page until someone taps through.

11 • Trade-offs

The design bought	And paid
O(1) timeline reads at 300K QPS	O(followers) write amplification, and a fanout p99 of five minutes
A few TB of RAM holding every active user's timeline	A hard 800-entry limit, and a slow disk-backed reconstruction path when a timeline is evicted
Cheap storage per entry (~20 bytes)	Every read requires hydration against two more services
Reverse-chronological correctness	No ability to rank — which is precisely why the design was eventually abandoned
Ranking over hundreds of millions of candidates (2023)	220 CPU-seconds per timeline view, and a pipeline nobody can reason about end to end

WHAT TRANSFERS

1. **Quantify the asymmetry before choosing a strategy.** 300K reads against 6K writes is what licensed O(followers) work per post. The lesson is not "fan out on write" — it is "measure the ratio, then put the work on the cheap side". When the requirement changed, the same reasoning produced the opposite answer.
2. **Store references; hydrate in parallel.** IDs plus flags, then multi-get against independently cached services. This is what makes a materialized view fit in memory.
3. **A bounded cache needs a reconstruction path.** 800 entries is safe only because the durable stores can rebuild a timeline from scratch. Without that, the cache silently becomes the system of record.
4. **Push and pull are duals, and the choice can be per-entity.** Twitter ran O(n)-write/O(1)-read for timelines and the exact inverse for search, simultaneously.
5. **Fan-out latency is a consistency property.** High-cardinality outliers do not merely cost more — they produce causality violations that look like bugs and are not.
6. **Two-stage retrieval with a hard candidate budget.** The budget is the contract between stages, and stripping provenance before ranking is what makes heterogeneous sources comparable.
7. **Decouple launching from deploying.** Decider is why a 25× traffic spike was survivable: the capability to change behavior at runtime, built years before it was needed.

PRACTICE

1. Design the hybrid Twitter described but did not ship. Where does the read-time merge happen? How does the client get a correctly ordered result when some posts came from cache and some from a live query? What is your threshold and why?
2. You must fix the replies-before-originals problem without reducing fanout latency. Two approaches: change what is delivered, or change what is displayed. Work out both.

3. The 800-entry limit means deep pagination falls off the cache. Design the reconstruction path, and state its cost in disk reads for a user following 2,000 accounts.
 4. Argue the case for keeping fanout-on-write in 2023 — what would have to be true about the product for the old design to still be correct?
 5. A single timeline view costs 220 CPU-seconds. Estimate the daily CPU-hours across 5 billion executions, and then argue whether that is a reasonable amount of computation to spend on showing someone a list of posts.
-

SOURCES

1. Raffi Krikorian, "Timelines at Scale", QCon / InfoQ, 2012 — infoq.com/presentations/Twitter-Timeline-Scalability; detailed gloss at highscalability.com/the-architecture-twitter-uses-to-deal-with-150m-active-users
2. Busch, Gade, Larson, Lok, Luckenbill & Lin, "Earlybird: Real-Time Search at Twitter", IEEE ICDE 2012
3. "New Tweets per second record, and how!", Twitter Engineering Blog, Aug 2013
4. "Twitter's Recommendation Algorithm", X Engineering Blog, Mar 2023; github.com/twitter/the-algorithm
5. "The Infrastructure Behind Twitter: Scale", Twitter Engineering Blog, 2017
6. Neo Kim & Yangshun Tay, "System Design Interview: Design Twitter/X Timeline — A Frontend Deep Dive", The System Design Newsletter #103, Nov 2025 (partly paywalled; front-end material above is from the free portion)

WhatsApp

A hundred billion messages a day, and almost no storage. What do you gain by refusing to keep anything?

AT A GLANCE

Messages per day	~100 billion (Oct 2020, Meta earnings call)
Users	2 billion (Feb 2020)
Peak concurrent connections	147 million (Mar 2014)
Peak messages/sec	342K inbound, 712K outbound (Mar 2014)
Connections per server	~1 million in production; 2.8M proved possible (2012–14)
Fleet	~550 servers, >11,000 cores (Mar 2014)
Engineers	32 total at 450M users; ~10 on the Erlang team (Feb–Mar 2014)

Every other system in this guide stores what it receives. WhatsApp deletes it. That single decision — messages live on the server only until delivered — cascades into an architecture that is genuinely, structurally simpler than its scale suggests, and it is the most transferable idea in the chapter.

READ THIS BEFORE THE CHAPTER

Essentially all of WhatsApp's *architecture* disclosure comes from two Erlang Factory talks, in 2012 and 2014.^{1,2} After the Meta acquisition, public engineering writing narrowed to security and cryptography. There is no modern equivalent of those talks.

So: FreeBSD 9.2, Erlang R16B01, Mnesia, Yaws, 550 servers on SoftLayer bare metal are all **2014 facts** about a company that has since grown roughly 4× in users and spent a decade inside someone else's infrastructure. Treat the architecture below as a superb worked example, not as a description of production today. The absence of newer disclosure is itself a lesson: acquisition tends to end architectural transparency.

1 • Requirements

Functional. Send and receive text in one-to-one chats. Group chats. Media attachments. Message status — sent, delivered, read. Online/offline presence and "last seen". And the requirement most people skip: **messages sent to an offline user must be queued and delivered later.**

Non-functional. Sub-500 ms delivery when both parties are online. Guaranteed delivery — messages cannot silently vanish. Billions of users. Resilience to component failure. And, explicitly: **do not store messages on servers longer than necessary.**³

That last line is not a privacy footnote. It is the load-bearing architectural constraint, and everything downstream follows from it.

2 • Back of the envelope

Users and traffic

```
1B registered, 500M DAU, 50M concurrent at peak
500M x 20 messages/day      = 10B messages/day
10B / 86,400                = ~115,000 msg/sec average
x 3-5 peak                  = 350,000-500,000 msg/sec peak
```

Storage – the interesting calculation

```
1 KB/message x 10B/day      = 10 TB/day
x 365                      = 3.6 PB/year    <-- if retained
```

```
But: deleted on delivery. Only undelivered messages persist
(30-day window), plus ~10% of users keeping history.
      = 400-500 TB active    <-- an order of magnitude less
```

Bandwidth

```
50M concurrent x ~10 KB/s   = ~500 GB/s at peak
```

The storage line is the whole chapter in four numbers. A retention policy turned petabytes into hundreds of terabytes, and in doing so removed compaction, replication factor, tiering, and archival from the design entirely. The source states the consequence directly: because media and text are transient, "not having to store all these assets for the long term simplifies the system greatly. The architecture can revolve around throughput, caching, and partitioning."²

3 • Architecture

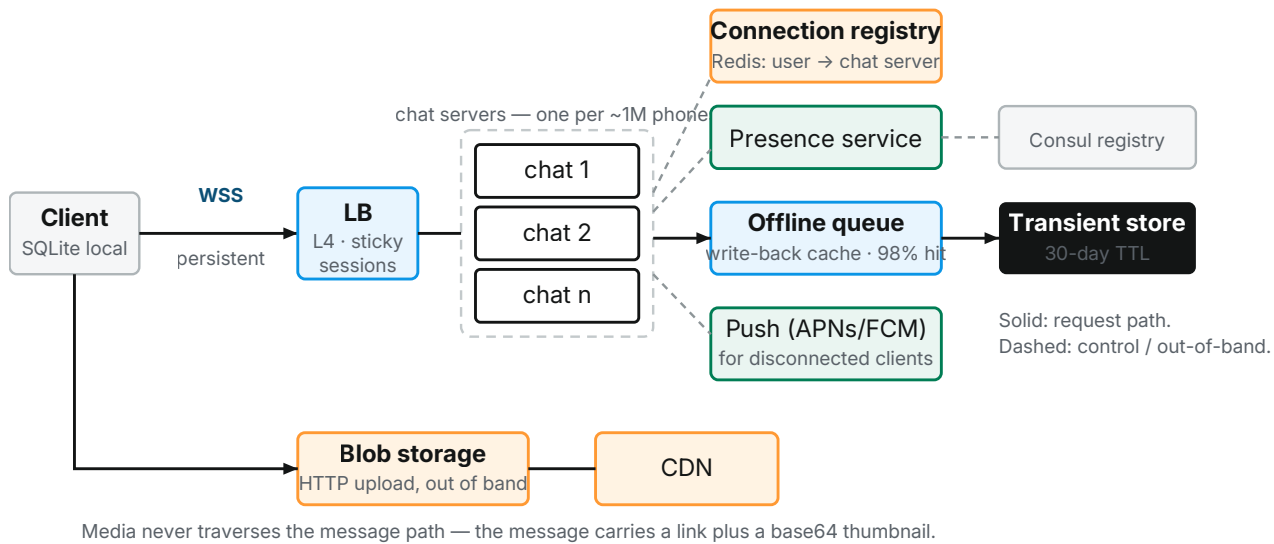


Figure 2.1 — The shape is conventional; the labels are not. The durable store at the right has a 30-day TTL and holds only *undelivered* messages. There is no message history database, no search index over content, and no derived view to materialize — because the server cannot read the content and does not keep it.

Chat servers hold the persistent WebSocket connections and route messages. The **connection registry** — Redis — records which chat server each online user is attached to, so routing is a microsecond lookup rather than a millisecond query. The **message queue** decouples acknowledgment from persistence: the chat server acknowledges to the sender immediately, then pushes asynchronously for storage and delivery. **Media never enters this path**: it is uploaded over HTTP to blob storage, and the message carries a link plus a base64-encoded thumbnail.³

Why WebSockets, stated properly

Option	Why it fails
Polling	A message arriving just after a poll waits a full interval. Multiply millions of clients asking "anything new?" against mostly-empty answers and you are spending the entire budget on the question.
Long polling	Better, but every message costs a full HTTP handshake. At scale the servers spend more time managing connection lifecycles than delivering messages, and idle HTTP connections still hold memory and thread-pool slots without WebSocket's efficiency.
WebSocket	Chosen. One persistent bidirectional connection; handshake cost paid once at app open. Price: L4 rather than L7 load balancing , and more memory per connection because connections stay open.

Idle connections are managed with heartbeats — a lightweight ping every 30 seconds, and the server closes a connection it has not heard from in 60. Without that, zombie connections accumulate and consume memory forever.

4 • The first hard part: holding a million sockets on one box

This is the best-documented performance investigation in this guide, and worth studying as method rather than as a list of fixes. The starting point was **200 simultaneous connections per server**. The target was a million.¹

Milestone	What was observed
425K connections	"35–45% CPU was being used across the system but the schedulers are at 95% utilization." The schedulers were spinning on sleeping locks. CPU percentage was a misleading health signal.
>1M	After the first round of lock fixes. VM at 76%, CPU 73%, emulator at 45% — now matching user time.
2M	A month later. Emulator ~80%, near the point where FreeBSD would start paging.
2.8M	Peak achieved, at 571K packets/sec and >200K distribution messages/sec. 3 million was attempted and failed.
~1M	2014: deliberately reverted. Three stated reasons — more headroom for peaks, users are busier per connection than two years earlier, and functionality from elsewhere was moved onto these servers. ²

THE LESSON IN THAT LAST ROW

The famous number — two million connections on one server — is what they *proved possible*, not what they ran. Having proved it, they chose to run at half of it, because headroom was worth more than density.

Twitter's version of the same lesson appears one chapter earlier: the celebrated fanout architecture was eventually deleted. **In both cases the widely-quoted number is the one the company later walked away from.**

Where the wins actually came from

Profiling with `pmcstat` produced the finding that redirected the entire effort: **only 16% of CPU time was in the Erlang emulator loop**. Eliminating *all* application execution would have saved 16%. So the work moved into the runtime and the kernel:¹

Layer	Fix
Application	"Originally had two Erlang processes per connection. Cut that to one. " Best return of any single change on the list.

Layer	Fix
Erlang VM (BEAM)	The time-of-day lock — every port message delivery updated one global time value behind a single lock shared by all schedulers. Multiple timer wheels instead of one wheel with one lock (millions of connections × set/reset per event is hundreds of thousands of timer operations per second). The check-IO time table switched from arithmetic to geometric growth , because reallocation was thrashing. <code>mseg</code> allocation made per-scheduler . A write-file variant taking an already-open port, to cut port contention.
Kernel (FreeBSD)	Raised file and socket limits. Enlarged the network-stack PCB hash table — profiling showed heavy time in PCB lookup. Backported the TSE time counter from FreeBSD 9 for a cheaper time-of-day read, and the <code>igp</code> NIC driver for multi-queue lockups.
Scheduling	Lowered the scheduler wake-up threshold (schedulers were sleeping and not waking). Preferred <code>mseg</code> over <code>malloc</code> . Started carrier sizes large and grew them, to induce FreeBSD superpages and cut TLB thrash. Ran BEAM at real-time priority so a cron job could not preempt it and create a user-traffic backlog.
Garbage collection	"When message queue backlogs became large garbage collection would destabilize the system. So pause GC until the queues shrunk ." An unusual and instructive inversion — GC deferred in favor of drain.

Two tooling decisions made all of it possible. First, they **compiled the emulator with lock counting enabled** — described as the single biggest diagnostic win. Second, they built their own instrumentation: `wsar`, a system activity reporter covering OS, hardware and BEAM statistics, initially at one-minute resolution and forced down to **one second** because "events that happened in the space of a minute were invisible". They patched BEAM to expose per-process message-queue statistics, because doing it in Erlang across a million connections is itself too slow, and kept those stats at three decay intervals — 1 s, 10 s, 100 s.

AND THEY COULD NOT LOAD-TEST IT

"If supporting a million connections on a server it would take **30 hosts** to open enough IP ports to generate enough connections to test just one server. For two million... **60 hosts**."¹

So they stopped trying to simulate production and made production safe to experiment in: tee'd real traffic to a separate system, DNS-based over-subscription (listing a server multiple times to give it 2–3× load — unreliable, because clients ignore TTLs), and `ipfw` forwarding to dial in an exact connection count. Combined with hot code loading, that gave a compile-load-observe loop under genuine production load.

This is the reusable part. When the test harness costs more than the system, invest in observability and safe mutability instead of fidelity.

5 • Queueing discipline, or: head-of-line blocking four ways

Every queueing system's default failure is that one slow consumer stalls everything behind it. WhatsApp applied the same fix — **more queues, keyed by the thing that can fail** — at four separate layers:²

- **Per-destination queues.** "When sending to different nodes the messages are given to different procs so *only* messages destined for a problem node are backed up."
- **Separate read and write queues,** because the read path is faster and any write latency piles readers up behind it.
- **Round-robin across async file-IO worker threads,** which "smoothed writes in the cases where there was a large mailbox or a slow disk".
- **FIFO dispatch wherever latency is non-deterministic** — that is, anywhere touching IO.

Paired with it is the opposite discipline: a deliberate refusal to propagate backpressure. Sends use `nosuspend` for best-effort delivery, because otherwise "distribution buffers back up on the sending node and **procs that try to send start to get suspended by the scheduler, which causes a cascading failure where everyone is waiting and no work is getting done**". Calls use timeouts rather than monitors, to reduce far-end contention.

Work

distribution went through three generations, and the third is the interesting one: a single-threaded `gen_server`, then `gen_factory` spreading work across many workers, then `gen_industry` — *multiple dispatch processes*, because at high fan-in the single dispatcher's own process locks became the bottleneck independently of how fast it executed.

The offline queue, and a write-back cache

Offline storage was "a big bottleneck under load spikes — just couldn't push stuff to the file system fast enough". The fix exploits a measured property of the workload: **roughly 50% of messages are read within 60 seconds**. So a write-back cache lets a message be *delivered before it is written to the filesystem* at all. Result: a **98% cache hit rate**, plus a buffer for when the IO system is backed up.²

And then the anti-pattern fix worth remembering: "Some people are in a large number of groups and get thousands of messages per hour. They polluted the cache and slowed things down. **Evict them from the cache.**" A small number of extreme users can destroy a cache designed for the median — and the correct answer is sometimes to exclude them explicitly rather than to size for them.

6 • The second hard part: encryption relocates the fanout

End-to-end encryption is usually taught as a privacy feature. Architecturally it is a constraint on *where work can happen*, and the consequences are precise.

WhatsApp uses the Signal Protocol.⁴ Three key types are published at install: a long-term **Identity Key Pair**, a medium-term **Signed Pre Key** signed by the identity key, and a queue of single-use **One-Time Pre Keys**, replenished as consumed. Session state is a **Root Key** (32 bytes), **Chain Keys** (32 bytes),

and per-message **Message Keys** of exactly 80 bytes: 32 for AES-256, 32 for HMAC-SHA256, 16 for the IV.

The ratchet is two mechanisms composed — a hash ratchet within a chain and a Diffie-Hellman ratchet across round trips:

```
Message Key = HMAC-SHA256(Chain Key, 0x01)
Chain Key   = HMAC-SHA256(Chain Key, 0x02)      # advance

ephemeral_secret = ECDH(Ephemeral_sender, Ephemeral_recipient)
Chain Key, Root Key = HKDF(Root Key, ephemeral_secret)
```

THE PROPERTY THAT MAKES THE WHOLE ARCHITECTURE POSSIBLE

From the whitepaper: "Because of the way Message Keys and Chain Keys are calculated, **messages can arrive delayed, out of order, or can be lost entirely without any problems.**"⁴

That is exactly what licenses the server to be a dumb store-and-forward buffer with no ordering guarantee. The cryptography absorbs the disorder, so the transport does not have to provide order — and providing order at this scale is expensive. **A correctness property in one layer bought a simplification in another.**

Where the fan-out happens, and what it costs

This is the detail most explanations get wrong, so it is worth stating carefully.

	One-to-one chat	Group chat
Mechanism	Pairwise sessions, one per device	Sender Keys
Who fans out	The client. It "encrypts and transmits it N number of times to N number of different devices" — every device in both the sender's and the recipient's device lists.	The server. The sender encrypts once, signs the ciphertext, and sends a single copy "to the server, which does server-side fan-out to all group participants".
Per-message cost	O(devices) client-side encryptions	O(1) client-side encryption
Setup cost	One X3DH exchange per device pair	O(members × devices) pairwise encryptions to distribute the Sender Key
On membership change	—	Full rotation. "Whenever a group member leaves, all group participants clear their Sender Key and start over."

That last row is the real cost model of group messaging. Paying O(members × devices) every time someone leaves a group is what makes very large groups expensive — and it is a plausible reason why group size is capped. **But note carefully:** WhatsApp has never published that as the reason. The limit is 1,024 participants, raised from 512 in late 2022, and it is documented only in consumer help pages and

tech press. The cost argument is sound engineering reasoning; presenting it as WhatsApp's stated rationale would be invention.

THE ARCHITECTURAL CONSEQUENCE NOBODY STATES

Because the server cannot read message content, WhatsApp **cannot build a "For You" timeline**. No ranking, no content-aware fanout, no deduplication, no server-side search over messages, no ML on content. Chapter 1's entire 2023 architecture is unavailable here — not for lack of engineering, but because the encryption model forbids it.

Put the two chapters side by side and you have the cleanest illustration in this guide that **architecture is downstream of product commitments**, not the reverse.

What multi-device changed (2021)

The original model made the phone the source of truth and the only device able to encrypt for another user; companions held a secure connection *to the phone* and mirrored it, which meant they were slow, frequently disconnected, and limited to one at a time.⁵

The redesign gave **each device its own identity key**, with the WhatsApp server maintaining a mapping from account to device identities. That mapping is security-critical — a compromised server could add a device — so two mitigations were added: security codes now represent the combination of *all* of a user's device identities, and Automatic Device Verification means a contact re-verifies only when someone re-registers their whole account rather than on every new link.

The genuinely novel piece is **app-state sync**. The server stores a copy of each application state (contact names, archive and mute flags, starred messages) that all of a user's devices can read — and "all the information, and even the metadata about the information (what kind of user data is stored or accessed), is end-to-end encrypted with constantly changing keys known only to that person's devices."

7 • Two failure stories worth studying

The hash chain bug

The account table had 512 Mnesia fragments, sparsely populated. Doubling the number of hosts *reduced* throughput, because the hashing scheme produced many empty buckets and a few extremely long chains — **chain sizes over 2,000 against a target of 7**. "A **two line change** improved performance by 4 to 1."² A related patch: "seed ets hash to break coincidence with `phash2`" — the classic double-hashing collision, where two layers using the same hash function collapse each other's distribution.

The 210-minute outage, February 2014

Not load-related, which is the point. A backend router **dropped a VLAN**, causing mass node disconnection and reconnection. On reconnect the cluster entered a state never seen before and had to be fully stopped and restarted — something they "hadn't done in years". Root cause: `pg2` **can do n**³

messaging under a disconnect/reconnect storm, with "pg2 message queues going from zero to 4 million within seconds".²

Compare this with Twitter, whose worst incidents came from traffic. WhatsApp's came from a **partition-heal storm** in a tightly coupled membership protocol. The general lesson: *design and test the reconnect path, not just the steady state*. Stateful clustering fails on recovery, not on load.

8 • Trade-offs

The design bought	And paid
Storage measured in hundreds of terabytes rather than petabytes	No server-side history, no cross-device search over old messages without a client-side sync mechanism
A server that cannot leak message content	No ranking, no content-aware features, no server-side moderation of message bodies
A million-plus connections per box	A patched OS and a patched language runtime, permanently. This is a real maintenance liability, not a one-off.
Sub-500 ms delivery via persistent sockets	L4 load balancing only, and memory held per idle connection
$O(1)$ per-message cost in groups, via sender keys	$O(\text{members} \times \text{devices})$ rotation on every departure — the practical ceiling on group size
A very small team	Achieved largely by <i>refusing features</i> — no ads, no feed, no social graph, no server-side history. Headcount efficiency was a product-scoping outcome.

WHAT TRANSFERS

1. **Choosing not to store is a scalability strategy.** Ask "what is the retention requirement?" before "what database?". The answer can remove an entire dimension of the problem.
2. **Match the concurrency model to the workload's unit of concurrency.** N mostly-idle connections each holding a little state maps onto a per-process model — and the best optimization tightened that mapping (two processes to one), not loosened it.
3. **At the limit, everything is lock contention — so instrument the runtime.** "Optimize the hot path" is wrong advice when the hot path is 16% of the budget. And watch for misleading health signals: 45% CPU alongside 95% scheduler utilization.
4. **Head-of-line blocking is the default; more queues is the fix.** Key them by the thing that can fail. Then decide deliberately whether to propagate backpressure or drop it.
5. **Exploit measured workload properties.** "50% of messages are read within 60 seconds" is what justified a write-back cache that delivers before persisting, at a 98% hit rate.
6. **Encryption relocates the fan-out, and each location has a cost model.** Client fanout for pairwise chats; sender keys plus server fanout for groups; rotation cost on membership change.
7. **Test the reconnect path.** Tightly coupled membership protocols fail when a partition heals, not when load rises.
8. **Evict your outliers.** A few extreme users can destroy a cache sized for the median, and excluding them explicitly is a legitimate answer.

FOUR CLAIMS TO STOP REPEATING

"WhatsApp handles 70 million messages per second." Misquoted. The figure is >70 million *Erlang* messages per second — internal VM message passing between processes. User-message peak from the same talk is 342K in / 712K out per second.

"WhatsApp uses client-side fanout." True for one-to-one, false for groups, where the whitepaper explicitly says the server does the fan-out.

"WhatsApp stores no data." Overstated. Undelivered messages are held; multimedia metadata occupied roughly 2 TB of in-memory Mnesia across ~18 billion records in 2014; prekey bundles and encrypted app-state blobs are server-stored. The accurate claim is narrower: *message content is not retained after delivery.*

"WhatsApp is XMPP / ejabberd." It *started* from ejabberd and then "switched from XMPP to an internally developed protocol". "XMPP-derived, ejabberd-descended, since replaced" is the defensible phrasing.

PRACTICE

1. Design the delivery-receipt mechanism. A message must show sent, delivered, and read, correctly, for a recipient with four devices, when messages may arrive out of order. What does "delivered" mean when one device has it and three do not?

2. You are told to add server-side full-text search over message history. Enumerate everything about the architecture that must change, and state whether the product promise survives.
 3. Groups are capped at 1,024. Work out the cost of a member leaving a full group where each member has three devices. Now propose a design that reduces that cost, and name what it gives up.
 4. Your service holds 500,000 WebSocket connections per host and CPU sits at 40% while latency degrades. List, in order, the five things you would measure.
 5. Argue against the store-and-forward model. What product would be impossible to build on it, and what would you do instead?
-

SOURCES

1. Rick Reed, "Scaling to Millions of Simultaneous Connections", Erlang Factory SF, 2012 — slides at erlang-factory.com; gloss at highscalability.com/the-whatsapp-architecture-facebook-bought-for-19-billion
2. Rick Reed, "That's 'Billion' with a 'B': Scaling to the Next Level at WhatsApp", Erlang Factory, 2014; gloss at highscalability.com/how-whatsapp-grew-to-nearly-500-million-users-11000-cores-an
3. Neo Kim & Hayk Simonyan, "System Design Interview: Design WhatsApp", The System Design Newsletter #106, Dec 2025 (partly paywalled)
4. *WhatsApp Encryption Overview*, technical white paper v7, 27 Sep 2023 — whatsapp.com/security. Protocol primaries: signal.org/docs/specifications/x3dh/, [doublerratchet/](https://signal.org/docs/specifications/doublerratchet/), [sesame](https://signal.org/docs/specifications/sesame/)
5. "Facebook Engineering: WhatsApp multi-device", engineering.fb.com, Jul 2021
6. "WhatsApp is now delivering roughly 100 billion messages a day", TechCrunch, Oct 2020, reporting Meta's Q3 2020 earnings call

YouTube

One upload becomes dozens of files. Find the boundary where that work becomes parallel, and the problem dissolves.

AT A GLANCE

Upload rate	"Multiple hundreds of hours of video every minute" (Google, ASPLOS 2021)
Watch time	1 billion hours/day (Feb 2017 — the most recent official figure)
Live streams	"Hundreds of thousands of concurrent streams" (2021)
Video share of internet traffic	>60% global; YouTube >15% (2019, via ASPLOS 2021)
Latency envelope	Live 100 ms; upload processing minutes to hours (2021)
Edge cache hit rate	70–90% of cacheable traffic from Google Global Cache (current, undated)
Transcoding ASIC gain	4.4–7.0× perf/TCO on H.264; 20.8–33.3× on VP9 (2021)

Broadcast television encodes video once, in one format, because every playback device supports that format. A video platform cannot. That single asymmetry generates the entire architecture.

1 • Requirements

The clarifying questions and their consequences, which Chapter 3 of Part I used as its worked example:¹

Constraint	Consequence
1M uploads/day, 100M DAU	Sharding and caching strategy for metadata
~100 views per upload	Overwhelmingly read-heavy; optimize the streaming path
Files up to 256 GB	Multipart upload with chunk-level retry and resume
240p to 4K, several container formats	A transcoding pipeline producing many renditions per input
First frame in under 500 ms	CDN plus adaptive bitrate streaming
10–30 minutes acceptable for processing	Transcoding is fully asynchronous — a queue, not a request

Constraint	Consequence
Eventual consistency acceptable for new uploads	Availability over immediacy, decided explicitly

2 • The API, and one decision that shapes the whole upload path

The design philosophy is stated crisply in the source: **application servers handle lightweight metadata; heavy video data bypasses them entirely.**¹

Endpoint	What it does
<code>POST /v1/videos</code>	Reserves a video ID and returns a pre-signed URL . The client then uploads directly to blob storage. The application server is almost untouched; bandwidth never passes through it.
<code>GET /v1/videos/{id}</code>	Returns metadata plus the manifest URL . The client uses the manifest to begin adaptive streaming from the CDN. The API server never serves a byte of video.
<code>POST /v1/progress/{id}</code>	Watch position. Millions of writes per second, so it prioritizes speed over consistency — fire the request, don't wait for confirmation, store in a high-throughput key-value store.
<code>GET /v1/search</code>	Cursor-based pagination, which avoids the "skip ten million rows" problem that offsets create.

The pre-signed URL pattern is worth internalizing as a general move: *when the payload is large and the logic is small, give the client a capability token and let it talk to storage directly.* The same idea appears in Chapter 8, where Spotify's API server generates signed URLs and the client streams audio straight from blob storage.

3 • The hard part: why one upload becomes dozens of files

Google's own framing of the problem, from the ASPLOS 2021 paper on their transcoding hardware — the best primary source on this subject in existence:²

- **Resolutions.** Each upload is converted into a standard group of 16:9 resolutions: 256×144, 426×240, and upward through 3840×2160 (4K) and 7680×4320 (8K).
- **Formats.** Laptops decode in software and track new codec specifications; TVs and phones use fixed-function decoders and stay on older ones. So "to leverage new specifications when the viewer's device supports it and use older ones when the device does not, videos must be encoded in a plethora of different formats."
- **The consequence.** Resolutions × formats × audio tracks means "a majority of work in the video processing platform [is] spent on transcoding." This "production of multiple outputs per input is a key difference between a video sharing service and a consumer video application like video chat."

Closed GOPs: the boundary that makes it parallel

"Transcoders can also shard the video into chunks (also known as closed Groups of Pictures, or GOPs) that can each be processed in parallel."²

The closed-GOP boundary is *why* the parallelism is legal. A chunk beginning at a keyframe, with no inter-frame prediction crossing its edge, is independently decodable. That makes it a position-independent unit of work: dispatch it to any machine, and the results reassemble into a playable stream with no cross-chunk state.

THE GENERALIZATION

Find the boundary at which your work becomes independently processable, and parallelism is free. For video it is a keyframe. For an event pipeline it is an idempotent record. For a database it is a shard key. For a batch job it is a date partition. **Most "this doesn't parallelize" claims are really "I haven't found the boundary yet".**

SOT versus MOT — the distinction that drives everything

	Single-Output Transcoding	Multiple-Output Transcoding
Mechanism	Read chunk, decode, scale, encode <i>one</i> output. A separate task per resolution and format.	Read and decode the chunk once , then downscale and encode all output variants in parallel from the shared raw frames.
Redundant work	The decode is repeated for every output	None — and control parameters derived from source analysis (fade and flash detection) are shared across outputs too
Measured throughput	Baseline	1.2–1.3× SOT
When to use	Only when memory or latency constraints force it	"Generally preferred to SOT, as it avoids redundant decodes for the same group of outputs"

There is a lovely piece of arithmetic hidden here. The pixel cost of the lower rungs is a geometric series: 720p + 480p + 360p + 240p + 144p sums to roughly 1.7 megapixels, against 1080p's ~2 megapixels.² So **the marginal cost of producing the entire lower ladder is about the same as producing one more top rung**. Once you are decoding anyway, the cheap qualities are nearly free — which is why every video platform ships a full ladder rather than two or three options.

The encode DAG

"Processing begins by identifying which output variants a given video needs based on its characteristics and the application. Based on the required output variants, an acyclic task dependency graph is generated to capture the work to be performed. The graph is placed into a global work queue system, where each operation is a variable-sized 'step' that is scheduled on machines in the data center."²

The step scheduler distributes load and absorbs both performance variation and infrastructure failure; a separate layer orchestrates chunking, dispatch to parallel workers, and reassembly. Videos are normally processed geographically near the uploader, with global spillover when local capacity runs short. Scheduling runs on Borg.

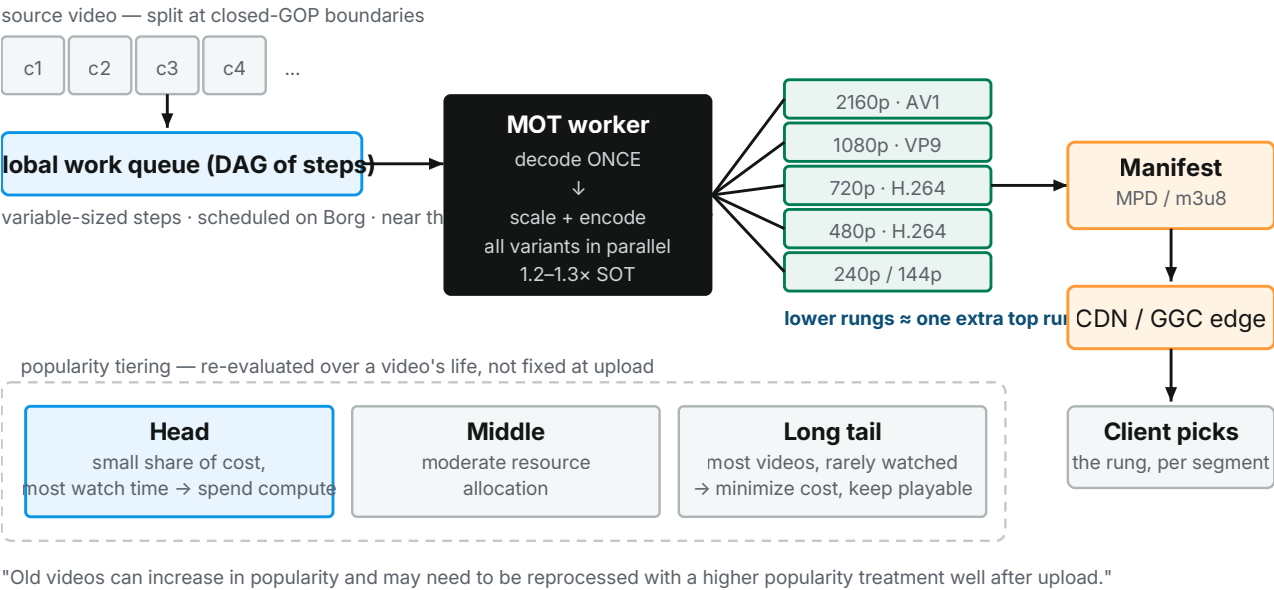


Figure 3.1 — Chunk at keyframes, decode once per chunk, encode the whole ladder from the shared frames, publish a manifest, let the client choose. The popularity tiering at the bottom governs both how much transcoding a video earns and whether the edge caches it — one policy, two subsystems.

4 • When software is not enough: the Argos VCU

By 2019 video was over 60% of internet traffic and growing with 4K, 8K and cloud gaming — while single-thread performance had stopped improving. Prior video ASICs targeted consumer devices; almost nothing targeted the data center. Google built one.²

System	H.264 Mpix/s	VP9 Mpix/s	Perf/TCO H.264	Perf/TCO VP9
2-socket Skylake (baseline)	714	154	1.0×	1.0×
4 × Nvidia T4	2,484	—	1.5×	—
8 × VCU	5,973	6,122	4.4×	20.8×
20 × VCU	14,932	15,306	7.0×	33.3×

Offline two-pass single-output transcoding, normalized to the CPU baseline. Perf-per-watt reached 6.7× for single-output H.264 and **68.9× for multiple-output VP9**.

The hardware is ten encoder cores per ASIC, two ASICs per full-length PCIe card, and ten cards — twenty VCUs — per production host. But the interesting part is *why it looks the way it does*, because the shape was dictated by the distributed system rather than by the codec:

- **Stateless, interchangeable cores.** All encoder state — reference frames, temporal motion vectors, probability tables — is read from and written back to DRAM, so firmware can dispatch any block of work to any idle core. That exists because data-center software wants workers that are interchangeable, preemptible and reschedulable.
- **MOT was treated as foundational.** The chip was sized so an entire MOT graph fits on one device, which "simplifies scheduling and reduces" cross-node traffic. DRAM was provisioned at roughly 700 MiB per MOT, forcing more than 4 GiB per VCU. **The software's preferred execution model set the memory budget of the silicon.**
- **Only the expensive, slow-changing parts were ossified.** A software fallback covers rare features, and reliability comes from redundant fleet deployment rather than per-part reliability.
- **Blast radius became a hardware concern.** A video is spread over hundreds of VCUs, so one faulty chip corrupts many videos. Mitigations: "golden" transcode self-tests at worker startup, end-to-end integrity checks where output length must match input, per-chunk VCU provenance logging for fault correlation, and a proposal to move to consistent hashing to shrink the set of VCUs any one video touches.

THE GAIN WAS A NEW CAPABILITY, NOT JUST A CHEAPER OLD ONE

VP9 software encoding cost 6–8× more than H.264 — a 150-frame 2160p chunk took about 15 wall-clock minutes and over an hour of CPU. So VP9 was produced *only for popular videos*, after upload, in preemptible batch, using SOT.

The VCUs made VP9 affordable at upload time via MOT, cut live latency to roughly five seconds end to end, and enabled 4K60 at 35 Mbps for Stadia.² **A sufficiently large efficiency gain stops being an efficiency gain and becomes a product change.**

5 • Adaptive bitrate: the manifest is the contract

Two standards, one abstraction. MPEG-DASH uses an XML **MPD** — Period, then AdaptationSet per media type, then Representation per ladder rung carrying bandwidth, dimensions and codec, then segment templates or byte ranges. HLS uses text `.m3u8` — a master playlist of variants, each pointing at a media playlist of segments.³

Both express the same idea: **the manifest is what allows a dumb HTTP cache to serve an adaptive stream.** All the intelligence is in the client, and every byte in between is just a cacheable file.

Segment duration	Shorter (2 s)	Longer (10 s)
Live latency	Lower — buffer is typically ≥ 3 segments	Higher floor

Segment duration	Shorter (2 s)	Longer (10 s)
Reaction to bandwidth change	Faster	Coarser
Request overhead	More requests, larger manifests	Fewer requests
Compression efficiency	Worse — more keyframes per second	Better

The client-side result worth knowing

Throughput-based adaptation picks the highest rung below an estimate of future capacity. The estimate is the weak point: HTTP throughput over a shared variable path is noisy, and biased *downward* when the client is buffer-full and idling.

The canonical counter-argument is Huang et al.'s buffer-based approach: **in steady state, capacity estimation is unnecessary**. Mapping current buffer occupancy directly to a bitrate through a monotone function is sufficient; throughput estimation is needed only during startup, when the buffer is growing from empty. On Netflix production traffic this cut the rebuffer rate by 10–20% at a similar average video rate.⁴ Real players are hybrid: throughput-driven at startup and after seeks, buffer-driven in steady state.

THE TRADE — PUTTING THE BITRATE DECISION IN THE CLIENT	
CHOSEN	The client picks the rung, per segment, driven by buffer occupancy in steady state. The server publishes a manifest and serves files.
OVER	A server-chosen bitrate, or client adaptation driven by throughput estimation alone.
COSTS	Every byte of adaptation logic now lives where you control it least — in a client you did not ship and cannot upgrade on demand, so quality becomes a function of client version. Buffer-based adaptation also needs a <i>large</i> buffer to work, which is exactly what live streaming cannot afford; and startup still requires throughput estimation, so you maintain two mechanisms rather than one.
REVERSES WHEN	<i>your latency budget is seconds rather than minutes, or when you hold signal the client cannot see — fleet-wide congestion, per-user history, cost per delivered byte. That is the argument for server-side adaptation, and it is the direction YouTube has since moved.</i>

FLAG: "YOUTUBE USES DASH" IS NOW AN INCOMPLETE DESCRIPTION

YouTube has moved substantially toward **server-side ABR** over a proprietary format, delivering audio and video in a single request and requesting by player time plus buffered-range metadata rather than byte ranges — moving the rung decision to the server, where fleet-wide congestion signal and per-user history are available.

This is documented only through reverse-engineered clients, not by Google. So it is *unofficial* rather than wrong, and DASH/HLS remains the right thing to design with in an interview. But note the direction of travel: the industry spent a decade moving the decision to the client, and the largest operator is moving part of it back.

6 • Vitess: how YouTube scaled MySQL instead of leaving it

This is the part of the story most often told backwards. The documented path:⁵ primary for writes plus a replica for reads → more replicas → writes outgrow the primary → shard → and now **application code had to pick the shard before every query**. Vitess's founding purpose was to lift that routing logic out of the application into a proxy.

Component	Role
vtgate	A stateless proxy that speaks the MySQL wire protocol, so applications connect to it <i>as if it were MySQL</i> . It parses the query, consults the schema and topology, routes to the right tablets "considering the sharding scheme, required latency and the availability of tables", scatters where necessary, and merges results.
vttablet	A sidecar in front of each <code>mysqld</code> — the only thing that talks to MySQL. That is what makes server-side connection pooling possible: thousands of application connections multiplex onto a small MySQL pool, avoiding MySQL's per-connection memory cost. It also enforces row-count limits and query timeouts, consolidates duplicate in-flight reads , and maintains a per-shard denylist used during cutover.
Query rewriting	Vitess "works hard to create an illusion of the user having a single connection to a single database. In reality, a single query might interact with multiple databases and may use multiple connections to the same database" — so it rewrites session variables, <code>last_insert_id()</code> , transaction semantics, and adds the sharding-key column so scattered results can be reassembled.
Online resharding	<code>Reshard</code> builds replication streams from source to target shards, copies existing rows, then tails the binlog to stay caught up. <code>SwitchTraffic</code> moves reads first, then for primaries: stop writes on the source, wait for the target to reach the write stop point, rewrite routing rules, deny the old tables. <code>ReverseTraffic</code> rolls back. Downtime is the catch-up window — seconds, not a maintenance night.

TWO FLAGS ON THIS SECTION

"YouTube was rewritten off MySQL" is backwards. Vitess exists precisely so that it did not have to be.

"YouTube runs on Vitess" is unconfirmed as a present-tense claim. It is true historically and per Vitess's own documentation, but no current Google statement describes YouTube's database topology. Vitess today is maintained largely outside Google, and its named production users are Slack, GitHub, Etsy and others — not YouTube. The widely-quoted "70,000 MySQL nodes across 20 data centers" is vendor marketing material, undated and uncorroborated.

7 • The edge, and why the long tail is treated differently

Google Global Cache is Google-supplied hardware placed *inside ISP networks*: Google provides and remotely operates the servers, the ISP provides rack, power, transit and a BGP feed. Stated hit rate: "typically between 70–90% of cacheable traffic can be served from GGC."⁶

Three details from Google's own documentation are worth having:

- **It is hierarchical, and fill sources are chosen dynamically.** A node's cache-fill can come from another GGC node in the same network, a GGC node in a different autonomous system, or Google core. Selection depends on data availability, drain state, spare serving capacity and measured inter-node latency — and a node is automatically excluded as a peer source if its latency is poor or it lacks headroom.
- **Caches are never pre-filled.** "GGC servers cache content dynamically, according to local user demand and the storage capacity available." So fill volume starts at 100% of demand and decays as the popular content lands.
- **Some high-volume content is deliberately not cached.** "Non-cached requests may be served from Google core data locations, *or proxied via GGC*." That is the elegant bit: the tail is served from core but *routed through* the edge node, so it reuses the good network path without consuming edge storage.

And the reason this is rational is the popularity distribution — a stretched power law with three regimes. Very popular videos are a small fraction of transcode and storage cost but the majority of watch time, so they are worth extra compute to reduce delivered bandwidth. The long tail is the *majority of videos*, watched rarely enough that the goal is simply to "minimize storage and transcoding costs while maintaining playability".²

The most important word in that policy is **reversible**: "old videos can increase in popularity and may need to be reprocessed with a higher popularity treatment well after upload." Tiering is a continuously re-evaluated decision, not a one-time choice made on the upload path.

8 • Trade-offs

The design bought	And paid
Playback on every device, at every connection speed	Dozens of renditions per upload, and the storage to hold them
Embarrassingly parallel transcoding	Chunking, reassembly, and a distributed failure model where one bad chip corrupts many videos
20–33× perf/TCO on VP9	Custom silicon: a multi-year hardware program, a software fallback path, and blast-radius machinery
Adaptive streaming from dumb HTTP caches	All adaptation logic in the client, where you control it least
Application servers untouched by video bytes	Pre-signed URL lifecycle, expiry and abuse handling
MySQL kept, with sharding made invisible	A substantial proxy layer to build and operate — Vitess is not a small dependency

WHAT TRANSFERS

1. **Find the boundary that makes work independently processable.** Closed GOPs turn a serial codec problem into an embarrassingly parallel one. Every workload has an equivalent boundary.
2. **Amortize the expensive shared prefix across all outputs.** Decode once, fan out many. Any pipeline producing N variants from one input should pay the parse/load/analysis cost once and share the derived state.
3. **Insert a protocol-compatible proxy rather than rewriting the datastore or the application.** Because vtgate speaks MySQL's wire protocol, sharding and pooling became infrastructure concerns invisible to application code.
4. **Tier treatment by popularity, and keep the tiering reversible.** Spend compute on the head to save bandwidth; spend nothing on the tail beyond playability; re-evaluate when popularity moves. One policy governs both transcode depth and edge caching.
5. **Keep large payloads off the application tier.** Pre-signed URLs let the client talk to storage directly; the API server handles only metadata.
6. **Hardware/software co-design runs both ways.** The software's preferred execution model set the chip's memory budget, and the chip's failure characteristics created new software requirements.

NUMBERS TO BE CAREFUL WITH

"500 hours of video uploaded per minute" is universally repeated and attributed to YouTube, but traces to marketing aggregators and has been frozen since around 2019. Google's own peer-reviewed phrasing is the vaguer "multiple hundreds of hours per minute". Use that.

"1 billion hours watched per day" is real and official — from February 2017. It is nine years old. Do not present it as current.

"Argos gives a 20–33× speedup" is wrong twice: it is perf-per-TCO, not throughput, and the 20–33× band is VP9-specific. H.264 perf/TCO is 4.4–7.0×.

"Each Argos chip has 10 encoder cores, so the board has 10." The board has two ASICs — twenty cores — and a host has ten cards.

PRACTICE

1. A 90-minute 4K upload must be transcoded into six resolutions and three codecs. Design the job graph. How many chunks, how many steps, and what is the critical path?
2. Work out the pixel-cost argument yourself: given a 1080p top rung, compute the total pixel cost of the full ladder below it and confirm the geometric-series result. Then decide at what ladder depth adding a rung stops being nearly free.
3. Design the resume mechanism for a 256 GB upload over a connection that drops every few minutes. What does the client need to remember, and what must the server track?
4. Write the buffer-occupancy-to-bitrate function for a buffer-based ABR client. What happens at the two ends, and how do you avoid oscillating between rungs?
5. A video uploaded three years ago suddenly goes viral. Describe everything the system should do, in order, and what it should be careful not to do.

SOURCES

1. Neo Kim, "System Design Interview: Design YouTube", The System Design Newsletter #117, Jan 2026 (partly paywalled)
2. Ranganathan et al., "Warehouse-Scale Video Acceleration: Co-design and Deployment in the Wild", ASPLOS 2021 — dl.acm.org/doi/10.1145/3445814.3446723
3. RFC 8216, "HTTP Live Streaming"; ISO/IEC 23009-1 (MPEG-DASH)
4. Huang, Johari, McKeown, Trunnell & Watson, "A Buffer-Based Approach to Rate Adaptation: Evidence from a Large Video Streaming Service", ACM SIGCOMM 2014
5. Vitess documentation — vitess.io/docs (History, VTGate, Query Rewriting, Connection Pools, Reshard, SwitchTraffic)
6. "Introduction to Google Global Cache", "Cache-Fill", "Content Served" — Google Interconnect Help, support.google.com/interconnect

Uber ETA

Half a million shortest-path queries a second, over a graph with half a million nodes in one city alone.

AT A GLANCE

ETA requests	~500,000 per second
ETA requests per trip	~1,000
Trips completed daily	>18 million
Road intersections, SF Bay Area alone	~500,000
Nodes touched after partitioning	~700
Production routing latency	Single-digit milliseconds (Uber, 2015)
Contraction-hierarchy preprocessing	~12 h full world graph; ~10 min incremental (Uber, 2015)

Every other chapter in this guide is about moving data. This one is about an algorithm — and it contains the single cleanest example in the book of a complexity argument that changes an architecture.

1 • Requirements

ETA is not one feature. It is computed in four distinct situations, each with different constraints:¹

Scenario	What is being asked
Eyeball	The rider has entered a destination and wants to know how long the trip takes, before committing. Latency-sensitive, and wrong answers here lose the booking.
Dispatch	Which car reaches this rider soonest? This is an ETA query <i>per candidate driver</i> — the reason one trip generates a thousand requests.
Pick-up	How long until the assigned driver arrives? Now the answer is displayed and counted down, so error is visible.
On-trip	Live updates to the destination arrival time, recomputed continuously as the vehicle moves.

Non-functionally: the answer must be *accurate*, because "a rider is likely to avoid future trips if the actual trip time is higher than ETA" — at 18 million trips a day, systematically optimistic ETAs are a revenue problem, not a UX problem.¹

2 • The core insight: a map is a graph, and Dijkstra is too slow

Model the physical map as a directed weighted graph. Every road intersection is a node; every road segment is a directed edge. Computing an ETA becomes finding the shortest path. One-way streets, turn restrictions, turn costs and speed limits all encode as graph properties.

Dijkstra's algorithm finds shortest paths in $O(n \log n)$ where n is the node count. The San Francisco Bay Area alone has about **half a million intersections**. At 500,000 requests per second, that is not a candidate.¹

The geometric argument

Uber's published explanation of the fix is an analogy worth reproducing exactly, because it makes the complexity change intuitive:¹

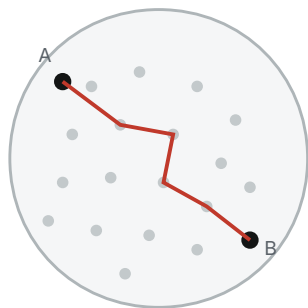
AREA VERSUS PERIMETER

Imagine a dense graph mapped onto a circle. Searching it naively means traversing *every node inside the circle* — a cost that scales with the **area**, πr^2 .

Now partition the graph and precompute the best path *within* each partition. Finding a route between two points no longer requires visiting interior nodes: it requires interacting only with nodes on the **partition boundaries** — a cost that scales with the **perimeter**, $2\pi r$.

For the Bay Area, that takes the number of nodes involved from roughly **500,000 down to about 700**.

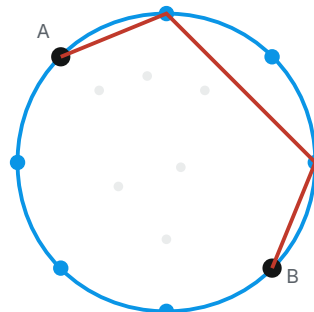
NAIVE SEARCH — COST $\propto$ AREA



every interior node may be traversed
 $\sim 500,000 \text{ nodes} \cdot \pi r^2$

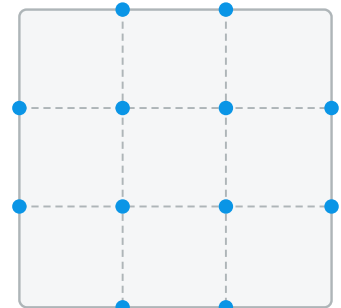
partition
+ precompute

PARTITIONED — COST $\propto$ PERIMETER



only boundary nodes are touched
 $\sim 700 \text{ nodes} \cdot 2\pi r$

IN PRACTICE



paths inside each cell
are precomputed

Figure 4.1 — The complexity change, geometrically. Partition the graph, precompute the best path inside each partition, and the search only ever touches boundaries. The cost stops scaling with the area of the region and starts scaling with its perimeter.

3 • Edge weights: traffic is the accuracy problem

A shortest path by distance is not the fastest path. Traffic is a function of time of day, weather, and vehicle density, and it goes into the graph as **edge weights**.¹

Uber combines **aggregated historical speed** with **real-time speed**, because more traversal data makes the traffic estimate more accurate. That combination is the interesting design choice: historical data gives coverage of segments nobody is currently driving, and real-time data corrects for today. Neither alone is sufficient — historical alone misses the accident, real-time alone has no signal on a quiet residential street.

WHY LIVE TRAFFIC IS ARCHITECTURALLY EXPENSIVE

If edge weights change continuously, then *any precomputation over those weights is continuously invalidated*. The entire difficulty of production routing lives in this tension, and the next section is about how Uber resolved it.

4 • Map matching: the GPS lies

GPS signals are noisy and sparse — worst in tunnels, and degraded near tall buildings by the **multipath effect**, where the signal reflects off a building and travels further than it should, producing a position that is simply wrong. A poor fix degrades ETA accuracy directly.¹

The fix is **map matching**: snapping raw GPS observations onto actual road segments. Two algorithms do the work, and both are worth being able to describe:

- **A Kalman filter** estimates position by combining a prediction from prior state with the new noisy observation, weighted by how much each is trusted. The source's analogy: "a person who makes a correct guess about something's location," taking both new and old information into account.
- **The Viterbi algorithm** — dynamic programming — finds the most probable *sequence* of road segments. The source's analogy is the better one: "a person who figures out the correct story even if some words were spelled wrong... by looking at the nearby words and fixing the mistakes so that the story makes more sense."

The distinction matters. The Kalman filter cleans up *where you are*; Viterbi decides *which road you have been on*, using the fact that road networks constrain plausible sequences. A single point can be ambiguous between two parallel streets; a sequence of points usually is not.

5 • What Uber actually shipped, and what it rejected

Uber published a detailed account of its routing engine, and the most instructive part is the list of well-regarded techniques they *evaluated and discarded*.²

Approach	Why it is attractive	Why it was rejected
Plain Dijkstra	Correct and simple	"In a production environment, Dijkstra, or any other algorithm that works on top of an unprocessed graph, is usually too slow."

Approach	Why it is attractive	Why it was rejected
Contraction hierarchies as used by OSRM	A few milliseconds per route; "below 100 milliseconds in the 99th percentile"	Preprocessing. "Roughly 12 hours to build the contracted graph using all the roads of the world." Dynamic updates that keep node ordering fixed got that to ~10 minutes with 10% of segments changing speed — still too slow for live traffic.
Geographic sharding	Obvious partition of a geographic problem	Not correctness — capacity shape: "if one region received too many requests during peak hours, the other servers couldn't share the load."
A*	No precomputation at all, so real-time edge weights are free. Good for short en-route legs.	"Response grows geometrically in relation to the depth of nodes traversed." Measured at ~120 ms for one cross-San-Francisco route — several times slower than contraction hierarchies. Landmark variants did not close the gap.
Layered cell partitioning shipped Apr 2015	—	Chosen. Partition the graph into layers of small, relatively independent cells, and re-run preprocessing <i>only for the affected cells</i> , in parallel. Result: "hundreds of thousands of ETA requests per second, with single-digit milliseconds response time."

THE TRADE — LAYERED CELL PARTITIONING	
CHOSEN	Partition the graph into layers of small, relatively independent cells, and re-run preprocessing only for the affected cells, in parallel.
OVER	Contraction hierarchies (fastest queries, unusable preprocessing), A* (no preprocessing, geometrically growing cost), and geographic sharding (wrong load shape).
COSTS	A route crossing many cell boundaries is stitched together from per-cell optima, so it can be measurably worse than a full graph search would find. You are accepting a small, permanent accuracy loss to make the query affordable. The partition scheme also becomes a thing that must be maintained as the road network changes.
REVERSES WHEN	<i>your edge weights are static. If nothing invalidates the precomputation, contraction hierarchies are strictly better — they were rejected here for freshness, not for speed.</i>

THE GENERALIZABLE LESSON FROM THAT TABLE

Preprocessing that cannot be partially invalidated is preprocessing you cannot use with live data.

Contraction hierarchies were not too slow to *query* — they were the fastest option. They were unusable because a change anywhere required rebuilding everything. The winning design is not faster at querying; it is *incrementally invalidatable*.

This generalizes far beyond routing. It is the same property that makes a good cache key, a good materialized view, and a good build system. When you design a precomputed artefact, the first question is not "how fast is it to read?" — it is "what is the blast radius of one change?"

6 • Then put machine learning on top — carefully

A routing engine produces a *physical* ETA: the sum of predicted traversal times over the chosen segments. Reality differs, systematically, for reasons the graph does not model — how long it takes to find a parking spot, how a particular airport pick-up zone behaves, what this driver does at this intersection on a Friday.

Uber's first attempt layered an ML offset on top of OSRM using historical trip data. Its failure mode is named explicitly: **cold start in new cities** — no history, no correction.²

The current approach, DeepETA, is architecturally cleaner: the routing engine produces the physical ETA, and a deep model predicts the **residual** between that ETA and observed reality, using origin, destination, time of day, live traffic, and request type — a rideshare pick-up behaves differently from a delivery drop-off. The model is an embedding module feeding a linear-transformer layer and a calibration layer, and Uber describes it as "the highest QPS model at Uber".³

RESIDUAL PREDICTION IS THE REUSABLE PATTERN

Do not ask the model to predict the answer. Ask it to predict *your error*.

You keep a physically interpretable baseline that works everywhere including day one in a new city, and the model only has to learn the correction — a much smaller, better- conditioned problem. If the model is unavailable or untrained, you degrade to the baseline instead of to nothing. This pattern appears throughout applied ML and is almost always the right first architecture.

7 • Trade-offs

The design bought	And paid
500,000 shortest-path queries/sec at single-digit millisecond latency	A precomputation layer that must be kept fresh, and a partitioning scheme that has to be maintained as the road network changes
Routes that respect live traffic	Rejecting the fastest-querying algorithm available, because its preprocessing could not be incrementally invalidated

The design bought	And paid
Accurate positions in urban canyons	Two probabilistic filters in the hot path, each with parameters to tune and failure modes of their own
ETAs that reflect reality, not just the graph	An ML layer with a cold-start problem — mitigated, not eliminated, by predicting residuals
A partitioned graph small enough to search	Routes that cross many partition boundaries are less optimal than a full search would find. This is a real accuracy cost, accepted deliberately.

WHAT TRANSFERS

1. **Partition and precompute to change the complexity class.** Area to perimeter. The cost stops scaling with the size of the region and starts scaling with its boundary — and 500,000 becomes 700.
2. **Design precomputation for incremental invalidation, not just fast reads.** The best-querying algorithm lost because one change invalidated everything. Ask what the blast radius of a single update is.
3. **Combine historical and real-time signal.** Historical gives coverage where there is no current traffic; real-time corrects for today. Neither is sufficient alone.
4. **Clean the input before trusting it.** Kalman filtering for position, Viterbi for sequence. Sensor data is a hypothesis, not a fact, and sequences disambiguate what points cannot.
5. **Have the model predict your error, not the answer.** You keep an interpretable baseline that works on day one in a new market, and you degrade gracefully when the model is absent.
6. **One user action can be a thousand backend queries.** A single trip generates ~1,000 ETA requests, mostly from dispatch evaluating candidate drivers. Always ask what the request amplification factor is before sizing anything.

TWO FLAGS

"Uber replaced Dijkstra with contraction hierarchies" is backwards. Uber evaluated contraction hierarchies and *rejected* them for the production engine, because 12-hour full preprocessing and 10-minute incremental updates both block live traffic. It ships a layered-cell partition scheme instead.

Be careful attributing H3 to driver proximity search. Uber's H3 hexagonal grid is published as the grid for marketplace analysis, surge pricing and dispatch optimization. Chapter 14 covers it properly; the claim that it is the index behind real-time nearby-driver lookup is an extrapolation, not a citation.

PRACTICE

1. Work out the area-versus-perimeter argument numerically. For a region of 500,000 nodes, what partition size minimizes total work — and what happens to route quality as partitions get larger?
2. A road closes. Trace exactly what must be recomputed under (a) contraction hierarchies and (b) layered cell partitioning. This is the whole chapter in one exercise.

3. Design the dispatch query. Given a rider and 40 nearby drivers, you need an ETA for each. How do you avoid 40 independent shortest-path searches?
 4. Your ETAs are 12% optimistic in one city and accurate elsewhere. List five possible causes, and say how you would distinguish between them.
 5. You are launching in a city with no historical trip data. What does your ETA system do on day one, and what is your plan for weeks two through eight?
-

SOURCES

1. Neo Kim, "How Uber Computes ETA at Half a Million Requests per Second", The System Design Newsletter #26
2. Thi Nguyen, "ETA Phone Home: How Uber Engineers an Efficient Route", Uber Engineering Blog, Nov 2015
3. "DeepETA: How Uber Predicts Arrival Times Using Deep Learning", Uber Engineering Blog, Feb 2022
4. Isaac Brodsky, "H3: Uber's Hexagonal Hierarchical Spatial Index", Uber Engineering Blog, Jun 2018

Instagram

2.5 billion users on Python, Postgres and Memcache. Where you put the data turns out to matter more than what you store it in.

AT A GLANCE

Users served	~2.5 billion (2024)
Growth to 30M users	Under 2 years from launch
Application language	Python — processes, not threads, because of the GIL
Feed store	Cassandra, one cluster <i>per continent</i>
Relational store	PostgreSQL, leader–follower replication
Cache	Memcache, deliberately <i>not</i> replicated across data centers
Request timeout	12 seconds, then terminated

This is the reassuring chapter. There is no custom silicon, no bespoke database, no new concurrency model — just Python, Postgres, Cassandra and Memcache, arranged carefully. The case study's own conclusion is worth stating up front: proven technologies are enough to handle internet-scale traffic.¹

1 • The three problems hypergrowth created

Instagram scaled vertically first — bigger machines — until that became expensive, then horizontally by adding servers. That solved capacity and created three new problems, and the chapter is organized around them.¹

Problem	What it actually meant
Resource usage	More hardware allowed scale-out, but each server was not used to capacity. Throughput per server was low, so resources were wasted.
Data consistency	Data centers across the world are necessary for latency, but consistency across them is hard — and a good user experience depends on it.
Performance	Python's global interpreter lock means processes rather than threads, and a synchronous application tier that blocks on external services executes fewer instructions per second than the hardware can deliver.

2 • Resource usage: make the code cheaper, not the machines bigger

The response was not to buy faster hardware. It was to **reduce the CPU instructions needed to finish a task** — which is to say, to write better code, and thereby fit more users onto each server.¹

- **Rewrite the hot, stable functions.** Frequently-used, rarely-changing functions were reimplemented in Cython and C/C++. Note the qualifier: *stable* functions. You pay a maintainability cost for this, so you spend it only where the code is not going to change.
- **Move objects from private to shared memory.** The number of Python processes a server can run is bounded by available memory, and each process has private memory plus access to shared memory. So common data objects that only need one copy were moved into shared memory — which directly raises the process count the machine can host.
- **Delete unused code,** because code in memory is memory not available for processes.

WHY THIS IS A SCALING TECHNIQUE AND NOT JUST TIDINESS

Process count per server is a hard function of memory. Every megabyte moved out of per-process private memory multiplies across every process on the box. In a process-per-request architecture, **memory layout is capacity planning**.

3 • The hard part: data placement as a latency strategy

Instagram uses Cassandra for the user feed, and Cassandra nodes can be deployed in different data centers and synchronized by eventual consistency. The obvious design is one global cluster with replicas everywhere. Instagram does not do that, because running nodes of one cluster across continents makes latency worse and consistency harder.¹

Instead: **a separate Cassandra cluster per continent.** The European cluster stores European users' data; the US cluster stores US users'. Put another way — and this is the sentence to remember — **the number of replicas is not equal to the number of data centers.**

That creates a disaster-recovery gap, so a *single* replica is kept in another region, and requests are routed using quorum at the cost of some extra latency. A deliberate, quantified trade: accept a little latency to avoid a regional outage becoming data loss.

Finding the right cluster: Akkio

If a user's data lives in exactly one continental cluster, every request must first discover *which*. Instagram uses Akkio, a data-placement service that splits data into logical units with strong locality — "a service that knows when and how to move data for low latency".¹

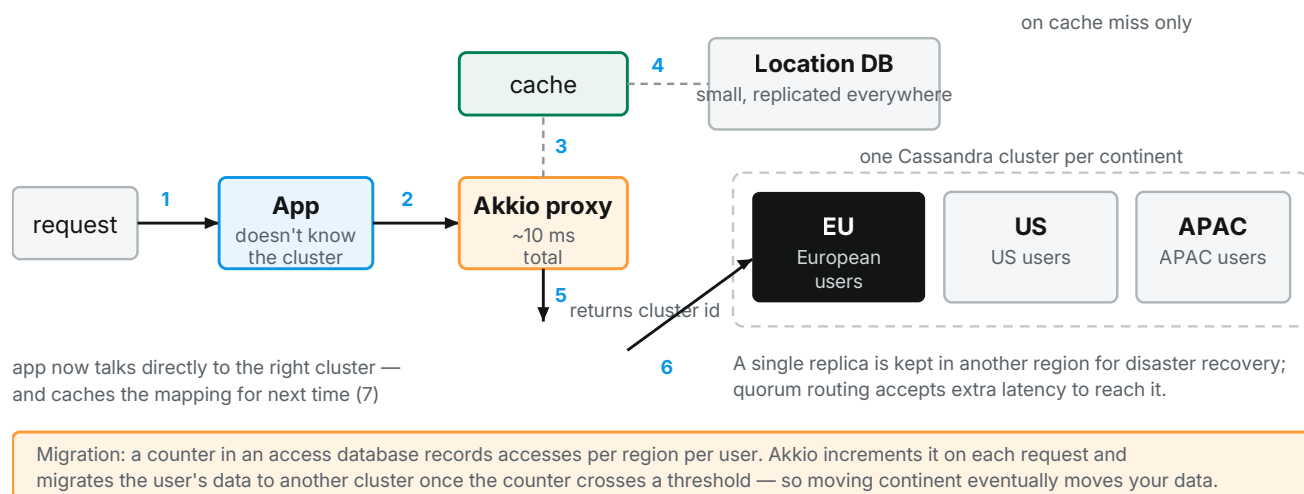


Figure 5.1 — Locating a user's data costs about 10 milliseconds, paid once and then cached. The important design property is at the bottom: placement is not static. An access counter per region drives migration, so the data follows the user rather than being pinned at signup.

The whole lookup adds roughly **10 milliseconds**, which the source calls a tolerable cost. Weigh that against a cross-continental round trip of ~150 ms and the trade is obvious: pay 10 ms once to discover where the data is, rather than 150 ms on every query to reach data in the wrong place.

Relational data — user information, friendships, picture metadata — lives in PostgreSQL in leader-follower replication, with writes to the leader and reads served by a follower in the same data center.

4 • Cache consistency without cache replication

Memcache protects Postgres from read traffic, handling millions of requests per second as a key-value store. And Instagram deliberately **does not replicate Memcache across data centers**, for latency.¹

The consequence is stale data, and the source gives a concrete example: comments added to a picture in one data center would not be visible to someone viewing that picture from another. So three mechanisms are layered on:

- 1. Invalidation from the database, not the application.** Data is not inserted directly into Memcache. A separate service watches PostgreSQL for updates and invalidates the corresponding cache entries. *The source of truth drives the cache; the application does not.*
- 2. Denormalized tables for expensive aggregates.** Counting likes on a popular picture with `select count(*) from user_likes_media where media_id = 42` is expensive; a maintained `media_likes` table makes it `select count from media_likes where media_id = 42`. Faster responses, less resource usage — at the cost of a denormalization to keep correct.
- 3. Memcache leases, to stop the thundering herd.** Invalidating many entries at once would send all of that traffic to Postgres simultaneously.

THE LEASE WORKFLOW — WORTH LEARNING AS A UNIT

1. The first request after invalidation performs a **lease GET** rather than a normal GET.
2. Memcache forwards that request to PostgreSQL and tells the client to wait.
3. A second request for the same key is asked to wait, because the first is not yet fulfilled.
4. Alternatively Memcache offers the second request **stale data** — "enough in most cases, such as finding the number of likes on a picture".
5. The first request's response populates Memcache.
6. Pending and subsequent requests are served fresh data from cache.

Step 4 is the one that matters. **Knowing which reads tolerate staleness is what makes this pattern cheap.** A like count can be a few seconds old; an account balance cannot. Most systems have far more of the former than engineers assume.

5 • Performance: async IO and a hard timeout

The application tier is synchronous and must wait for external services, which means fewer instructions executed and server starvation. Two fixes:¹

- **Python async IO** for communication with external services, so a process is not blocked while waiting.
- **A 12-second request timeout.** Because there are processes rather than threads, a request that takes a long time to finish wastes server capacity outright. So requests are terminated at 12 seconds.

THE TIMEOUT IS A CAPACITY DECISION DISGUISED AS AN ERROR-HANDLING DECISION

In a process-per-request model, a hung request does not merely fail slowly — it holds a process, and process count is your concurrency limit. A hard timeout converts an unbounded capacity loss into a bounded error rate.

This is worth generalizing: **any resource pool needs a deadline on the thing that holds a slot.** Chapter 6 shows Airbnb reaching the same conclusion about database connections, and Chapter 12 shows Cloudflare building an entire queueing system around it.

6 • Trade-offs

The design bought	And paid
Low-latency reads for users on every continent	A user's data lives in one place, so a lookup service sits in front of every request and migration machinery must exist
Memcache latency without cross-region replication	Stale reads across data centers, mitigated by invalidation, denormalization, and accepting staleness where it is harmless
More users per server	C and Cython in the hot path — a permanent maintainability cost, scoped deliberately to stable code

The design bought	And paid
Bounded capacity loss from slow requests	Legitimate slow requests are killed at 12 seconds
Disaster recovery for regional failure	One out-of-region replica plus quorum routing, at extra latency
Proven, boring technology	No exotic capability. Everything hard is solved by arrangement rather than by a better primitive — which is a feature, not a limitation.

WHAT TRANSFERS

1. **Data placement is a latency strategy.** Replica count need not equal data-center count. Put the data where the user is, keep one replica elsewhere for survival, and pay a small lookup cost to find it.
2. **Make placement dynamic.** An access counter per region, driving migration, means data follows users instead of being pinned wherever they signed up.
3. **Invalidate from the source of truth.** A service watching the database is more reliable than every write path remembering to update the cache.
4. **Denormalize for expensive aggregates.** A maintained counter beats a `count(*)` over a hot partition every time.
5. **Use leases against the thundering herd — and offer stale data.** One request does the work; the rest wait or accept staleness. Cataloging which of your reads tolerate staleness is high-value, low-effort work.
6. **Put a deadline on anything that holds a pool slot.** Requests, connections, locks, leases. Unbounded hold time converts into unbounded capacity loss.
7. **Cheaper code is a scaling technique.** Fewer CPU instructions and less per-process memory means more users per server — and in a process-per-request model, memory layout *is* capacity planning.

PRACTICE

1. A user moves from London to Tokyo permanently. Trace their data through Akkio over the following six weeks. What does the access counter do, when does migration trigger, and what does the user experience during it?
2. Go through the reads in a product you know and classify each as staleness-tolerant or not. What fraction can be served stale, and what would that let you simplify?
3. Design the cache-invalidation service. It watches Postgres for updates and invalidates Memcache. What happens if it falls behind? If it crashes? If it double-invalidates?
4. Your 12-second timeout is killing 0.3% of requests, and they are all legitimate uploads from slow connections. Fix this without removing the timeout.

5. Argue for a single global Cassandra cluster instead of one per continent. Under what conditions would that be the better design?

SOURCES

1. Neo Kim, "How Instagram Scaled to 2.5 Billion Users", The System Design Newsletter #49, Jun 2024 — drawing on "Scaling Instagram Infrastructure"; "Cross Continent Infrastructure Scaling at Instagram", SREcon19 Asia/Pacific; "Managing data store locality at scale with Akkio"; "Open-sourcing a 10x reduction in Apache Cassandra tail latency"
2. Nishtala et al., "Scaling Memcache at Facebook", USENIX NSDI '13 — the canonical treatment of leases and cache consistency at this scale

Airbnb

Two people click "Book" on the last room in the same second. What stops you charging both of them?

AT A GLANCE

Active listings	9M+, across 150K+ cities (May 2026)
Nights and seats booked	148.3M in the quarter, +10% YoY (Q2 2026)
Write rate (Booking.com-scale model)	~1.5M bookings/day ≈ 17 TPS
Read rate (same model)	~1,000 QPS
Payment consistency after Orpheus	99.999%, while annual volume doubled (Apr 2019)
Property page fan-out	20 downstream services (Dec 2018)
Engineering during SOA migration	200 → 1,000; page loads up to 10× faster (2015–18)

This is the only chapter where the interesting constraint is being *right* rather than being fast — and the numbers are what make it interesting. At seventeen writes per second, throughput is a non-problem. Everything hard here is about correctness under concurrency, and that is exactly the part most candidates handle worst.

A PROVENANCE NOTE, STATED PLAINLY

Airbnb has published nothing describing its booking-concurrency mechanism. Every claim in circulation — row locks on calendar-day rows, Redis holds, optimistic versioning — traces to interview-preparation content, not to Airbnb.

So section 4 below presents *standard industry solutions as options*, with the analysis being mine and the reasoning being general. Where Airbnb has published something relevant — and on idempotency they have published a great deal — it is marked and cited. Knowing which is which is part of the lesson.

1 • Requirements

Clarifying questions first, because several of them change the design fundamentally:¹ How many properties and rooms — a single chain or a platform? Is payment taken at booking or at check-in? Can users cancel, and under what policy? **Do we support overbooking** — hotels typically permit 5–10%, and if the answer is yes then the entire "prevent double booking" framing is wrong? Does price change with demand and date?

Functional. Hosts and admins manage properties, room types, pricing and inventory. Guests search by location, dates, guest count, price and amenities. Guests book, view and cancel. Payments integrate with gateways and fail gracefully. Notifications confirm.

Non-functional. Search under 500 ms, booking confirmation in 2–3 seconds, 99.9%+ uptime, horizontal scalability. And one that is called out as non-negotiable: **strong consistency — two users cannot book the same room for the same dates.**

2 • Back of the envelope, and what it licenses

Writes

1.1B room nights/year / 2 nights per booking	= 550M bookings/year
550M / 365	= ~1.5M bookings/day
1.5M / 86,400	= ~17 bookings/second

Reads

~516M visits/month, ~7.5 pages per visit	
→ ~17M visits/day → ~200 visits/second	
x ~5 backend reads per visit	= ~1,000 read QPS

Inventory storage

~85,000 properties x 10 room types x 730 days	= ~620M rows
x ~100 bytes	= ~62 GB raw
+ indexes (2-3x) + reservations, guests, etc.	= 500 GB+

SEVENTEEN WRITES PER SECOND CHANGES EVERYTHING

At that rate you can afford the expensive, simple, correct options: a single relational primary, local ACID transactions, pessimistic locking. The source's own conclusion is worth quoting because it is the right instinct and the opposite of what most candidates reach for: distributed transactions "add massive complexity for minimal benefit here... At ~17 write TPS, the added complexity is not justified. Local ACID transactions are simpler, safer, and faster."¹

The general rule: **compute the write rate before choosing a consistency model.** A great deal of distributed-systems machinery exists to solve problems that a single database handles trivially at low write volume.

3 • Architecture

Service-oriented with clear boundaries — but, critically, with **tightly coupled data kept in the same database.** Clients and CDN, then an API gateway handling authentication, rate limiting and routing, then services: Property, Rate (dynamic pricing), Reservation, Payment, Notification, Search. Underneath: PostgreSQL as the transactional store, Redis for hot data, and a search index.¹

THE LOAD-BEARING ARCHITECTURAL DECISION

Reservation and inventory data live in the same database. That is what makes local ACID transactions possible instead of distributed ones. There is service-level separation at the application layer and shared data for tightly coupled entities.

The alternative — separate databases coordinated by two-phase commit or sagas — was rejected for stated reasons. 2PC gives strong consistency but adds latency, blocking, and a coordinator whose failure stalls the system. Sagas avoid locking but introduce eventual consistency and compensating actions, which is "risky for user-facing booking flows where 'eventually consistent' means angry users."

Note the shape of the argument: services are a *deployment and ownership* boundary, not automatically a *data* boundary. Splitting the data too is a separate decision with its own cost, and it should be made on evidence.

Services scale independently according to their own load: the Property service might need ten instances for search traffic while the Reservation service needs three, because booking TPS is low.

4 • The hard part: preventing the double booking

The invariant: for property *P*, no two confirmed reservations may overlap in date. And note the asymmetry with ticketing — hotel inventory is a **continuous interval**, not a counted unit, so "decrement a counter" does not express the constraint at all.

Here is the race, precisely:

THE RACE — WHY "CHECK THEN WRITE" IS NOT A DESIGN

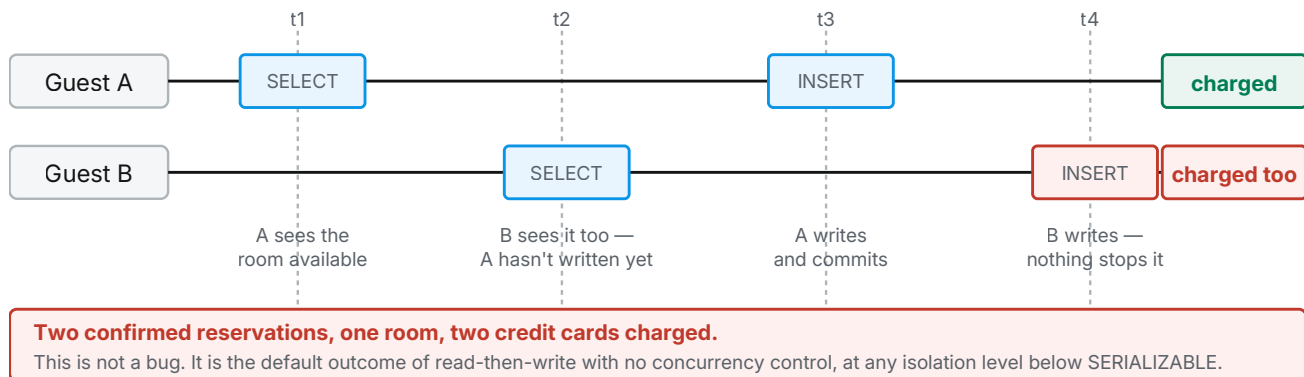


Figure 6.1 — The window between t2 and t3 is the entire problem. Every mechanism below is a different way of making that window impossible, or making the second write fail.

Option A — pessimistic locking: `SELECT ... FOR UPDATE`

Model one row per property-night. The booking transaction locks every night in the requested span, verifies all are available, flips them, commits.

Guarantees. Correct at any isolation level of READ COMMITTED or above, because the lock serializes conflicting writers. The strongest and simplest correctness story available.

Failure modes. Lock *ordering* matters — two transactions locking nights in different orders will deadlock, so always lock `ORDER BY date`. But the killer is subtler:

NEVER HOLD A LOCK ACROSS THE PAYMENT CALL

Payment authorization takes 300 ms to 3 seconds. A lock held that long serializes a popular property to a handful of bookings per second and exhausts the connection pool.

This is precisely the pattern Airbnb's own payments team rules out. Orpheus forbids "service interaction over networks" inside the database-transaction phases, having "learned the hard way that network calls (RPCs) during the Pre and Post-RPC phases... can result in... rapid connection pool exhaustion and performance degradation."²

The correct shape is: **lock → write a short-lived hold → commit → pay → a second transaction confirms.** The lock is held for milliseconds, not seconds.

Throughput. Per-property writes serialize, so throughput is roughly 1 divided by the lock hold time — fine at 5–20 ms. Contention partitions naturally by property, so aggregate throughput stays high even when one property is hot.

Option B — optimistic concurrency with a version column

Read `(availability, version)`, compute, then `UPDATE ... SET version = version+1 WHERE id = ? AND version = ?`. Zero rows affected means someone beat you; retry.

Guarantees. Prevents lost updates — but only at the granularity of the versioned row. If the version sits on the property as a whole, it is correct but produces false conflicts: two bookings for non-overlapping months abort each other.

Failure modes. Livelock under contention — retries collide and tail latency degrades superlinearly. It requires the entire read-modify-write to be replayable, so it composes badly with a payment side effect in the middle. And with a read replica in the path you can validate against a *stale* version; MVCC will not save you. Airbnb hit this class of bug and concluded that idempotency state must be read and written from the **sharded master only**, because "just a few seconds of replica lag could cause significant financial impact."²

Throughput. Best when conflicts are rare — under 1–2%. Higher ceiling than pessimistic locking at low contention because nothing blocks; worse under contention because work is thrown away rather than queued.

Option C — a unique constraint on per-night rows

Do not model availability as mutable state. Model a booked night as a *row whose existence is the claim*: `UNIQUE (property_id, date)` on a `booked_night` table, inserting one row per night in the span.

Guarantees. Enforced by the storage engine's B-tree. Correct under *all* isolation levels including READ COMMITTED, with no application-visible lock protocol and no read-then-write race at all. Immune to

replica lag, because the constraint is evaluated on the primary. On MySQL — which is Airbnb's transactional store² — this is the most robust option available.

Failure modes. The loser learns by *exception* rather than by predicate, so you must catch the duplicate-key error and translate it into a domain error — and, per Airbnb's own taxonomy, classify it **non-retryable**, because retrying a genuine conflict can never succeed. Multi-night inserts must be one transaction or you get partial claims. Cancellation means deleting rows, which reintroduces reuse questions.

Throughput. Excellent. Contention is per-night index page, non-overlapping ranges never conflict, and the check is $O(\log n)$ with no user-level lock.

Option D — the PostgreSQL exclusion constraint

The elegant answer, and badly under-used. Express the interval constraint *directly* — one row per reservation, no per-night fan-out:³

```
CREATE EXTENSION IF NOT EXISTS btree_gist; -- to mix = on a scalar with && on a range

CREATE TABLE reservation (
  id          bigserial PRIMARY KEY,
  property_id bigint      NOT NULL,
  stay        daterange  NOT NULL,
  status      text        NOT NULL DEFAULT 'confirmed',
  EXCLUDE USING GIST (
    property_id WITH =,
    stay         WITH &&
  ) WHERE (status = 'confirmed')          -- partial: canceled rows don't block
);
```

`daterange` canonicalizes to `[]` — lower inclusive, upper exclusive — which is exactly right for hotel semantics: `[2026-06-01, 2026-06-04)` and `[2026-06-04, 2026-06-07)` are *adjacent, not overlapping*, so check-out day equals next check-in day for free. That off-by-one is the most common bug this eliminates.

Guarantees. The overlap invariant becomes a *declarative database constraint*. No application code can violate it — not a buggy retry, not a backfill script, not a second service, not a concurrent transaction. Enforcement holds under READ COMMITTED; the constraint machinery takes its own internal locks, makes the second inserter wait, then fails it. It also composes: `tstzrange` for hourly inventory, `datemultirange` for split stays.

Failure modes. PostgreSQL-only, with no MySQL equivalent — which alone probably explains why Airbnb does not use it. It does not survive sharding, because the constraint is per-table: every reservation for a property must live on one shard, so shard by `property_id` and never by date. The partial `WHERE` clause is essential, or cancellations permanently poison those dates. GiST is slower to update than a B-tree.

Throughput. Very good, and the best write-amplification profile of all five: a 30-night stay is *one row and one index entry*, against 30 rows for options A and C.

Option E — distributed locks

Guarantees: none you should rely on for money. Single-instance Redis locks lose mutual exclusion on failover. Redlock's safety properties are contested. Any lease-based lock can expire mid-operation while the holder still believes it holds it — clock skew, a GC pause, a network partition — producing two simultaneous "holders".

Legitimate use: as *admission control in front of a real constraint*. Thin the herd on a hot property so fifty concurrent requests do not all enter database transactions. Never as the invariant itself. And note the cost: a network round trip on every booking, plus a new single point of failure whose outage stops bookings entirely.

Mechanism	Correct?	Breaks when	Write amplification
A Pessimistic lock	Yes, $\geq$ READ COMMITTED	Lock held across an RPC; wrong lock order deadlocks	1 row per night
B Optimistic version	Only at row granularity	Contention; validating against a lagging replica	1 row, but retried
C Unique constraint	Yes, all isolation levels	Nothing, within one shard. Must handle the exception.	1 row per night
D Exclusion constraint	Yes, declaratively	Not available on MySQL; cannot span shards	1 row per stay
E Distributed lock	No	Used as the invariant rather than as admission control	—

THE RECOMMENDATION

On PostgreSQL: **option D**, with a short-lived hold written under option A or C semantics to cover the payment window. On MySQL: **option C** as the invariant, with pessimistic locking scoped to the *hold write only* and never across the payment RPC. Option B as an optimization where contention is genuinely rare. Option E only as a shock absorber above a real constraint.

And the principle underneath all of it: **make the invariant a database constraint, not application logic**. Application-enforced invariants always eventually fail, because eventually there is a second code path.

5 • Orpheus: idempotency as a library, documented properly

This is the best-documented idempotency design in public engineering writing.² Airbnb built a general-purpose library — not a service — used across multiple payments services.

THE THREE-PHASE SHAPE

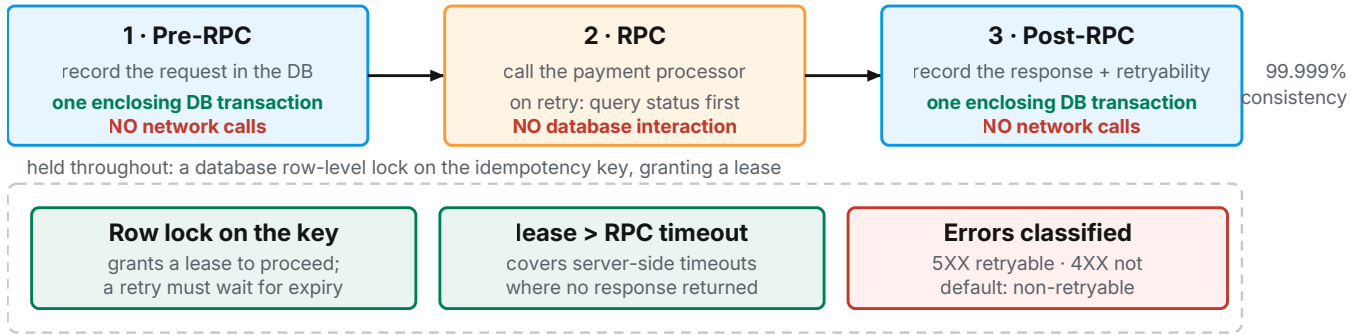


Figure 6.2 — Two hard rules make the shape work: no network calls in the phases that hold a database transaction, and no database work in the phase that makes the network call. The rigid shape is what lets non-specialists get idempotency right.

Several details are worth having exactly:

- **Why the rules exist.** Rule 2 — no network calls inside the transaction phases — is operational rather than theoretical: doing so caused "rapid connection pool exhaustion and performance degradation".
- **How library and application writes commit together.** Not nested transactions — **Java lambdas** passed as functional interfaces, so library-level and application-level writes land in one commit.
- **Why a library, not a service.** "We needed ultra low latency, so building a separate, stand-alone idempotency service wouldn't be sufficient. Most importantly, the service would suffer from the same problems it was originally intended to solve." An idempotency service is reached over the network — and the network is what you are defending against.
- **Key types.** Client-generated UUIDs for *request-level* idempotency; deterministic keys like `payment-1234-refund` for *entity-level*. Request payloads must never mutate between retries.
- **Where the state lives.** Sharded master only. Keys have high cardinality and even distribution, which makes them good shard keys.
- **The motive, stated once.** "The system should fail in a manner it could recover from" — without the phase structure, a failure mid-sequence leaves no systematic way to know where it stopped.

CHOSEN	Ship idempotency as a library with a rigid three-phase shape, linked into each payments service.
OVER	A standalone idempotency service that every caller reaches over the network.
COSTS	A library has to be adopted service by service and versioned across all of them — you cannot fix a bug centrally and have it take effect everywhere. It binds every consumer to one language runtime, which is a tax any polyglot fleet pays. And the rigid shape permanently constrains how product code in those services may be written.
REVERSES WHEN	<i>your fleet is polyglot and the latency budget tolerates a network hop — then a service is the only way to get one implementation everywhere. Note that Lyft reached the opposite conclusion about cross-cutting concerns (Chapter 14), and both are right: a sidecar is local, so it does not inherit the network failures a remote service does.</i>

Result: five nines of payment consistency while annual payment volume doubled.

6 • The migration, and a pair of opposite strategies

"Monorail" was the Ruby on Rails monolith. Engineers lost roughly **15 hours a week** to deploy delays from reverts and rollbacks; headcount went from 200 engineers in 2015 to 1,000; page loads improved up to 10×.⁴

The four data-layer tenets are the substantive output, and the first is the one that costs the most:

1. **Services own both reads and writes to their own data**, and other services reach it only through the owning service's API. This is what kills cross-domain joins and forces availability, pricing and booking into separate stores.
2. Services address a specific concern — avoid mini-monoliths, and avoid a fine-grained distributed monolith.
3. Avoid duplicate functionality via shared services and libraries.
4. **Data mutations publish via standard events.**

How they migrated without breaking it

The mechanics are reusable and worth stealing wholesale:⁴

- A custom **ActiveRecord adapter** diverted database calls from inside Monorail to the new service — so product engineers kept writing ordinary ActiveRecord and the redirection happened underneath them.
- **Reads were dark-launched** by dual-querying old and new and comparing results. Safe, because reads are idempotent and have no side effects.
- **Writes were validated against a shadow database**: write to both, then read both and diff. Load ramped 1% to 100% with comparison at each step.

- **SpinalTap** (open-sourced) tails database changes into Kafka — the change-data-capture substrate for tenet 4.

PAYMENTS DELIBERATELY DID THE OPPOSITE — AND THAT IS THE LESSON

Where "many teams at Airbnb chose to create a one-to-one replacement", payments redesigned fundamentally and accepted a much longer migration.⁵ The reason was structural: legacy billing models were coupled to *reservations for a stay*, so every payment feature had to be rebuilt to support Experiences.

What came out of it: a product-agnostic **"bill"** model; **data immutability** via append-only events plus **versioning**, so adding a night creates a new plan version and full money-movement history survives; every major workflow as a **DAG of retryable idempotent steps** on Orpheus; a Kafka message bus where Orpheus upgrades at-least-once into **exactly-once**; four phases with stays-bookings alone split into **30+ milestones**; a transformation layer reading transparently from legacy-or-new into a unified model; and asynchronous "matchup" jobs diffing old against new. Two parallel orchestration systems ran for nearly the whole migration.

Like-for-like replacement is the right default; fundamental redesign is correct when the legacy model encodes an assumption the business has outgrown. Knowing which situation you are in is the judgment call.

7 • Search: why availability is the hardest field in the corpus

Airbnb's published account of embedding-based retrieval contains the single best explanation of why calendar queries are expensive — and it is not the reason most people give.⁶

They chose **IVF over HNSW** for approximate nearest-neighbor search *despite HNSW scoring better on recall*, because of "the **high volume of real-time updates per second** for Airbnb home listings, as **pricing and availability data is frequently updated**. This caused the memory footprint of the HNSW index to grow too large."

READ THAT AGAIN

Availability is the *highest-churn field in the entire corpus*, and that fact alone dictated the choice of vector index architecture. A data-freshness property in one field reached up and determined an algorithm selection two layers away.

The general form: **the update rate of your hottest field constrains every index that contains it**. Ask about write rate per field, not just per table.

The second cost driver is filtering: "most Airbnb searches include filters, especially geographic filters. We found that parallel retrieval with HNSW alongside filters resulted in poor latency performance." IVF won partly because cluster assignments can be expressed as "a standard search filter", composing with the existing filter machinery rather than fighting it.

Two more published details: the two-tower model's **listing tower is computed offline daily in batch** for all homes, so only the query tower runs at request time. And Euclidean distance was chosen over dot product because it produced **balanced clusters** — IVF quality is highly sensitive to cluster-size uniformity. Flexible-date search is called out as adding "yet another layer of complexity", since there is no fixed check-in or check-out to filter on.

Note: the retrieval index is described as **Lucene**-based. Airbnb does not say "Elasticsearch" in this post. Elasticsearch is Lucene-based so the common claim may well be true, but it is not what they published.

8 • Resilience: the arithmetic that justifies all of it

The property detail page fans out to **20 downstream services**. At 99.9% each with no mitigations, the page is $0.999^{20} \approx 98.0\%$ available — roughly **14.5 hours of downtime per month**.⁷

What Airbnb built in response is a good checklist for any fan-out read path:

- **Asynchronous request processing** so a slow dependency does not hold a thread.
- **A CoDel queue with adaptive LIFO**. Under overload, serving the *newest* request first is better than the oldest, because the oldest requests have probably already been abandoned by their clients.
- **Service back pressure** — the server signals overload, and smart clients stop retrying and trip circuit breakers rather than amplifying the problem.
- **Deadline propagation** from the gateway, so work whose answer is already too late is not performed.
- **Client quota-based rate limiting** — the listings service has **30+ upstream callers**, so one misbehaving caller must not consume the budget.
- **Bulkheads**: a separate thread pool per dependency, so one slow dependency cannot exhaust the pool for the others.
- **Outlier detection** ejecting bad hosts in under a minute.

9 • Trade-offs

The design bought	And paid
Local ACID transactions and a simple correctness story	Reservation and inventory must share a database — a permanent constraint on how those services can evolve independently
Five nines of payment consistency	A rigid three-phase shape that every payments developer must follow, and idempotency state pinned to sharded masters
Independent deploys and 10× faster pages	A multi-year migration, shadow databases, dual-write comparison, and two orchestration systems running in parallel for years
Search that tolerates constant availability churn	A vector index with measurably worse recall than the alternative
A 20-dependency page that stays up	Seven distinct resilience mechanisms, each with its own configuration and failure modes

The design bought	And paid
Correctness on the booking path	Lower throughput than an optimistic design would allow — which costs nothing at 17 TPS and would be unacceptable at 17,000

WHAT TRANSFERS

1. **Make the invariant a database constraint, not application logic.** A unique index or exclusion constraint cannot be bypassed by a buggy retry, a backfill, or a second service.
2. **Never hold a database lock or transaction across a network call.** Split into Pre-RPC / RPC / Post-RPC, commit a durable intent before the RPC, and reconcile afterwards.
3. **Put strong-consistency state on the primary and shard it** rather than caching or replicating it. Seconds of replica lag on an inventory or idempotency check is a correctness bug, not a latency one.
4. **Ship cross-cutting correctness as a library with a rigid shape.** A service reached over the network inherits the failure it was built to prevent.
5. **Compute the write rate before choosing a consistency model.** A lot of distributed-systems machinery solves problems a single database handles trivially at low write volume.
6. **The update rate of your hottest field constrains every index containing it.** Availability churn is why Airbnb's vector search uses the less accurate algorithm.
7. **Run the fan-out availability arithmetic once.** $0.999^{20} = 98\%$ makes circuit breakers and bulkheads stop looking like over-engineering.
8. **Dark-launch reads by comparison; validate writes against a shadow.** The safest migration is one where both systems run and you diff them.

CLAIMS TO BE CAREFUL WITH

"Airbnb prevents double-booking with row locks on calendar_day rows." Unverified. Airbnb has published nothing on booking concurrency; every such claim originates in interview-prep content, often written in Airbnb's voice.

"Airbnb uses HNSW for vector search." Backwards — they evaluated it and *rejected* it for IVF.

"Airbnb completed its microservices migration." Airbnb consistently says *service-oriented architecture*, not microservices, and the payments account describes migration still winding down with two orchestration systems co-resident for years.

"3–4 billion calendar rows." Unverified arithmetic from secondary sources, based on a stale listing count and presuming a per-night materialization Airbnb has never described.

PRACTICE

1. Implement the hold-then-confirm flow for options C and D. Write the actual SQL for both transactions, and say exactly what happens if the process dies between them.

2. The product team now wants 10% overbooking. Rework your chosen mechanism. Which of the five options survives this requirement, and which becomes useless?
 3. Your booking table must be sharded. Trace what breaks under each of the five mechanisms, and choose a shard key.
 4. Design the idempotency key for "guest changes a reservation's dates". Is it request-level or entity-level? What happens on a retry where the payload differs slightly?
 5. A property is listed on Airbnb and two other platforms simultaneously. You now have three systems that can each sell the same night. Design for this — and be honest about what you cannot guarantee.
 6. Run the fan-out arithmetic for a page calling 8 services at 99.95%. Then compute what per-service availability you need for a 99.9% page, with and without graceful degradation.
-

SOURCES

1. Neo Kim & Hayk Simonyan, "System Design Interview: Design Airbnb", The System Design Newsletter #113, Jan 2026 (partly paywalled)
2. Chew & Khisti, "Avoiding Double Payments in a Distributed Payments System", Airbnb Tech Blog, Apr 2019
3. PostgreSQL documentation §8.17, "Range Types" — postgresql.org/docs/current/rangetypes.html
4. Jessica Tai, "Airbnb's Migration from Monolith to SOA", QCon SF 2018, via InfoQ, Feb 2019; github.com/airbnb/SpinalTap
5. Bryon Ross, "Rebuilding Payment Orchestration at Airbnb", airbnb.tech
6. Abdool et al., "Embedding-Based Retrieval for Airbnb Search", Airbnb Tech Blog, Mar 2025
7. He & Guo, "Building Services at Airbnb, Part 3", Airbnb Tech Blog, Dec 2018
8. Airbnb Newsroom fast facts (May 2026); Airbnb Q2 2026 Shareholder Letter

Redis at Scale

One in-memory datastore, eight different jobs. Which of them should it actually be doing?

Redis is the only entry in Part II that is a component rather than a product, and it earns its place because it appears in nine of the other fourteen chapters. Learning what it can do — and, more usefully, what it costs each time — makes the rest of the book cheaper to read.

The framing below follows the source: two engineers at a hackathon who know exactly one tool, and use it for everything.¹ That is a better teaching device than it sounds, because it forces the question *should it be doing this?* at every step.

1 • Caching

The default use, and the one everyone knows. A fast in-memory store sits in front of the database and absorbs repeated reads.

Mechanism. The database response is cached in a Redis hash. The query itself is hashed and used as the key. Every request checks the cache first: a hit is served from memory; a miss goes to the database and then populates the cache. This is the cache-aside pattern, and it is what Tinder uses (Chapter 9).

Offers. Roughly 100× faster responses, and a large reduction in database load.

Costs. Everything in Chapter 1's four cache decisions: what to cache, expiry, consistency, eviction. Instagram's lease mechanism (Chapter 5) exists because the naive version of this stampedes the database on invalidation.

2 • Queueing — with streams

Some work is too slow for the request path. Redis Streams provide an append-only log where every entry is immutable, which makes it usable as a durable queue.

Mechanism. On a cache miss the request is wrapped in a message, assigned a unique identifier, and appended to the stream. A worker consumes it when it has capacity, talks to the database, and the result is written back.

Offers. Buffering for asynchronous processing, decoupling of serving from processing, and the ability to autoscale workers on queue depth — which is a far better scaling signal than CPU.

Costs. The user-visible question the source's own commenters asked: *what does the user get back on a cache miss?* Either you hold the request open until the worker finishes (in which case the queue bought you buffering, not latency) or you return something provisional. Asynchronous processing is not free — it moves a latency problem into a user-experience problem, and you have to answer that question in the product.

REDIS STREAMS VERSUS KAFKA

Both are append-only logs with consumer groups. Kafka is built for very high throughput, long retention, and partitioned parallelism with strong ordering guarantees per partition. Redis Streams are built for low latency and operational simplicity, with retention bounded by memory.

The rule of thumb: if the log is a *buffer*, Redis Streams are usually enough. If the log is the *system of record* — replayable, retained, consumed by many independent teams — you want Kafka. Compare Chapter 9, where Tinder uses Kafka specifically for its per-partition ordering guarantees.

3 • Distributed locking

Many expensive queries arriving at once will overload the database. A distributed lock orchestrates access to a shared resource across many clients.

Mechanism. A worker acquires a lock before talking to the database, with an expiry time set on the lock. The number of workers touching the database concurrently is capped; the rest wait.

Offers. Resilience by avoiding a flood of simultaneous queries, and avoidance of the noisy-neighbor problem.

THE DISTINCTION THAT MATTERS: ADMISSION CONTROL VERSUS MUTUAL EXCLUSION

What is described above is **admission control** — limiting concurrency to protect a downstream resource. That is a legitimate and valuable use of a Redis lock, because if the lock is wrong you serve a few extra queries.

Mutual exclusion for correctness is different, and Redis should not be your guarantee. A single-instance lock loses exclusion on failover. Any lease-based lock can expire while its holder still believes it holds it — a GC pause, clock skew, a partition — producing two simultaneous holders. Chapter 6 works through why this disqualifies it as the mechanism preventing a double booking.

Use a Redis lock to protect a resource. Use a database constraint to protect an invariant.

4 • Throttling and backoff

Lock acquisition can fail. If failures simply retry immediately, you have rebuilt the overload you were preventing.

Mechanism. A worker that fails to acquire the lock puts its message *back* on the stream with a delay — and an additional delay each time the same message is re-inserted. That is **exponential backoff**, implemented in the queue rather than in the client.

Offers. Congestion management by controlling the number of parallel database requests.

Costs. Retries that are exponential but not *jittered* will still synchronize — every client that failed at the same moment retries at the same moment, forever. Tinder's implementation (Chapter 9) specifies exponential backoff *with jitter*, and that word is not decoration.

5 • Session store

The web server holds user data and preferences, and a stateful web server is hard to scale — this is rung 8 of Chapter 1's ladder.

Mechanism. Session data lives in a Redis hash with an expiry per user, and the expiry is renewed on each request. The web tier becomes stateless.

Offers. Easy horizontal scaling of stateless web servers and better absorption of traffic spikes.

Costs. A lookup on every request that used to be a local memory read, and Redis becomes a hard dependency of authentication. Consider what your service does when the session store is unavailable — the honest answer is usually "nothing", and that is worth knowing in advance.

6 • Rate limiting

Mechanism. A counter in a Redis hash represents the requests allowed on an endpoint in a period. Each request decrements it; at zero, requests are rejected; the counter resets after the period.

Costs. This is a fixed-window counter, and it has a known flaw: a client can send the full allowance at the end of one window and again at the start of the next, achieving double the intended rate across the boundary. Sliding-window or token-bucket algorithms fix it at the cost of more state. Worth knowing the flaw exists before someone finds it.

7 • The other structures, and what each one buys

The source's bonus list is the most useful part of the article, because each item maps onto a real problem elsewhere in this book:¹

Structure	Use	Seen in this guide
Sorted sets	Leaderboards — key-value pairs ordered by score	Chapter 14. Lyft stores driver IDs scored by last-report timestamp, making "who is nearby" and "who is still alive" the same read, with expiry as an $O(\log N)$ range delete.
Geospatial index	Find nearby points within a radius	Chapters 9 and 14 — though both use an explicit cell scheme (S2) for shard routing rather than relying on the built-in index.
Lists	A social network timeline	Chapter 1. Twitter's materialized timelines were Redis lists — later a customized fork with a purpose-built HybridList type.
Hashes	Field-addressable records	Chapter 2. WhatsApp's connection registry: which chat server holds this user's socket.
Pub/Sub	Real-time messaging	Presence and notification fan-out, though note that pub/sub is fire-and-forget — a disconnected subscriber misses messages entirely.

Structure	Use	Seen in this guide
HyperLogLog	Estimate set cardinality in fixed memory	Unique-visitor counting. The trade is explicit: a small, bounded error for a very large memory saving.
Time series	Metrics and analytics	The observability rung of the ladder.
RedisSearch	Full-text and SQL-like queries	Useful when the index fits in memory or changes often — which is exactly Chapter 6's problem, where availability churn made a memory-resident index too expensive.
RedisJSON	Nested JSON without deserialization cost	Avoids the parse-on-every-read tax of storing JSON as a string.

8 • Why it is fast, and what that buys you

Three properties from the source, each with a consequence worth understanding:¹

- **It runs in memory**, so reads and writes are fast. Consequence: your dataset must fit in RAM, and RAM is expensive. This is what bounded Twitter's timelines to 800 entries.
- **It is single-threaded, so there is no need for locks**. Consequence: commands are atomic by construction — which is why counters, rate limiters and leaderboards are trivial to implement correctly. But also: *one slow command blocks everything*. A `KEYS *` on a large database is an outage. Bluesky ran into the related limitation that Redis sets do not use multiple CPU cores, which is why they built their own in-memory graph service (Chapter 15).
- **It survives a crash with disk persistence**, and supports atomic operations via transactions. Consequence: durability is configurable, and the configuration is a real decision — the fast setting can lose the last window of writes.

ON THE LICENSE CHANGE

Redis changed its licensing in 2024, and forks exist — Valkey being the most prominent, adopted by several cloud providers. The source notes this in passing, and it matters for a practical reason: if you are designing today, "Redis" increasingly means "a Redis-protocol-compatible in-memory store", and which one you get may be decided by your cloud vendor rather than by you. The protocol is the durable part, not the implementation.

9 • The real lesson: one tool, eight jobs, and when that is wrong

The hackathon framing makes Redis look like a universal answer, and for a prototype it nearly is. The honest version at production scale is narrower.

Use it as	Because
A cache	Unambiguously the right tool. Losing it degrades performance, not correctness.
A session store	Right, with the caveat that it becomes a hard dependency of login.
A counter, rate limiter or leaderboard	Right, and the single-threaded atomicity is what makes it correct without effort.
An ephemeral spatial or liveness index	Right, and Chapter 14 shows the sorted-set pattern doing real work at 25 million QPS.
A buffer queue	Defensible, if retention bounded by memory is acceptable and the log is not a system of record.
Admission control	Defensible, as long as it protects a resource rather than guarding an invariant.
A correctness-critical lock	Wrong. See Chapter 6.
A primary datastore	Usually wrong. Memory-resident, with configurable rather than guaranteed durability. Every system in this guide keeps a durable store behind it — and Chapter 1's 800-entry limit exists precisely because there is a reconstruction path from disk.

WHAT TRANSFERS

1. **Single-threaded execution is a feature.** It makes every command atomic, which is why counters and rate limiters are correct without coordination — and why one slow command is an outage.
2. **Let one data structure carry two concerns when it can.** A sorted set scored by timestamp answers both "which" and "when", and expiry becomes a range delete instead of a separate reaper.
3. **Exponential backoff needs jitter.** Without it, everything that failed together retries together, forever.
4. **Protect resources with locks; protect invariants with constraints.** The distinction is the difference between a performance mechanism and a correctness mechanism.
5. **Autoscale on queue depth, not CPU.** Queue depth is a leading indicator of the thing you actually care about.
6. **Asynchronous processing relocates a latency problem into a UX problem.** Decide what the user sees while the work is queued, before you queue it.
7. **Know your fixed-window rate limiter can be doubled at the boundary.** Every simple algorithm has a known flaw; the skill is knowing which one you accepted.

PRACTICE

1. Implement a rate limiter that cannot be doubled at the window boundary. Compare sliding-window log, sliding-window counter, and token bucket on memory cost and accuracy.
 2. Your Redis cache fails completely at peak. Walk through what happens to each of the seven uses in this chapter, in order of severity.
 3. Design driver-location storage using sorted sets. What is the key, the member, the score? Write the commands for update, query-nearby, and expire-stale.
 4. You are asked to use Redis as the primary store for a shopping cart. Argue both sides, then decide — and state what you would need to be true.
 5. A single `KEYS *` in a production console causes a two-minute outage. Explain why, and name three ways to make that impossible.
-

SOURCES

1. Neo Kim, "Why Is Redis a Distributed Swiss Army Knife", The System Design Newsletter #50
2. Redis documentation — redis.io/docs — for command semantics, persistence modes and the Streams consumer-group model

Spotify

Playback starts in a quarter of a second. Most of those bytes never came from Spotify at all.

AT A GLANCE

Users	777M MAU, 300M Premium, 184 markets (Q2 2026)
Catalog	100M+ tracks, 7M podcast titles, 700K audiobooks (Aug 2026)
Playback latency	265 ms median, 515 ms p75, 1,047 ms p90 (Mar 2010)
Byte sources	8.8% servers, 35.8% peers, 55.4% local cache (Mar 2010)
Events	>8M/sec peak, 500+ event types, >350 TB/day raw (Q1 2019)
Services	810 active (2015 — the only first-party count)

Spotify's published architecture teaches one idea better than any other system in this guide: split the object into a latency-critical head and a bulk tail, and route them completely differently.

1 • Requirements

Functional. Artists upload. Users search and play. Users create and manage playlists and profiles. Basic observability.¹

Non-functional. Fast playback start, minimal rebuffering, straightforward operations. Rebuffering is handled by adaptive quality switching.

Capacity, roughly. At 30 million songs and ~3 MB per track at standard quality, that is about 90 TB of raw audio — and 2–3× that with regional replicas and versioning when artists re-upload. Metadata is trivial by comparison: ~100 bytes per song is about 3 GB. The key insight from the arithmetic: **audio dominates both storage and bandwidth; metadata is a rounding error.**

2 • The read path

What happens when a user presses play:¹

1. The app sends `GET /songs/{id}`.
2. The API server validates the auth token.
3. It queries the relational database for song metadata.
4. It generates a **signed URL** for blob storage access.
5. The app fetches audio chunks — range requests — **directly from blob storage or CDN**, using HTTP adaptive streaming.

6. The app periodically posts play events for analytics.

Note step 4 and 5 together: this is the same pre-signed-URL pattern as YouTube (Chapter 3). The application server handles metadata and authorization; the bytes never pass through it. Files are organized hierarchically in object storage and accessed via URLs that expire after a few hours.

3 • The hard part: where 265 milliseconds goes

The canonical measurement, from the published P2P paper: **median playback latency 265 ms, 75th percentile 515 ms, 90th percentile 1,047 ms**, with under 1% of playbacks suffering any stutter.²

And then the observation that reframes the number. Because more than half of all bytes came from the local cache, the authors say the median "likely measures the time needed for DRM together with local processing time" — **the median is dominated by local work, not by the network at all.**

SPLIT THE OBJECT: A WARM HEAD AND A CHEAP TAIL

On a random access — about 39% of playbacks — the client immediately requests the *first piece* from Spotify's servers over an **already-established long-lived TCP connection**. The paper is precise about why this matters: "no 3-way handshake is needed", and the server's congestion window on that long-lived connection is already large, letting it "respond to the latency-critical initial request without waiting for ACKs."²

Then, while streaming from a server, the client **throttles itself** to stay no more than about **15 seconds ahead** of the playback point — *if peers are available*. From peers there is no throttling and the client grabs the whole track.

Two costs, two paths: the expensive, low-latency path serves only the few hundred kilobytes the user is about to hear, and the cheap path serves everything else.

ONE TRACK, THREE SOURCES (measured, March 2010)

a single track, over time →

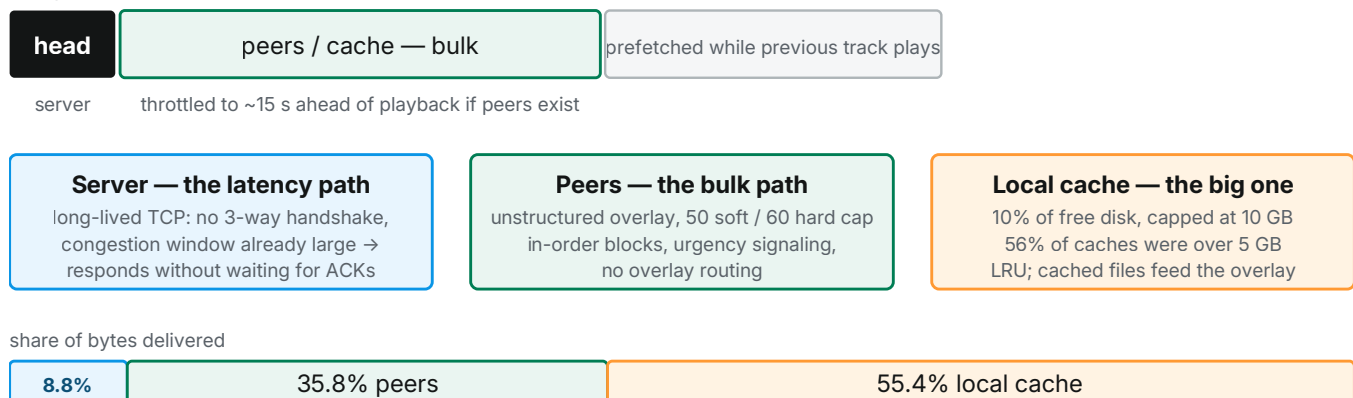


Figure 8.1 — The measured byte split. The widely-repeated claim that "80% of Spotify's traffic was P2P" is contradicted by the only published measurement: peers served 35.8%. What people are conflating it with is the cache, which served more than half of everything.

The prefetch measurement that nobody else has

Spotify's heuristics: start searching for peers and downloading the next track when **30 seconds or less** remain of the current one; when **10 seconds or less** remain, prefetch the beginning of the next track *from the server* if necessary.²

And then the best accidental experiment in this book. A client bug once disabled next-track prefetching for a week. **Median latency that week was 390 ms, against 265 ms otherwise.**

WHY THAT 125 MS IS WORTH MORE THAN THE 265 MS

It is a measured counterfactual. Most performance work is justified by a before-and-after comparison where everything else also changed; this is the same system, the same week, with exactly one mechanism removed.

The transferable practice: **build the ability to turn your optimizations off.** Twitter's Decider (Chapter 1) is the same capability, deliberately engineered rather than discovered by accident. If you cannot disable an optimization, you cannot measure it, and you will end up maintaining code whose value nobody can demonstrate.

4 • The P2P overlay, and why it was retired

Worth understanding as a design, because the choices are unusual and each one is justified:²

- **A proprietary protocol, not BitTorrent.** One *unstructured* overlay for the whole catalog — not per-torrent, not a distributed hash table.
- **A soft peer limit of 50 and a hard limit of 60** — the client stops making new connections at 50 and prunes back below it; 60 is the never-exceed ceiling. Neighbor eviction uses a six-criterion utility heuristic, and there is **no overlay routing**. Actual degree varies widely, and the paper notes that **30–50% of clients are disconnected from the overlay entirely** at any moment — which is the second reason the server path can never be removed.
- **Blocks downloaded in order**, peers are *not* told which blocks a peer holds, and clients signal request *urgency* rather than enforcing strict reciprocity. That inversion is the interesting one: BitTorrent optimizes for swarm throughput, so rarest-first and tit-for-tat make sense. Spotify optimizes for *this listener's next few seconds*, so in-order and urgency-signaled is correct. (The paper never uses the phrase "tit-for-tat", and its peer-utility heuristic does include bytes sent and received — so "no reciprocity at all" would be too strong.)
- **Two peer-discovery mechanisms run simultaneously** on every track: a server-side tracker that remembers only 20 peers per track and returns 10 online ones, plus a Gnutella-style two-hop broadcast query in the overlay neighborhood. Measured: 75.1% of searches found peers via both, 9.0% tracker-only, 7.0% overlay-only, and **8.9% found no peers at all** — which is why the server path can never be removed.

Why it was retired in April 2014. Spotify never published a rationale. The reasons in circulation come from press coverage: sufficient owned and CDN capacity globally, P2P adding overhead on top of

direct downloads, and a codebase that had to be maintained and shipped with every client release.³ One structural reason is solid, though, because it is in the 2010 paper itself: **mobile clients never had P2P**. As listening shifted to mobile, the P2P tier covered a shrinking share of traffic regardless of anything else.

THE TRADE — RUNNING A PEER-TO-PEER TIER AT ALL	
CHOSEN	A P2P overlay carrying the bulk of bytes, with servers reserved for the latency-critical head of each track.
OVER	Serving everything from owned servers and CDN — which is what they eventually returned to.
COSTS	A P2P client that had to be written, maintained and shipped inside every desktop release. An overlay whose peer discovery found nothing 8.9% of the time, so the server path could never be removed. A swarm fragmented by audio quality, because there was no server-side re-encoding. And no mobile support at all.
REVERSES WHEN	<i>bandwidth is not your dominant cost, or your clients are short-lived, mobile, or few. Every condition that made P2P pay here — long-lived desktop clients on good connections, expensive egress, a catalog with a very popular head — weakened over the following four years. That is a better account of the retirement than any single reason in circulation.</i>

5 • Why audio is a structurally easier problem than video

The 2010 paper compares itself to video-on-demand directly: lower bitrate, shorter objects, far more objects, and a different access pattern — more active users, who replay favourites often.²

Concretely: a 320 kbps track is roughly **two orders of magnitude** below a 4K video bitrate. So a whole track fits comfortably in a modest cache, and a year of listening fits on a laptop. That is why the cache served 55.4% of bytes — and why Spotify needs no segment-level ABR ladder, no multi-resolution encodes, and no per-title encoding, all of which Chapter 3 shows YouTube requiring.

Quality tier	2010 (desktop)	Current
Default	Ogg Vorbis q5, VBR, ~160 kbps	~160 kbps ("High")
Premium high	q9, ~320 kbps	~320 kbps ("Very High")
Web player	—	AAC 128 (Free) / AAC 256 (Premium)
Low bandwidth	96 kbps floor	~24 / ~96 kbps
Lossless	—	FLAC up to 24-bit/44.1 kHz (Premium)

One consequence of the 2010 design worth noting: there was no server-side re-encoding, so a q9 client could not serve a q5 requester — which **fragmented the swarm per quality level**. A neat example of how a codec decision reaches into the distribution topology.

6 • Backend evolution, and a Kafka story that is usually misquoted

The monolith-to-microservices move was driven by *organization*, not scale. The stated figures: **810 active services**, about ten systems per squad, 90+ teams, 600+ developers, five offices, one product. The driver was autonomous full-stack teams — and the costs were named candidly: harder to monitor, needs discovery tooling, and **increased latency**.⁴

On event delivery, the precise history matters because the summary version is wrong:⁵

- They *evaluated* **Kafka 0.8** for a new event-delivery system and it failed in production: MirrorMaker mirrored cross-datacentre only on a best-effort basis and **silently dropped data while acknowledging the source**; MirrorMaker leader confusion stalled mirroring; and the producer could enter an unrecoverable state if a broker restarted, requiring a full service restart.
- Pub/Sub load test: production peak was ~700K events/sec; they pushed **2 million events/sec** from a single datacentre across 29 machines with near-zero 5xx. A five-day consumer test at ~800K msg/sec averaged ~20 s median end-to-end latency *including backlog recovery*, with zero lost messages.
- They had to write their own asynchronous Java Pub/Sub client, because the auto-generated one was too slow.
- Architecturally: **one topic per event type**, stated as a prerequisite for real-time use cases.

FLAG: "SPOTIFY REPLACED KAFKA WITH PUB/SUB" IS A MISLEADING SUMMARY

They evaluated *Kafka 0.8* for *one new subsystem*, hit specific bugs — several of which the post itself notes were fixed in 0.9 — and chose Pub/Sub. This is not a verdict on modern Kafka, and it is not a company-wide replacement.

It is, however, a good lesson in evaluating a dependency on the failure mode that matters. The disqualifying bug was not slowness. It was **silent data loss with a successful acknowledgment** — the one failure you cannot detect from the outside.

On Cassandra: the January 2015 post describes Kafka into Storm and Hadoop into Cassandra, with two clusters (entity metadata, user profile), chosen for horizontal scale, cross-datacentre replication and tunable consistency.⁶ The best concrete lesson in it: switching a table to date-tiered compaction only worked after they **split TTL'd rows from non-TTL'd rows into separate column families** — SSTables-per-read went from a flat histogram to a 6:1–12:1 ratio of single-SSTable reads. *Mixing data with different lifetimes in one physical structure defeats any compaction strategy that assumes a lifetime.*

But note: the 2017 Scio post lists **Cassandra → Bigtable** as part of a managed-services consolidation, alongside Kafka → Pub/Sub and Hive → BigQuery. So "Spotify runs on Cassandra" is a 2015 statement that has been superseded.

7 • Two batch-processing results worth stealing

Spotify's personalization is overwhelmingly **batch**, and the cadence gives it away — a weekly playlist is a scheduled job, not a serving-time model. Discover Weekly was explicitly powered by Scalding, later Scio on Cloud Dataflow.⁷ The collaborative filtering underneath is matrix factorization on implicit

feedback — play counts, playlist adds, skips — producing user and item embeddings, with nearest-neighbor retrieval via Spotify's own Annoy library.⁸ Home-page ranking is a contextual bandit that jointly learns item, *explanation* and context.⁹

Two engineering results from the Wrapped work are more broadly useful than any of that:

MOVE THE JOIN TO WRITE TIME

For Wrapped 2020, **Sort Merge Bucket** joins — bucketing and sorting by hex `user_id` at write time — let them join roughly **1 PB without a conventional shuffle**, at about 50% lower cost than the prior approach, and avoided scaling a Bigtable cluster to ~1,500 nodes at peak.¹⁰

The general form: if you know the join key in advance, pay for the ordering when you write the data, not every time you read it.

MAKE CONCURRENCY A SCHEMA PROBLEM, NOT A LOCKING PROBLEM

The 2025 Wrapped Archive pre-generated roughly **1.4 billion reports** for ~350M users over four days of continuous batch work before launch, served from a column-oriented key-value store keyed by user — with **one column qualifier per date**. Because the five parallel per-day writers for a given user touch *different cells*, there are no locks and no read-modify-write.¹¹

This is the most elegant single idea in the chapter. The contention was designed out of existence by choosing the key layout, rather than managed with coordination.

8 • Trade-offs

The design bought	And paid
265 ms median playback start	A dedicated low-latency server path that can never be removed — 8.9% of peer searches found nothing
Only 8.8% of bytes served from Spotify's own servers	A P2P client that had to be maintained and shipped with every release, never worked on mobile, and was eventually deleted
Half of all bytes from local disk	Up to 10 GB of the user's disk, and a cache that fragmented per quality level
Autonomous teams shipping independently	810 services, harder monitoring, discovery tooling, and explicitly increased latency
Zero-loss event delivery at 2M/sec	Writing their own client library, and a migration away from a system that acknowledged writes it had dropped
A 1 PB join with no shuffle	The write path must know the join key in advance — flexibility traded for cost

WHAT TRANSFERS

1. **Split the object into a latency-critical head and a bulk tail, and route them differently.** A warm long-lived connection with a large congestion window for the first piece; the cheap path, throttled, for everything else.
2. **Prefetch on a predictable next-item signal — and be able to measure the counterfactual.** 265 ms versus 390 ms is only knowable because the mechanism could be off.
3. **Move the join to write time.** Bucket and sort by the known key when you produce the data.
4. **Design contention out via the key layout.** One column per date means parallel writers never touch the same cell — no locks, no read-modify-write.
5. **Don't mix lifetimes in one physical structure.** Splitting TTL'd from non-TTL'd rows is what made compaction work at all.
6. **Evaluate dependencies on the failure you cannot detect.** Silent data loss behind a successful ack is disqualifying in a way that slowness never is.
7. **Optimize for your access pattern, not the textbook one.** In-order blocks and urgency signaling beat rarest-first and tit-for-tat, because the goal was this listener's next few seconds rather than swarm throughput.

NUMBERS TO BE CAREFUL WITH

"Spotify starts playback in under 250 ms." Not a published figure. The only primary number is 265 ms median / 515 ms p75 / 1,047 ms p90 — from 2010, on desktop, in the P2P era, and dominated by DRM and local processing rather than network.

"80% of Spotify's traffic was P2P." Contradicted by the only measurement: 35.8%. The 55.4% cache share is what this is being conflated with.

"Spotify runs 1,600 / 2,000 / 4,000 microservices." These circulate widely with no first-party source. The only published count is 810, from 2015.

"Discover Weekly uses NLP crawlers over music blogs plus CNNs on spectrograms." This three-model story originates in a widely-shared third-party explainer, not a Spotify publication. Spotify's own material supports implicit-feedback matrix factorization and batch processing.

PRACTICE

1. Design the head/tail split for a different product — a photo feed, a document editor, a game asset loader. What is the head, how big is it, and what serves the tail?
2. 8.9% of peer searches found no peers. Design the fallback so the user cannot tell, and state what it costs you in server capacity.
3. Work out why audio needs no segment-level ABR ladder but video does. Be quantitative, and find the bitrate at which your answer flips.

4. Your batch job joins two 500 TB datasets daily and the shuffle dominates the cost. Apply the write-time bucketing idea. What has to change upstream, and what flexibility do you lose?
 5. You need to pre-generate a personalized report for 350 million users before a fixed launch moment. Design the write path so parallel writers never contend, and the read path so launch-minute traffic does not melt it.
-

SOURCES

1. Neo Kim & Hayk Simonyan, "System Design Interview Question: Design Spotify", The System Design Newsletter #93, Oct 2025 (partly paywalled)
2. Kreitz & Niemelä, "Spotify — Large Scale, Low Latency, P2P Music-on-Demand Streaming", IEEE P2P '10, Aug 2010
3. TorrentFreak and TechCrunch coverage, Apr 2014 (no Spotify primary source exists for the retirement rationale)
4. Kevin Goldsmith, "Microservices @ Spotify", GOTO Berlin 2015
5. "Spotify's Event Delivery: The Road to the Cloud", Parts I-III, engineering.atspotify.com, 2016; "Life in the Cloud", Nov 2019
6. "Personalization at Spotify using Cassandra", engineering.atspotify.com, Jan 2015
7. "Big Data Processing at Spotify: The Road to Scio", engineering.atspotify.com, Oct 2017
8. Chris Johnson, "Logistic Matrix Factorization for Implicit Feedback Data", NIPS 2014 workshop; github.com/spotify/annoy
9. McNerney et al., "Explore, Exploit, and Explain: Personalizing Explainable Recommendations with Bandits", ACM RecSys 2018
10. "How Spotify Optimized the Largest Dataflow Job Ever for Wrapped 2020", engineering.atspotify.com, Feb 2021
11. "Inside the Archive: 2025 Wrapped", engineering.atspotify.com, Mar 2026
12. Spotify Newsroom company info (Q2 2026); support.spotify.com audio quality page

Tinder

Find people near me, on a planet that is mostly ocean and partly Manhattan.

AT A GLANCE

Swipes per day	1.6 billion
Matches per day	~26 million
Microservices	~500, communicating via a service mesh
Shards queried for a 100-mile radius	3 out of 55 geoshards
Primary store	DynamoDB, with Dynamo Streams propagating changes
Geospatial index	Google S2 — hierarchical, 64-bit cell IDs, Hilbert-curve ordering

Every geospatial system faces the same first decision, and the obvious answer is wrong. Divide the world into an evenly spaced grid and most of your shards hold nothing — the ocean — while a handful hold everyone.

1 • Requirements

Functional. Create and update a profile. See profiles of people nearby, filtered by preferences. Swipe. Be notified in real time when a swipe is mutual. Change your location deliberately, and have the world's view of you follow.

Non-functional. Nearby-user queries must be fast enough to feel instantaneous while swiping. Reads vastly outnumber writes. Load is *geographically* and *temporally* uneven — which turns out to be the harder of the two.

2 • Architecture, following one user through it

The source tells this as a narrative, and it is a good structure to borrow.¹

User information is stored in a key-value database — DynamoDB — with **Dynamo Streams** pushing table changes out to other systems automatically. The same information also goes onto a message queue to update the location index. Public APIs are exposed through an API gateway acting as the central entry point, handling authorization and security rules. Behind it, roughly **500 microservices** communicate through a service mesh.

CHANGE DATA CAPTURE AS AN INTEGRATION PATTERN

Dynamo Streams here is doing the job Airbnb's SpinalTap does (Chapter 6) and that Bluesky's crawler does (Chapter 15): **the database's change log becomes the integration point**, rather than every writer remembering to notify every interested system.

This is worth naming as a pattern because the alternative — application code publishing events alongside its writes — is almost impossible to keep correct. There is always one code path that writes without publishing. Instagram's cache-invalidation service (Chapter 5) is the same idea applied to cache coherence.

3 • The hard part: sharding the surface of the Earth

Finding nearby people from raw latitude and longitude does not work at scale. The naive fix — divide the world map into evenly spaced grids — is explicitly rejected, and the reason is the recurring villain of this book:¹

UNIFORM GRIDS GUARANTEE HOT SHARDS

Grids in the ocean will be empty. Grids in big cities will hold enormous numbers of users. A hot shard is an excessive load on a single partition, and a uniform grid over a non-uniform distribution produces them by construction.

Note that this is the *same problem* as Chapter 1's celebrity accounts and Chapter 14's city-keyed sharding. The lesson generalizes: **shard on the dimension the load actually varies along, and let cell size adapt to density.**

S2, and why hierarchy is the answer

Tinder uses Google's **S2** library — a square-shaped hierarchical geospatial indexing system — both to recommend people in real time and to shard the location database.¹

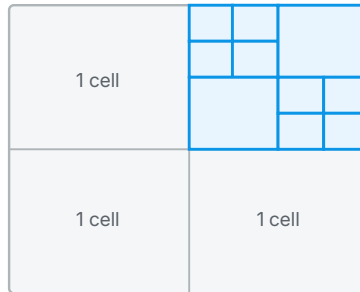
- **It divides Earth's surface into cells on a flat grid, each with a unique 64-bit integer identifier.** A small cell represents a small area.
- **It is hierarchical.** Cell size varies from square centimetres to square kilometres — so you choose a granularity that puts a manageable number of users in each shard, and dense areas can use finer cells.
- **It supports the two operations you need:** find the cell containing a given latitude and longitude, and find the cells surrounding a given cell.
- **It is based on the Hilbert curve** — "a line that covers every point in a square by folding and looping in a special way". The property that matters: **two points close on the Hilbert curve are also close in physical space.** It preserves spatial locality.
- Each small Hilbert curve segment is an S2 cell; four adjacent cells form a bigger cell; a quadtree — a tree where each node has exactly four children — represents the 2D curve.

UNIFORM GRID — HOT SHARDS

0	0	0	0	0
0	2			0
0		4.1M	2.8M	0
0	1			
0				0

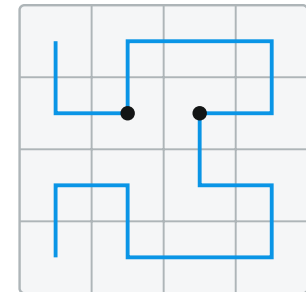
most cells hold nothing;
two cells hold a city

HIERARCHICAL CELLS — BALANCED



dense areas subdivide;
each cell holds a similar count

HILBERT ORDER PRESERVES



adjacent on the curve →
adjacent on Earth → same shard

Result: users physically close to each other land in the same shard, so ~3 shards answer a 160 km radius query instead of all of them.

Figure 9.1 — Three properties doing three jobs. Hierarchy balances cell population against real population density. The Hilbert curve makes the 64-bit cell ID a locality-preserving sort key, which is what lets it double as a shard key. And the surrounding-cells operation is what turns "within 160 km" into a small set of shard lookups.

The payoff, stated concretely: **users physically closer to each other are stored in the same database shard**, which reduces the need to query many shards to find nearby users. Tinder's own engineering write-up puts a number on it — a query with a 100-mile (~160 km) circle "will only look up **3 out of 55 geoshards**" — and the results are then filtered by user preferences before being recommended.^{1,2}

THE TRADE — SHARDING ON GEOSPATIAL CELLS

CHOSEN	Hierarchical S2 cells, ordered along a Hilbert curve, serving as both the geospatial index and the shard key.
OVER	A uniform grid, or sharding by city or country.
COSTS	Cell size becomes a permanent tuning parameter that has to be revisited as population density shifts — and changing it means resharding. Shard keys stop corresponding to anything a human recognizes, so "which machine holds Denver" no longer has a simple answer, which makes operational debugging harder. And a query near a cell boundary still fans out across several shards.
REVERSES WHEN	<i>your entities really are evenly distributed, or the whole set fits comfortably on one shard. A uniform grid is simpler and adequate right up to the point where it isn't — and the point where it isn't arrives without warning.</i>

4 • The ordering bug hiding in "change my location"

Tinder's Passport feature lets a user change their location. That requires **adding the user to the new location index and removing them from the old one** — and these operations are not atomic.¹

The source gives the exact failure: a user updates their profile to a new location, then immediately changes back to the original. Their data still pointed to the new location, because the operations were not executed in the same order they were issued.

THE FIX: MAKE ORDERING A PROPERTY OF THE TRANSPORT

Route all updates for the same user to **the same Kafka partition**. Consumers acquire locks on partitions to avoid contention. The result is a FIFO queue with ordering guarantees and very high throughput — and Kafka is checkpointed, so processing resumes after a crash.¹

The generalizable form is worth stating carefully, because it is one of the most useful tools in distributed systems: **if you partition a stream by an entity key, you get serialized processing per entity for free**, without any locking in your application. WhatsApp reached the same conclusion by a different route — hashing a record onto a single process on a single node (Chapter 2). Two different technologies, one idea: *route all operations on an entity through one place, and ordering stops being a problem you have to solve.*

5 • The match path

Swipes go to a data stream — Amazon Kinesis. **Match workers** read the stream and check for a match against the **Likes cache**, which stores who each user has liked. When there is a match, users are notified over **WebSockets** for a real-time experience.¹

Disliked profiles are written to object storage — S3 — and used for analysis to improve recommendations.

WHY A MATCH IS A STREAM-PROCESSING PROBLEM AND NOT A DATABASE QUERY

"Did B already like A?" looks like a point lookup. But at 1.6 billion swipes a day — about 18,500 per second on average — every swipe would be a read plus a conditional write against a hot store, and the two sides of a mutual like can arrive milliseconds apart.

Putting swipes on a stream and letting dedicated workers evaluate them against a cache does three things: it absorbs the burst, it makes the match check cheap, and it decouples notification latency from swipe latency. The swipe returns immediately; the match arrives over a socket a moment later.

6 • The second hard part: hot shards from time zones

This is the most instructive part of the chapter, because the problem is invisible until you think about it.

Load on a shard is a function of the unique users in it — and **users within a shard are usually in the same time zone**, because the shard key is geographic. So peak traffic differs across locations by hour of day, and traffic across shards is unbalanced in a rolling wave that follows the sun.¹

THE FIX: BREAK THE CORRELATION BETWEEN SHARD AND MACHINE

A single shard per physical server would make it worse. So Tinder **randomly assigns many shards to the same physical server**, which means each server holds shards from many time zones and its aggregate load is smoothed.

The principle: **if your partition key correlates with a load pattern, do not let it also determine physical placement**. Decouple the logical partition from the physical host and you can mix uncorrelated load onto the same hardware.

The remaining mitigations are the standard toolkit, and note that read and write hot shards get *different* answers:

- **Reads:** most operations at Tinder are reads, so a Redis cache with the **cache-aside** pattern — check the cache, fall back to the database on a miss, then populate the cache. This solves the read hot-shard problem.
- **Writes: rate limiting** handles the write hot-shard problem. You cannot cache your way out of a write hotspot; you can only throttle it.
- **Failures: exponential backoff with jitter** — and the word *jitter* is the load-bearing one. Without it, every client that failed together retries together, forever.
- **Drift:** a background job runs periodically to synchronize the data stores. With this many derived indexes, they will diverge; the answer is to reconcile continuously rather than to pretend they will not.

7 • Trade-offs

The design bought	And paid
~3 shards queried instead of all of them	A hierarchical cell scheme to maintain, and cell-size decisions that must be revisited as density changes
Correct ordering of location changes	All updates for a user funnel through one Kafka partition — a per-entity throughput ceiling
Smoothed load across time zones	Shard placement is deliberately randomized, so you lose the ability to reason about which machine holds what
Real-time match notification	WebSocket connections to maintain, plus a stream-processing tier between swipe and notification
Read scaling via cache-aside	Staleness in the nearby-user list, and no help at all for write hotspots
~500 microservices with independent ownership	A service mesh, and the debugging burden that comes with 500 hops

WHAT TRANSFERS

1. **Never use a uniform grid over a non-uniform distribution.** Hierarchical cells let granularity follow density, which is the only way cell population balances.
2. **A locality-preserving space-filling curve turns 2D proximity into a 1D sort key** — which is what lets one identifier serve as both index and shard key.
3. **Partition a stream by entity key and per-entity ordering becomes free.** No application locking, and crash recovery comes with the checkpoint.
4. **Decouple logical partition from physical placement.** If the shard key correlates with a load pattern, randomized assignment mixes uncorrelated load onto each host.
5. **Read hotspots and write hotspots need different answers.** Cache the reads; throttle the writes. You cannot cache a write.
6. **Exponential backoff without jitter is synchronized retry.** The jitter is the mechanism, not a refinement of it.
7. **Use the database's change log as your integration point.** Application-published events always eventually miss a code path.
8. **Assume derived stores will diverge and reconcile continuously.** A periodic sync job is a design component, not an admission of failure.

PRACTICE

1. Choose a cell size for a nearby-user query with a 160 km radius. How many cells must you fetch? What happens to that number if you halve the cell size, and what do you gain?
2. Design the location-change operation to be atomic without Kafka. Two approaches: make the write idempotent, or make the index derive from a single source. Work out both.
3. The Likes cache is the hottest thing in the system. What is in it, how big is it, how is it partitioned, and what happens when it is lost?
4. Your shard key is `country_code`. Enumerate the hot-shard problems this creates, and fix each one.
5. 1.6 billion swipes per day. Compute average and peak rates, size the stream, and decide how many match workers you need and on what signal you scale them.

SOURCES

1. Neo Kim, "How Tinder Scaled to 1.6 Billion Swipes per Day", The System Design Newsletter #40 — drawing on "Tinder and DynamoDB: It's a Match!" (AWS re:Invent 2017), "Building resiliency at scale at Tinder with Amazon ElastiCache", Google's S2 geometry documentation, and "Using Apache Kafka as a Scalable, Event-Driven Backbone for Service Architectures"
2. "Geosharded Recommendations, Part 1: Sharding Approach", Tinder Engineering — the primary source for the S2 scheme and the 3-of-55 geoshard figure

Zoom

Three hundred million video calls a day, on servers that barely look at the video.

AT A GLANCE

Video calls per day	300 million (2020)
Video encoding	Scalable Video Coding — one layered stream, not many streams
Server role	Routing only. No transcoding, no compositing.
Transport	UDP primary, with TCP / HTTPS / HTTP fallback
Two-participant calls	Peer-to-peer, bypassing the server entirely
Topology	Meeting zones, each managed by a zone controller

Zoom's architecture is the clearest example in this guide of scaling by *refusing to do work*. Three things a video conferencing server might plausibly do — transcode, composite, and process streams — it does none of.

1 • Requirements

Functional. Many participants see and hear each other in real time, on devices ranging from phones on mobile data to desktops on fibre to conference-room hardware.

Non-functional. Latency low enough for natural conversation — a few hundred milliseconds end to end, far tighter than any streaming system in this book. Graceful degradation rather than failure when the network is poor. And scale: 300 million calls a day means server cost per participant-minute is the whole business model.

2 • The five decisions

1 • Layered encoding instead of multiple streams

Different devices need different resolutions — adaptive streaming, adjusting quality to device type and bandwidth. But **sending many separate streams at different resolutions is not scalable**, because the count multiplies with participants.¹

So Zoom uses **Scalable Video Coding**. The source's analogy is the right one: a video in Lego blocks, where the lower blocks contain the basic picture and the upper blocks add detail. A *single* video stream is divided into hierarchical layers, each holding a different resolution. Lower layers carry basic information; upper layers carry extra information for higher resolution. **The receiving client decodes only the layers matching its device.**

Two consequences: bandwidth drops because there is only one stream, and server CPU drops because there is no need to encode and decode many streams.

CONTRAST WITH CHAPTER 3

YouTube encodes a *ladder of independent renditions* and the client picks one per segment. Zoom encodes *one layered stream* and the client picks how deep to decode.

The difference is driven entirely by the latency budget. YouTube can afford to transcode a file into six variants over minutes, because playback happens later. Zoom cannot — the encode is happening live, on a participant's laptop, with a few tens of milliseconds to spare. When you cannot afford N encodes, you need an encoding where one output serves N consumers.

2 • Separate processing from routing — and do neither on the server

Zoom separates video stream processing from routing, and does not process streams on the server because that is not scalable. **The server only routes; the client processes.**¹

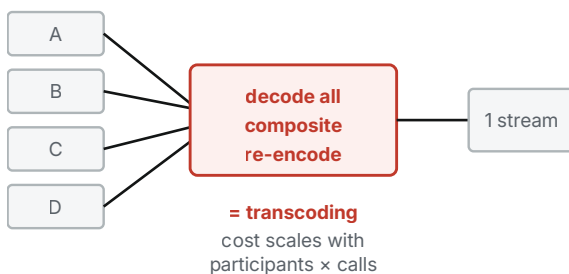
3 • Do not composite on the server

The obvious design for a grid of faces is to combine the participants' streams into one video on the server and send that. Zoom does not. Instead, **separate streams from each participant are sent to the client, which decodes them.**

The reason is precise: combining streams means **transcoding** — converting video to a different format — and transcoding is the most expensive thing a video server can do. By refusing to composite, the server never transcodes.

What the server does instead is **multimedia routing**: finding the best network paths to send video between participants with low latency.

SERVER-SIDE COMPOSITING (rejected)



ZOOM — ROUTE ONLY

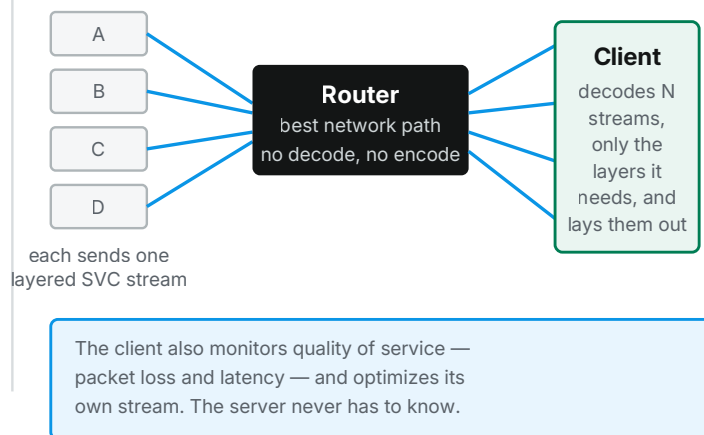


Figure 10.1 — The server's job is reduced to finding good paths for opaque packets. Every expensive operation — decode, composite, encode, quality adaptation — happens at an endpoint, where the cost is paid by hardware Zoom does not own.

4 • Let the client monitor quality

The network is unreliable, especially on mobile. So the **Zoom client** monitors quality of service by measuring packet loss and latency, then optimizes the video stream to give the best experience.¹

That placement is not incidental. The client is the only place with ground truth about what actually arrived — the server knows what it sent, which is a different and less useful fact. Compare Chapter 3: the whole adaptive-bitrate literature puts the decision at the client for the same reason.

5 • UDP first, with a fallback ladder

Video calls need fast delivery, so Zoom uses **UDP** — "a person sending postcards without a confirmation recipient", lightweight and connectionless. But UDP is often blocked on corporate and campus networks, so the client falls back through **TCP, HTTPS and HTTP** for a consistent experience.¹

WHY UDP IS RIGHT HERE AND WRONG IN CHAPTER 2

TCP guarantees delivery by retransmitting lost packets. For a message, that is exactly what you want — WhatsApp must never lose one. For live video it is actively harmful: a retransmitted frame arrives after the moment it belonged to, so you have paid latency for data that is now useless.

Late is worse than missing, for real-time media. Which is why a dropped video frame is a brief artefact and a dropped message is a bug.

3 • Topology

Two participants only: **peer-to-peer**, which reduces server load and lowers latency — there is no reason to involve infrastructure at all. Above two, a client-server architecture with microservices running on AWS.¹

Clients connect to the **closest data center** for low latency. Servers are grouped into **meeting zones**, and a **zone controller** manages every activity occurring within a zone.

THE TRADE — A SEPARATE PATH FOR THE TWO-PERSON CALL

CHOSEN	Peer-to-peer for two-participant calls; routed client-server above two.
OVER	One routed path for every call size.
COSTS	Two code paths, with different NAT-traversal behavior, different failure modes and different debugging stories — plus a transition between them when a third person joins mid-call. And the rare path is the less-exercised path, which is where incidents come from.
REVERSES WHEN	<i>the degenerate case is not dramatically cheaper, or is rare enough that a second code path costs more in defects than it saves in servers. Special-casing earns its keep only when the special case is both common and much cheaper — here it is both.</i>

The source's closing assessment is the honest one: they engineered Zoom for video streaming by keeping its architecture simple. Almost every decision in this chapter is a decision *not* to add a capability.

4 • Trade-offs

The design bought	And paid
Servers that never transcode	Clients must decode N streams and lay them out — so the CPU cost lands on the participant's device, which is why large calls heat up laptops
One stream serves every resolution	Layered encoding is more complex than plain encoding and slightly less efficient than a dedicated encode at each resolution
Low latency via UDP	No delivery guarantee, and a fallback ladder that must be built, tested and maintained for four transports
Quality adaptation where the ground truth is	Proprietary client algorithms — so quality depends on client version, and the server cannot reason about the call
Zero server cost for two-party calls	Two code paths — P2P and routed — with different NAT-traversal and failure behavior
A simple architecture	Features that need server-side access to media — recording, live transcription, cloud effects — sit outside this design and must be built separately

WHAT TRANSFERS

1. **Move work to the client when the client has the information and the capacity.** Quality adaptation belongs where the packets actually arrive, not where they were sent from.
2. **When you cannot afford N encodes, find an encoding where one output serves N consumers.** Layered coding is the generalization; the specific technology matters less than the shape.
3. **Refuse the expensive operation rather than optimizing it.** Zoom does not have a fast transcoder. It has no transcoder.
4. **For real-time media, late is worse than missing.** Choose the transport by whether stale data has value.
5. **Always provide a fallback transport ladder.** Your preferred protocol will be blocked by someone's network, and "it doesn't work at my office" is indistinguishable from "it doesn't work".
6. **Special-case the trivial case.** Two participants need no server at all. A great many systems have a degenerate case that deserves its own cheap path.
7. **Simplicity is a scaling strategy.** Most of the decisions here are subtractions, and the result serves hundreds of millions of calls a day.

PRACTICE

1. A 50-person call. Compute the bandwidth and decode cost at a client under (a) server-side compositing and (b) Zoom's model. At what participant count does the client become the bottleneck, and what would you do about it?
 2. Design cloud recording for this architecture. The server cannot decode — so where does the recording come from, and what does it cost?
 3. A participant's packet loss rises to 8%. Walk through exactly what the client should do, layer by layer, and what the other participants see.
 4. Your UDP is blocked and you fall back to TCP. Describe precisely how the call degrades, and what you would change to soften it.
 5. Compare Zoom's SVC approach with YouTube's rendition ladder on five axes: encode cost, bandwidth, client cost, latency, and quality. Then state the condition under which each wins.
-

SOURCES

1. Neo Kim, "5 Reasons Why Zoom Was Able to Support 300 Million Video Calls a Day", The System Design Newsletter #28

Figma

Five ways to scale Postgres, applied in order. The order is the lesson.

AT A GLANCE

Starting point	A single Postgres database holding all metadata
Users at the time of the write-up	4 million; later 40 million+
What hit the wall first	CPU, then storage capacity, then IOPS
Techniques applied	Replication → federation → pooling → vertical partitioning → horizontal partitioning
Connection pooler	PgBouncer
Conclusion	"Postgres can easily handle internet-scale traffic"

This is the most immediately useful chapter in the book, because it is the only one describing a journey you are likely to actually make. Nobody is going to ask you to build a transcoding ASIC. Somebody will eventually ask you why the database is slow.

1 • Where it started, and what broke

Figma stored metadata — file information, user comments — in a **single Postgres database**, for simplicity. Traffic was massive, and high CPU usage affected latency and performance. So they scaled *vertically*: same design, bigger machine, more CPU.¹

That bought time and created two new problems, and both are worth stating precisely because they are different failures:

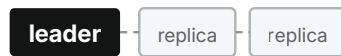
Problem	Why vertical scaling could not fix it
Storage capacity	Some tables grew extremely fast — and the smallest unit of partition is a table . A single database could not hold what one table needed.
Performance (IOPS)	Write volume exceeded the input/output operations per second limit of a single database. You cannot buy your way past an IOPS ceiling indefinitely.

"THE SMALLEST UNIT OF PARTITION IS A TABLE"

This sentence is the hinge of the entire chapter. It means that until you are willing to split a *single table* across databases, your largest table is your hard limit — no amount of moving other tables elsewhere helps. Techniques 1 through 4 below all work around this constraint. Technique 5 breaks it, which is why it is both the most powerful and by far the most expensive.

2 • The five techniques, in the order applied

1 Replication



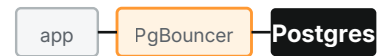
reads to replicas · writes to leader
but: replication lag → strong-consistency reads go to the leader

2 Federation



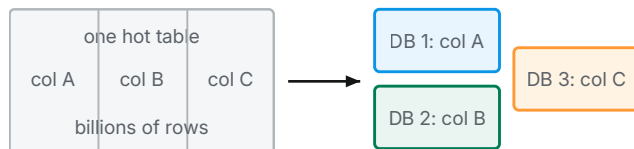
tables split by domain — they grow at different rates
and have different workloads

3 Connection pooling



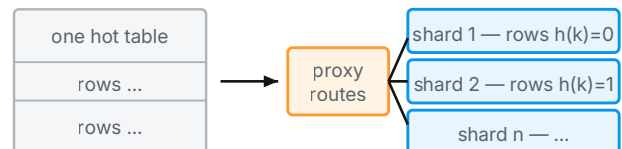
a TCP proxy holding a pool of connections; clients reuse them instead of reconnecting

4 Vertical partitioning — move columns out



Each table still lives entirely in one database — a table does not spread across machines. So this hits a wall under high write traffic: one database cannot hold a table with billions of rows.

5 Horizontal partitioning — split rows



Hash of the partition key spreads rows across shards. Locating tables sharded by the same key keeps joins possible.

The proxy does more than route

It rewrites expensive queries, manages database topology, supports transactions across co-located tables, and drops requests above a threshold — load shedding as a first-class feature of the data layer.

Figure 11.1 — The five techniques, in the order Figma applied them. Each is cheaper than the next; each is exhausted before the next is attempted. The jump from 4 to 5 is where the cost changes character, because it is the first technique that splits a single table.

1 • Database replication

Set up replicas and route read traffic to them. This buys three things: more read capacity, lower latency by keeping data closer to users worldwide, and fault tolerance because a replica can take over when the leader fails.¹

Two caveats are stated, and both matter:

- **Replicas cannot serve some reads** — because of replication lag and strong-consistency requirements. Those reads are routed to the leader. This is exactly Chapter 1's rung-5 problem, and Chapter 6 shows what happens in payments when you get it wrong.
- **A caching layer** was added to hold frequently accessed data and reduce database load further. Note the order: caching came *with* replication, not instead of it.

2 • Database federation

Move tables into separate databases **by domain**. This scales horizontally by adding more databases and spreads workload to reduce bottlenecks.¹

The example given is the instructive part: file information and user comments go into separate databases *because file information grows faster than comments, and the workload for each table is different*.

FEDERATE ON GROWTH RATE AND WORKLOAD SHAPE, NOT ON TIDINESS

The temptation is to federate along whatever boundary looks cleanest in the domain model. The useful criterion is different: **put tables with different growth rates and different read/write mixes in different databases**, because that is what lets you tune, size and scale them independently.

Two tables that grow at the same rate and are read the same way gain almost nothing from being separated, and lose the ability to join.

3 • Connection pooling

PgBouncer, acting as a TCP proxy that holds a pool of connections to Postgres. Clients connect to PgBouncer instead of to Postgres directly.¹

Three benefits: clients reuse the same connection, improving throughput; the number of database connections is capped, avoiding connection starvation; and persistent connections with the client avoid expensive reconnection requests.

This is the single highest-leverage item on the list relative to effort, and Chapter 12 is an entire case study about what you can build once PgBouncer is in the path.

4 • Vertical partitioning

Move *columns* from high-traffic tables into separate tables, then store those tables in different databases.¹

The crucial qualifier: **each table has its own database — a single table does not spread across many databases**. This is still working within the "smallest unit of partition is a table" constraint.

What it buys: isolation of sensitive data for security, reduced workload by routing traffic across many databases, and high query performance because a query scans only a subset of the data.

What it costs: **it becomes a bottleneck with high write traffic**, because a single database cannot store a table with billions of rows. That is the wall that forces technique 5.

5 • Horizontal partitioning

Split tables at the *row* level and store them across many databases, for maximum scalability. A proxy server routes queries to the correct database.¹

The proxy does four things, and the last one is unusual enough to be worth noting:

- Rewrites expensive queries for performance.

- Manages database topology.
- Supports transactions.
- **Drops extra requests when they exceed the threshold limit** — load shedding built into the data access layer.

Two design choices make sharding workable:

- **Route writes by the hash of the partition key**, which distributes data uniformly across databases. Contrast with range-based partitioning, which is easier to reason about but produces hotspots on monotonically increasing keys.
- **Co-locate tables partitioned by the same key**. If `file_id` shards both the files table and the comments table, then all of a file's comments sit on the same database as the file — which makes transactions and joins across them still possible. This is the single most important decision in a sharded schema, and it is easy to get wrong once and then live with forever.

3 • Why the order matters more than the list

#	Technique	Raises which ceiling?	Relative cost
1	Replication	Read throughput, availability	Low. Some application awareness of which reads must hit the leader.
2	Federation	Total storage, aggregate IOPS	Moderate. Loses cross-domain joins.
3	Connection pooling	Concurrent client count	Very low. Largely invisible to the application.
4	Vertical partitioning	Per-table storage and IOPS	Moderate-high. Schema changes; a row is now assembled from several databases.
5	Horizontal partitioning	Write throughput — the only one that does	High. A proxy layer, a shard key you live with for years, resharding, and constrained joins.

Read that ceiling column again. Techniques 1 through 4 raise read capacity, storage, and concurrency. **Only technique 5 raises the write ceiling** — and it is by a wide margin the most expensive. That is precisely why it comes last: every cheaper option is exhausted first, and many systems never need it at all.

THE COMPARISON WITH CHAPTER 3

Figma built a query-routing proxy. YouTube's answer to the same problem was Vitess — a proxy that speaks the MySQL wire protocol so applications cannot tell it is there, with online resharding that moves data with seconds of downtime rather than a maintenance window.

Two companies, the same conclusion: **the sharding logic belongs in a proxy, not in the application**. If you are writing shard selection into your application code, you are building the thing they both extracted, in the place they both took it out of.

4 • Trade-offs

The design bought	And paid
Postgres all the way to internet scale	Five layers of infrastructure around it, and a proxy that is now on the critical path for every query
Read capacity via replicas	Replication lag, and application knowledge of which reads must go to the leader
Independent scaling per domain	No joins across federated boundaries
Unbounded write throughput via sharding	A shard key chosen once and lived with; resharding; constrained transactions; and co-location decisions that shape the schema permanently
Protection from overload	Requests are dropped above a threshold — deliberate, visible failure instead of collapse
No exotic database to learn or operate	All the complexity moved into the layer <i>around</i> the database, which is where the team's expertise now has to be

WHAT TRANSFERS

1. **Exhaust the cheap techniques in order.** Replication, federation and pooling are low-cost and high-return; sharding is the expensive last resort and the only one that raises the write ceiling.
2. **Your largest table is your hard limit** until you are willing to split a single table. Everything else is working around that constraint.
3. **Federate on growth rate and workload shape**, not on domain tidiness. Separating two tables that behave identically costs joins and gains nothing.
4. **Put connection pooling in early.** It is nearly free, invisible to the application, and Chapter 12 shows it becoming a control point for far more than connections.
5. **Hash the shard key.** Range partitioning on monotonic keys produces a permanently hot shard.
6. **Co-locate tables sharded by the same key.** This is what preserves transactions and joins, and it is the decision you will least be able to change later.
7. **Put sharding logic in a proxy, not in the application.** Both Figma and YouTube reached this conclusion independently.
8. **Build load shedding into the data layer.** Dropping requests above a threshold is a feature; discovering the threshold during an incident is not.

PRACTICE

1. Your single Postgres instance is at 80% CPU and 60% of its IOPS limit, with reads eight times writes. Which technique do you apply, and what do you measure to know it worked?
2. Choose a shard key for a collaborative document editor storing documents, revisions, comments and presence. Justify your co-location choices and name the query you have just made expensive.
3. You must reshard from 8 to 16 shards with no downtime. Write the plan. Where is the cutover, and what happens to in-flight writes?
4. Vertical partitioning has moved a row's columns across three databases. A query needs all three. What have you actually built, and was it worth it?
5. Argue for skipping straight to technique 5. Under what conditions is starting with sharding the right call?

SOURCES

1. Neo Kim, "How Figma Scaled to 4M Users — Without Fancy Databases", The System Design Newsletter #69
2. PgBouncer documentation — [pgbouncer.org](https://www.pgbouncer.org)

Cloudflare

Fifty-five million requests a second, fifteen Postgres clusters, and queueing theory doing the heavy lifting.

AT A GLANCE

HTTP requests	55 million per second
Share of internet traffic	~20%
Postgres clusters	15
Database servers, including replicas	>100
Cluster topology	2 regions per cluster, 3 servers per region
Deployment	Bare metal, no virtualization
Workload	Multi-tenant OLTP — service metadata

Read Chapter 11 first, then read this one. Figma shows you where to put a connection pooler. Cloudflare shows you what you can build once it is there — and the answer involves a TCP congestion-control algorithm being used to throttle database tenants.

1 • The problem: multi-tenancy

Cloudflare uses Postgres to store service metadata and handle the online transaction processing workload. The hard part is stated plainly: **supporting many tenants with different load conditions within a cluster is a hard problem**. A tenant is an isolated data space for a specific user or group.¹

The specific difficulties compound, and it is worth following the chain:

Difficulty	Why the obvious fix does not work
Connections are expensive	Each Postgres connection is a separate operating-system process . Clients compete for them.
No global throttle value exists	Each tenant has a unique workload, so any single limit is simultaneously too tight for one tenant and too loose for another.
Manual throttling does not scale	Identifying and throttling misbehaving tenants by hand is "a huge effort" — and by the time a human is involved, the incident has happened.
One tenant can starve the rest	A single expensive query starves neighboring tenants and blocks them — the noisy-neighbor problem.

Difficulty	Why the obvious fix does not work
Queries cannot be isolated after the fact	"A query is difficult to isolate once it reaches the database server." So the intervention must happen <i>before</i> the server.

THAT LAST ROW IS THE ARCHITECTURAL INSIGHT

Once a query is executing inside Postgres, your options are bad: kill it, or wait. So everything Cloudflare built lives **in front of** the database, in the connection pooler, where a request is still a queued object you can reason about and reorder.

Generalized: **put your control point where requests are still schedulable**. Once work is in flight inside a component you do not control, you have no policy left — only cancellation.

2 • The control point: PgBouncer

PgBouncer sits in front of Postgres as a TCP proxy holding a pool of connections. Tenants connect to PgBouncer rather than to Postgres directly, which caps the number of Postgres connections and prevents connection starvation. Persistent connections avoid the cost of constantly opening and closing.¹

So far, this is Chapter 11's technique 3. What follows is what Cloudflare built on top of it.

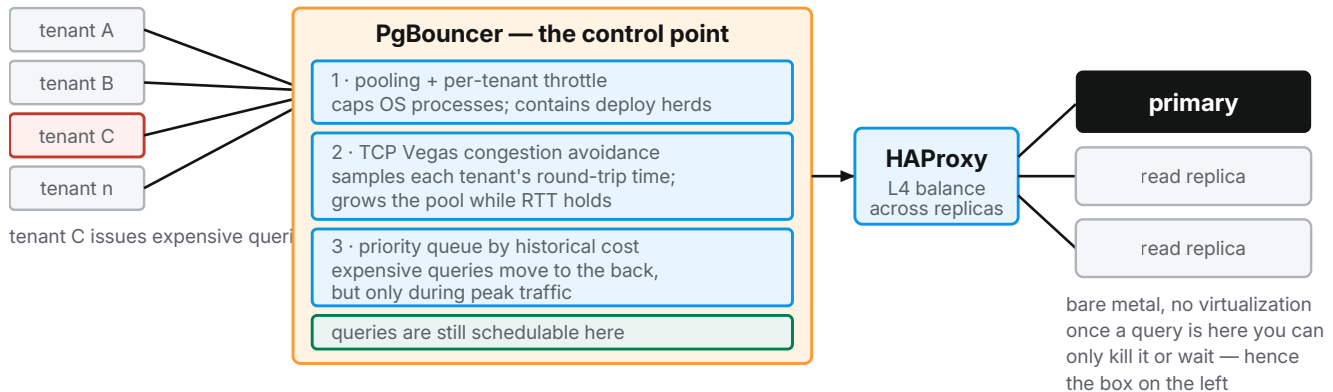
The thundering herd, in its least obvious form

Thundering herd occurs when many clients query a server concurrently, degrading performance. The trigger here is not a cache expiry — it is a **deploy**: an application's state is initialized when it is redeployed, causing it to create many connections to the database at once. Tenants then compete for connections.¹

PgBouncer throttles the number of Postgres connections created by a specific tenant, which contains it. Worth noting because it is a genuinely easy problem to miss: **your own deployment process is a load generator**.

3 • The hard part: four mechanisms from queueing theory

tenants, each with a different workload



4 • High availability — Stolon

Replicates across instances, elects leaders, performs failover during peak traffic.

Each cluster spans 2 regions with 3 servers each: writes to the primary region, async replication to the secondary, reads from the secondary. `pg_rewind` replays missing writes to a primary that returns after failover.

Figure 12.1 — Everything interesting happens to the left of the database. The queries are ordered, throttled and shed while they are still queued objects; Postgres only ever sees work that has already been scheduled.

Performance: bare metal, and an L4 load balancer

Cloudflare does not run Postgres in the cloud — it runs on **bare metal servers without virtualization**, for performance. **HAProxy** sits as an L4 load balancer: PgBouncer forwards queries to HAProxy, which balances traffic across the primary and the secondary read replicas.¹

Concurrency: TCP Vegas, used on database queries

This is the most interesting idea in the chapter. Performance degrades when many tenants issue concurrent queries — so Cloudflare uses the **TCP Vegas congestion avoidance algorithm** to throttle tenants.¹

How it works: Vegas finds the optimal number of packets that can be sent concurrently by **sampling each tenant's transaction round-trip time to Postgres**. It then **increases the connection pool size as long as the round-trip time does not degrade**. The effect is that it throttles traffic *before* resource starvation happens.

WHY THIS IS ELEGANT

Recall the earlier difficulty: no global throttle value exists, because every tenant's workload is different. Vegas does not need one. It **discovers the right limit empirically, per tenant, continuously** — by pushing until latency starts to suffer and then holding there.

And note the reuse: this is a network congestion-control algorithm applied to a database connection pool. The shapes are identical — a shared resource of unknown capacity, many competing senders, and latency as the only available feedback signal. **When your problem has that shape, thirty years of congestion-control literature is available to you.**

Ordering: priority queueing by query cost

Cloudflare ranks queries at the PgBouncer layer using queues. Queries are ordered based on their **historical resource consumption** — queries that need more resources are moved to the back.¹

And the crucial refinement: **priority queueing is enabled only during peak traffic**. In normal traffic, a query does not wait forever at the back of the queue.

The stated outcome is honest about who pays: this approach improves the latency of most queries, *yet a higher latency is observed for tenants that issue expensive queries in peak traffic*. That is the correct trade and it should be stated out loud — you are choosing to degrade the heaviest users to protect everyone else, and someone will notice.

COMPARE WITH AIRBNB'S ADAPTIVE LIFO

Chapter 6 describes a CoDel queue with adaptive LIFO: under overload, serve the *newest* request first, because the oldest have probably already been abandoned.

Cloudflare orders by *cost*; Airbnb orders by *age*. Both are rejecting FIFO, which is the default and is optimal for neither throughput nor fairness under overload. **Queue discipline is a design decision, and FIFO is a choice you should have to justify.**

4 • High availability

Cloudflare uses the **Stolon** cluster manager for Postgres high availability. Stolon replicates data across Postgres instances, elects leaders, and performs failover during peak traffic.¹

The topology: each database cluster is replicated across **two regions**, with each cluster consisting of **three database servers within a single region**. Writes are routed to the primary region; data is asynchronously replicated to the secondary; reads are routed to the secondary.

Three operational practices are named, and they are the part most organizations skip:

- **Proactive network partition detection.** They test connectivity between components to find network partitions *before* those partitions cause an incident.
- **Chaos testing** for resilience, plus extra network switches and routes to avoid partitions in the first place.

- `pg_rewrite` to synchronize clusters — replaying missing writes to a primary server that came back online after a failover. This is the unglamorous but essential piece: a recovered former primary has writes the new primary never saw, and something has to reconcile that.

THE TRADE — ASYNCHRONOUS CROSS-REGION REPLICATION

CHOSEN	Writes to a primary region, asynchronous replication to a secondary, reads served from the secondary.
OVER	Synchronous cross-region replication, or a single-region cluster.
COSTS	A regional failure can lose the most recent writes — <i>asynchronous</i> means exactly that, and no amount of tooling changes it. Reads from the secondary trail by the replication lag, so any read that gates a write must be routed to the primary anyway. And a recovered former primary holds writes nobody else ever saw, which is why <code>pg_rewrite</code> is part of the design rather than an afterthought.
REVERSES WHEN	<i>losing a single committed write is unacceptable. Then you pay synchronous replication's latency on every write and accept that a partition can stall writes outright. Chapter 6 is the case where that trade goes the other way, because there the writes are money.</i>

The source's own summary is a good one: Cloudflare uses "a combination of OS resource management, queueing theory, congestion algorithms, and even statistics" for PostgreSQL scalability. Note what is *not* in that list — a different database.

5 • Trade-offs

The design bought	And paid
55M requests/sec served with 15 Postgres clusters	A substantial control layer — PgBouncer with custom throttling, priority queueing and congestion control — that the team must own
Per-tenant limits discovered automatically	Behavior that is harder to predict and explain than a fixed limit, because the limit moves
Better latency for most queries	Worse latency for heavy tenants during peak — a deliberate, visible unfairness
Bare-metal performance	No cloud elasticity. Capacity is a procurement decision with a lead time.
Cross-region survivability	Asynchronous replication, so a regional failure can lose the most recent writes
Tenants isolated from each other	Every mechanism is a place where a legitimate query can be delayed or dropped

WHAT TRANSFERS

1. **Put the control point where requests are still schedulable.** Once work is executing inside a component you do not control, you have only cancellation left.
2. **Discover limits empirically rather than configuring them.** When no single threshold can be right for all tenants, push until latency degrades and hold there — per tenant, continuously.
3. **Borrow from congestion control.** A shared resource of unknown capacity, many competing senders, latency as the only feedback — that is a congestion problem, whatever it is called locally.
4. **FIFO is a choice.** Order by cost (Cloudflare) or by age (Airbnb), but decide deliberately — FIFO is optimal for neither throughput nor fairness under overload.
5. **Enable aggressive policy only under pressure.** Priority queueing during peak only, so nothing starves in normal conditions.
6. **Your deploy process is a load generator.** Application restart initializes state and opens connections in a burst.
7. **Detect partitions proactively and chaos-test the recovery path.** Chapter 2's 210-minute outage was a partition heal, not a load event.
8. **Plan for the returning former primary.** It has writes nobody else saw. `pg_rewind` exists because this is not a rare case.

PRACTICE

1. Implement per-tenant concurrency limits using round-trip-time feedback. What do you do on a latency spike caused by something other than your own load?
2. Design the query-cost estimator. How do you classify a query's expected resource consumption *before* running it, and what happens when you are wrong?
3. A single tenant accounts for 60% of your database load and is a major customer. Work out the policy, and then write the sentence you would say to them.
4. Your primary fails over. The old primary returns with 400 committed writes the new primary never received. What happens, and what should have happened?
5. Compare the shapes of Chapter 11 and Chapter 12. Figma grew a pooler into a sharding proxy; Cloudflare grew one into a scheduler. Why did each end up where it did?

SOURCES

1. Neo Kim, "How Cloudflare Was Able to Support 55 Million Requests per Second With Only 15 Postgres Clusters", The System Design Newsletter #32
2. Brakmo & Peterson, "TCP Vegas: End to End Congestion Avoidance on a Global Internet", IEEE JSAC, 1995; Stolon — github.com/sorintlab/stolon

PayPal

They scaled out to more than a thousand machines. Then they scaled back in to eight.

AT A GLANCE

Transactions per day	1 billion
Virtual machines serving them	8
Machines before the change	>1,000
Framework	Akka (Java) — the actor model
Threads	Proportional to CPU cores, regardless of actor count
Routing	Consistent hashing, so a customer reaches the same server

Almost every chapter in this book is about scaling out. This one is about discovering that you have scaled out too far, and that the concurrency model — not the machine count — was the problem all along.

1 • How they got to a thousand machines

PayPal scaled vertically at first, buying newer hardware as users grew. Then the single-threaded performance gains of Moore's law slowed, and newer hardware stopped solving the problem. Growth was explosive regardless — a million transactions a day within two years — so they scaled out, running services across more than a thousand virtual machines.¹

That solved capacity and produced three new problems:

Problem	What it meant in practice
Network infrastructure	A request took more network hops to complete, which worsened latency. And the network infrastructure itself became expensive to maintain.
Maintenance cost	More servers meant more infrastructure complexity. Deploying a service across every machine took longer. Autoscaling needed extra effort. Monitoring became difficult.
Resource usage	They were not fully using the CPU of each server — throughput per server was low, which meant wasted resources and extra cost.

THE DIAGNOSIS WORTH INTERNALIZING

"The code doesn't take full advantage of the hardware unless it's run concurrently."¹

A thousand machines at low CPU utilization is not a capacity problem — it is a *concurrency* problem wearing a capacity problem's clothes. If each machine is mostly idle while requests queue, adding machines buys throughput at a terrible price and leaves the actual defect untouched.

Instagram reached the same conclusion from a different direction in Chapter 5: the answer was not faster hardware but fewer CPU instructions per task and better memory layout, so each server could hold more users.

2 • The actor model, and the four things it changed

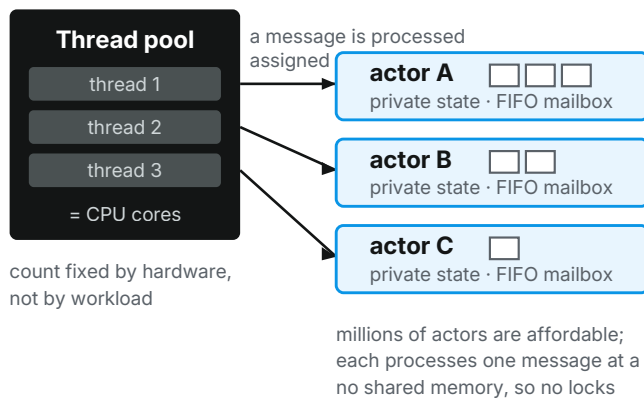
PayPal moved to the actor model on the Akka framework. An **actor** is the fundamental unit of computation: it performs an action when it receives a message, and it is decoupled from the source of that message.¹

1 • Resource usage: actors are not threads

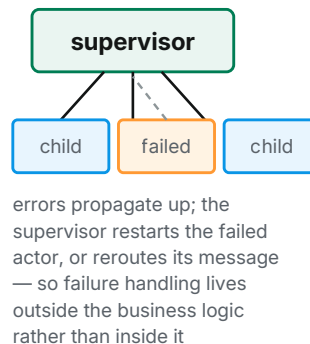
An actor is an extremely lightweight object, taking fewer resources than a thread — so it is easy to create millions of them if necessary.

The scheduling model is the key mechanism: **a thread is assigned to an actor only when that actor has a message to process**, and is released as soon as the message is handled, then reassigned to another actor. So the number of threads stays proportional to the number of CPU cores, while a small number of threads serves a very large number of concurrent actors.¹

A FEW THREADS, MANY ACTORS



SUPERVISION



STATEFUL SERVER

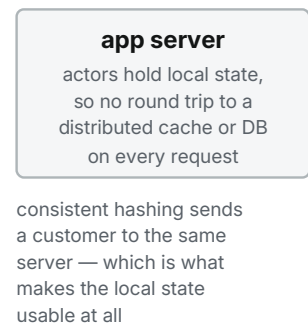


Figure 13.1 — Three properties doing three jobs. The thread pool is sized by hardware and shared across millions of actors. Supervision moves error handling out of business logic. And consistent hashing is what makes actor-local state a cache rather than a correctness hazard.

2 • State: private, and therefore useful

Actors do not share memory and are isolated from each other — an actor's state is private. They communicate only through messages, which are simple immutable data structures. Each actor has a

mailbox, effectively a message queue, and stores messages there until they are processed in FIFO order.¹

Then the consequence that actually delivered the machine-count reduction: because actors hold local state in the application server, the system avoids extra network calls to a distributed cache or a database. **A stateful application server improves performance, because it caches data locally.**

And the piece that makes that safe: PayPal uses **consistent hashing to route a customer to the same server**. Without that, actor-local state is worse than useless — it is state that might be on any machine, and therefore stale on all of them.

THIS IS A DELIBERATE REVERSAL OF THE LADDER

Chapter 1's rung 8 says: make the web tier stateless, because stateless tiers are trivial to scale and to replace. PayPal does the opposite on purpose — and gets an order of magnitude fewer machines for it.

The trade is real and should be stated. A stateful tier means sticky routing, a harder failover story (what happens to an actor's state when its machine dies?), and rebalancing work when the ring changes. PayPal accepted all of that because eliminating a network round trip per request was worth more than the operational simplicity of statelessness.

Statelessness is a default, not a law. The question is whether the network calls you avoid are worth the sticky routing you take on — and at a billion transactions a day, they were.

3 • Concurrency: parallel actors, sequential messages

Many actors can run at the same time, but **each actor processes only a single message at a time**, in order. So three messages processed in parallel requires three actors. Actors work asynchronously — they do not wait for another actor's response.¹

WHY SEQUENTIAL-PER-ACTOR IS THE WHOLE TRICK

Because an actor handles one message at a time and owns its state exclusively, **there is no concurrent access to that state — so there are no locks**. All the lock-contention pathology of Chapter 2's connection-tuning saga simply does not arise, because there is nothing to contend over.

This is the same insight as Chapter 9's Kafka partitioning and Chapter 2's "hash a record onto one process on one node". Three systems, three technologies, one idea: **route all operations on an entity to one place, and concurrency control becomes unnecessary rather than difficult.**

CHOSEN	A lightweight actor per entity, processing its mailbox strictly sequentially, on a thread pool sized to CPU cores.
OVER	Thread-per-request, or an event loop over shared mutable state.
COSTS	Actor granularity becomes a design decision you live with for years. Choose it too coarse — one actor per customer <i>segment</i> rather than per customer — and you have serialized your hot path by accident, with no lock anywhere to blame for it. Too fine, and you pay message-passing overhead for work that should have been a function call. And "no shared memory" removes data races but not protocol deadlock: two actors can still wait on each other.
REVERSES WHEN	<i>your work is a small number of long, CPU-bound tasks rather than a large number of short, independent ones. The model earns its keep on many-and-small; on few-and-large it adds indirection and buys nothing.</i>

PayPal also uses Akka's functional programming style, which prevents side effects and makes testing easier, with pluggable code pieces for easy scalability. And actors can run locally or remotely on another machine, **transparently to the system** — location transparency, which means the same code works whether the collaborator is in-process or across the data center.

4 • Fault tolerance: supervision

An actor can create more actors and supervise them. If a supervised actor fails, the **supervisor restarts it**, and the message can be routed to another actor. Errors propagate *to the supervisor*, which means graceful error handling happens without cluttering the business logic with it.¹

That last clause is the design value. In a conventional service, error handling is interleaved with the code that does the work — try/catch around everything, retry logic inline, degraded paths beside happy paths. Supervision hoists it into a separate structural layer: *children do the work and are allowed to fail; parents decide what failure means*.

3 • The honest caveats

The source is explicit that this is not a silver bullet:¹

- **A learning curve for developers.** Message passing, supervision hierarchies and location transparency are not how most engineers are trained to think.
- **Extra care is needed to prevent race conditions and deadlocks.** Worth being precise about why, since "no shared memory" sounds like it should rule them out: two actors can still each wait on a message from the other, and message ordering between *different* pairs of actors is not guaranteed. The actor model removes data races on shared state; it does not remove protocol-level deadlock.

COMPARE WITH CHAPTER 2

WhatsApp reached a very similar place — millions of lightweight isolated processes, each with a mailbox, communicating by message — via Erlang/OTP rather than Akka. The convergence is not a coincidence: both had a workload of many long-lived, mostly-idle entities each holding a little state, and that workload maps onto a per-process model almost exactly.

Then note Chapter 12's inversion. Cloudflare's hardest problem is that **each Postgres connection is a separate OS process** — the same structure that is a virtue for WhatsApp and PayPal is a liability there, because an OS process is orders of magnitude heavier than an actor. **The unit of isolation has to be cheap enough to have millions of them.** That, not the message passing, is what makes the model work.

4 • Trade-offs

The design bought	And paid
1 billion transactions/day on 8 VMs	A stateful application tier, with sticky routing and a harder failover story
No locks, because state is never shared	Protocol-level deadlock is still possible, and now harder to see
Millions of concurrent units on a core-sized thread pool	Per-actor sequential processing — parallelism requires designing the actor granularity correctly
Error handling lifted out of business logic	A supervision hierarchy that must be designed, and failure semantics that are now implicit in tree shape
Location transparency	A remote call that looks identical to a local one — convenient until you are debugging latency
Far fewer machines to operate	A learning curve, and a codebase most new hires have never seen the shape of

WHAT TRANSFERS

1. **Low CPU utilization across many machines is a concurrency problem, not a capacity problem.** Adding machines buys throughput at a terrible price and leaves the defect in place.
2. **Route all operations on an entity to one place and concurrency control becomes unnecessary.** Actor mailbox, Kafka partition, or hash-to-one-process — same idea, three implementations.
3. **The unit of isolation must be cheap enough to have millions of them.** An actor works where an OS process does not.
4. **Statelessness is a default, not a law.** If eliminating a network round trip per request is worth sticky routing and a harder failover, a stateful tier is the right call — and consistent hashing is what makes it viable.
5. **Hoist error handling into a structural layer.** Let children fail and let parents decide what failure means, instead of interleaving recovery with business logic.
6. **"No shared memory" eliminates data races, not deadlock.** Two actors can still wait on each other.
7. **Scaling in is a legitimate outcome.** A thousand machines to eight is one of the largest efficiency wins in this book, and it came from changing the programming model rather than the hardware.

PRACTICE

1. Decide the actor granularity for a payments system. One actor per customer? Per account? Per transaction? For each choice, state what serializes and what parallelizes.
2. An actor holding local state dies with its machine. Design the recovery. Where does the state come from, and what does the customer experience?
3. Construct a deadlock between two actors with no shared memory. Then fix it, and state the rule your fix generalizes to.
4. Your service runs on 300 machines at 15% CPU. Write the investigation plan, in order, before you propose any architecture change.
5. Argue against the stateful tier. What would have to be true about PayPal's workload for statelessness to be the better choice?

SOURCES

1. Neo Kim, "How PayPal Was Able to Support a Billion Transactions per Day With Only 8 Virtual Machines", The System Design Newsletter #30 — drawing on "squbs: A New Reactive Way for PayPal to Build Applications", PayPal Technology Blog, and akka.io

Lyft

Thirty services, and engineers afraid to make a network call. The fix is now in half the industry's infrastructure.

AT A GLANCE

Rides	945.5M in FY2025, +14% YoY; 51.3M annual riders
When the network pain became acute	At roughly 30 services (~2014–15)
At Envoy's open-sourcing	Thousands of nodes, >100 services, >2M requests/sec (Sep 2016)
Redis fleet	~750 instances / ~15M QPS (2017) → ~2,000 instances / ~25M QPS (2018)
Map data	OpenStreetMap, indexed with Google S2
OSM accuracy, self-reported	Core road features >95%; turn restrictions >85%

Lyft's most significant contribution to software is not a ride-hailing feature. It is a sidecar proxy, and understanding why they had to build it is a better lesson in microservices than any amount of theory.

1 • Map data: build the graph on the client

Lyft uses **OpenStreetMap** for internal map data and Google's **S2** library on top of it to index and query efficiently — dividing the map into cells, each with a unique ID. Road metadata and road-segment sequences are stored in each cell, which is what allows turn restrictions to be understood.¹

The design decision worth noting is where the graph gets assembled:

THE CLIENT BUILDS THE ROAD NETWORK GRAPH DYNAMICALLY

Rather than serving routes from a server, the client **downloads S2 cells based on the user's location** and builds the graph locally, buffering nearby cells so it has enough map data to navigate. This keeps memory usage low — you hold the city you are in, not the world.¹

Note the shape: this is Chapter 10's "move the work to the client", applied to data rather than to media. And it composes with Chapter 4's insight — the cell is the partition, so cells are both the unit of download and the unit of precomputation.

The delivery path is layered carefully:¹

- Map data is stored in **serialized form on DynamoDB**. The S2 index is queried to find the relevant DynamoDB partition; data is downloaded, deserialized and filtered before being returned.

- A **CDN caches map data**, reducing latency and improving reliability.
- A **SQLite cache on the client** prevents duplicate downloads, with the in-memory cache updated only if the map data on the server has changed. Four cache layers — CDN, client SQLite, client memory, and DynamoDB itself — each solving a different problem.

THE TRADE — BUILDING THE ROAD GRAPH ON THE CLIENT

CHOSEN	The client downloads S2 cells for its own location and assembles the road network graph locally, buffering neighboring cells.
OVER	Serving routes from a server that holds the whole graph.
COSTS	The client can be wrong about the map until it refreshes — a road closed this morning is not closed on a phone holding yesterday's cells. Four cache layers (CDN, client SQLite, client memory, DynamoDB) now have to be kept coherent, and each is a place staleness can hide. And the download itself spends a driver's mobile data.
REVERSES WHEN	<i>the data changes faster than clients refresh, or correctness depends on every client agreeing. Move computation to the client only when its input is slow-moving — a road network is; live traffic, which is why that stays server-side, is not.</i>

And the data-quality problem: OSM has missing roads and mislabeled directions. Lyft uses a **Kalman filter over weeks of driver location data** to plot missing roads and update the map. A nice inversion — the fleet's GPS traces become a mapping instrument.

2 • Finding nearby drivers: the sorted-set pattern

Driver locations live in a **Redis cluster**, with locations spread across many instances — which prevents a global write lock and contention when many rides are ordered simultaneously.¹

But sharding Redis **by region causes a hot shard**, because big cities have far more drivers. Lyft's own framing of the original problem: the driver-GPS store was sharded by city, and San Francisco is not a small market. So the shard key became a **geospatial cell** rather than an administrative unit — load then follows geography at a granularity fine enough that no single cell holds a large share of the fleet.^{1,2}

Flag: the specific figures usually quoted here — "geohash level 5", "~1 km² cells", "125,000 GPS writes per second" — appear in secondary write-ups but not in any retrievable Lyft document. The *mechanism* is well sourced; those numbers are not. Treat them as plausible illustration, not citation.

The liveness problem, and the elegant answer

The Redis cluster may contain drivers who stopped driving hours ago. You want only *active* drivers — those still reporting.

Approach	Mechanism	Problem
Rotating time buckets	Create in-memory buckets periodically, store active drivers in the current one, and remove old buckets every 30 seconds. Only active drivers remain in the latest bucket.	Constant allocation and freeing of memory. Every window boundary is an alloc/free cycle, and a driver present across two windows is stored twice.
Sorted set per cell what Lyft uses	One Redis sorted set per geospatial cell. Member is the driver; score is the last timestamp they reported . <code>ZREMRANGEBYSCORE</code> expires everyone who has not reported in the last 30 seconds.	None of consequence. "They overwrite memory instead of reallocating it."

Additionally, driver location is stored in a hash structure, queried to ensure a driver does not show up in two cells at once while driving between them.¹

ONE STRUCTURE, TWO CONCERNS

`ZADD` on an existing member overwrites the score *in place* in the skiplist. So steady-state memory is one entry per driver, with no boundary effect where a just-rotated bucket looks empty.

More importantly: **"who is nearby" and "who is still alive" become the same read**, and expiry becomes an $O(\log N)$ range delete rather than a separate reaper process. Chapter 7 lists sorted sets under "leaderboards"; this is what they are actually good for.

THE TRADE — TIMESTAMP-SCORED SORTED SETS FOR LIVENESS

CHOSEN	One Redis sorted set per cell: driver as member, last-report timestamp as score, stale entries swept by range delete.
OVER	Rotating in-memory time buckets, or a separate heartbeat table alongside the location index.
COSTS	Every driver's location lives in RAM, so fleet capacity is bounded by cluster memory rather than disk. And liveness is only ever as fresh as the expiry window: with a 30-second sweep, a driver who stopped reporting 20 seconds ago is still a dispatch candidate. Shrink the window and you expire drivers on a brief signal drop; widen it and you dispatch to cars that have gone.
REVERSES WHEN	<i>staleness is genuinely unacceptable — then you need explicit acknowledgment from the driver at dispatch time, not merely a recent timestamp. The sorted set tells you who was there recently. It never tells you who is there now.</i>

3 • Matching: why waiting is faster

Lyft models dispatch as a **weighted bipartite graph** — riders on one side, drivers on the other, edge weight being the benefit of that pairing — formulated as an integer linear program with a binary variable per edge and degree-1 constraints per vertex.³

The structural fact they lean on: for bipartite graphs the **LP relaxation is integral**, so you can solve the linear program and get a valid assignment out. The Hungarian algorithm is polynomial, but Lyft notes commercial solvers are faster in practice and can absorb some genuinely binary constraints.

Graphs are generated and solved "at frequent intervals, such as every few seconds, on a rolling basis" — and the trade-off is stated plainly: longer batches "give a fuller picture of supply and demand, often producing better matches, lower wait times, and higher vehicle utilization, but they can delay ride assignments and risk cancellations."³

THE MYOPIA EXAMPLE, WHICH IS THE WHOLE ARGUMENT FOR BATCHING

At $t=0$ two drivers are available for one request. Greedy dispatch sends the nearer one. At $t=1$ a second request arrives, and only the far driver remains — producing a long wait that a joint assignment would have avoided.

Lyft calls greedy optimization **myopic** and lists three mitigations: augment the graph with *projected future demand* from high-density areas, put long-horizon objectives into the edge weights, and rebalance supply dynamically via incentives.

The transferable form: **buy global optimality with a few seconds of latency, then buy back the myopia with a forecast**. Batching alone still optimizes each batch in isolation; injecting predicted future demand is what connects them.

Shared rides are harder, and Lyft is explicit that they break the model: shared rides mean "the graph does not satisfy this condition" — it is no longer bipartite — and the published post excludes them. Earlier Lyft writing describes **route swapping**, moving a rider from one route to another before pickup if a better route is found, and accumulating matched riders for around 30 seconds before checking for improvements.¹ The specifics of the shared-ride solver have not been published in depth.

4 • The hard part: why Envoy exists

This is the section to read twice, because the problem statement is the best short argument about microservices in print.⁴

Before Envoy, Lyft used AWS load balancers for discovery and balancing, plus "a mishmash of different libraries across both PHP and Python", with HAProxy in a few places. At **roughly 30 services**, they already had:

IN LYFT'S OWN WORDS

"Continuous issues with sporadic networking and service call failures to the extent that **most developers were afraid to have high volume service calls in critical paths.**"

And — worse, because it is unfixable — no way to localize a failure: **"In the service code? In EC2 networking? In the ELB? Who knew?"**

Observability was whatever each application plus HAProxy plus "extremely primitive CloudWatch ELB statistics" happened to give them.

Note what the failure actually was. Not latency, not throughput, not cost. It was **engineers declining to use the architecture** because they could not trust or diagnose it. Thirty services was enough to produce that.

The sidecar argument

Lyft's case for an out-of-process proxy over per-language client libraries has two parts:⁴

1. **It works for any language.** One proxy per host means a single mesh spans Java, C++, Go, PHP and Python. Lyft was running all five.
2. **Library upgrades are painful.** "Deploying library upgrades can be incredibly painful", whereas Envoy "can be deployed and upgraded quickly across an entire infrastructure transparently."

What you get, implemented once and available to every service regardless of language: automatic retries, **circuit breaking**, global rate limiting via an external service, request shadowing, **outlier detection** (passive health checking), active health checking unioned with service discovery, statistics for every subsystem, and distributed tracing.

Compare that list against Chapter 6's resilience checklist — Airbnb's bulkheads, back pressure, deadline propagation, outlier detection — and notice that Airbnb *also* uses Envoy for the last of those. The sidecar is how those patterns stopped being bespoke.

THE GENERAL PRINCIPLE

Cross-cutting infrastructure concerns belong in a sidecar, not in N language clients. Anything every service needs, that is not the service's job, and that you will want to change without redeploying everything: retries, timeouts, circuit breakers, mTLS, tracing, rate limits, load balancing.

Airbnb reached the same conclusion about correctness libraries in Chapter 6 — and reached the *opposite* answer about deployment shape, choosing a library over a service because latency mattered and because an idempotency service would be reached over the network it was defending against. The distinction is worth holding: a sidecar is local, so it does not inherit the network's failures the way a remote service does.

THE TRADE — A SIDECAR INSTEAD OF A CLIENT LIBRARY

CHOSEN	One out-of-process proxy per host, carrying retries, circuit breaking, rate limiting, health checking, tracing and stats for every service regardless of language.
OVER	A client library per language — which is what Lyft had, across PHP and Python, plus HAProxy in places.
COSTS	An extra process on every host with its own CPU and memory footprint, and an extra hop on every call. A new per-host failure domain, and a new thing to operate, upgrade and debug. And it took roughly eighteen months of engineering before it was released.
REVERSES WHEN	<i>your fleet is one language and a handful of services. The sidecar's entire value is amortizing one implementation across many languages and many services; below that threshold a shared library is cheaper, simpler and faster. Lyft's own account puts the crossover at around thirty services.</i>

Envoy took about 18 months to build before release, later moved to the CNCF, and was extended to mobile. Lyft also uses it internally as the **Redis proxy layer** — having migrated that from Twemproxy — and for staging traffic overrides.^{2,4}

5 • Map matching, and a model replaced

Two use cases with different constraints: end-of-ride matching computes trip distance for the **fare** and may work backwards over the full history; real-time matching must be low-latency and causal, and feeds ETA, dispatch, and the car icon in the rider app.⁵

GPS degradation is named specifically: "urban canyons (where streets are surrounded by tall buildings)", stacked roads, tunnels — and Lyft notes "many Lyft rides happen there".

Then the part most teaching material has not caught up with. Lyft treats **HMM plus Viterbi** — the classic Newson & Krumm approach, and the same family as Chapter 4's approach — as the baseline it moved *away from*, citing three defects: inflexibility to new inputs, **$O(N^2)$** in candidates per state, and poor behavior at higher observation frequencies.⁵

The replacement is a **Marginalized Particle Filter**: each particle is one candidate road trajectory with a probability, a mean vector of position-along-trajectory and velocity, and a 2×2 covariance, with an Unscented Kalman filter running conditioned on each trajectory. Per observation: snap to nearest road positions, extend trajectories, run the filter, reweight, prune unlikely particles.

THE OUTPUT IS A DISTRIBUTION, NOT A POINT

That is the design payoff. Downstream teams can build an ETA *distribution* from it rather than being handed a single guessed position. Measured effect: lower ETA error and **fewer passenger cancellations**.

And the constraint Lyft names, which is worth remembering for any sensor-fusion problem: "we don't actually know the true locations of the drivers." There is no ground truth, so you cannot validate against it — you can only validate against downstream outcomes.

6 • Trade-offs

The design bought	And paid
Uniform retries, circuit breaking, tracing and stats across five languages	A proxy on every host — a new hop, a new deployment surface, and 18 months to build
Balanced shards for driver location	A cell scheme whose granularity must be tuned, and shard keys that no longer correspond to anything a human recognizes
Liveness and proximity from one read	All driver location lives in memory, so capacity is bounded by RAM across the cluster
Globally better matches	A few seconds of dispatch latency, and cancellation risk while riders wait
A position distribution rather than a point	A particle filter in the real-time path, with pruning heuristics and no ground truth to validate against
Low client memory via dynamic graph building	Four cache layers, and a client that can be wrong about the map until it refreshes

WHAT TRANSFERS

1. **Shard on the dimension load varies along, not the human-legible label.** City-keyed sharding cannot balance, because cities are not the same size.
2. **Let one data structure carry both the query and the freshness.** Score-as-timestamp sorted sets make proximity and liveness the same read, and expiry a range delete.
3. **Buy global optimality with a few seconds of latency — then buy back the myopia with a forecast.** Batching alone still optimizes each batch in isolation.
4. **Put cross-cutting reliability in a sidecar, not in N language clients.** Implemented once, upgradable without touching any application, uniform across a polyglot fleet.
5. **The failure mode that matters may be behavioral.** Lyft's crisis was engineers avoiding service calls because they could not diagnose them — a problem no latency graph would have shown.
6. **Emit a distribution when you have one.** Handing downstream consumers a point estimate throws away information they could have used.
7. **Thirty services is enough to need a mesh.** The number is much lower than most teams expect.

CLAIMS TO BE CAREFUL WITH

"Lyft does 15 million QPS on Redis." From a 2017 talk title; their own 2018 follow-up already said ~25M. Any present-tense use is nearly a decade stale.

"Lyft does HMM/Viterbi map matching." Outdated since at least 2020 — that is the baseline they replaced with a marginalized particle filter.

"Lyft fronts Redis with Twemproxy." Outdated; that layer moved to Envoy between the 2017 and 2018 talks.

"Uber uses H3 to find nearby drivers." Uber's H3 post describes it as the grid for marketplace analysis, surge pricing and dispatch optimization — not as the index behind real-time proximity lookup. The stronger claim is an extrapolation. For what Uber *did* publish about why hexagons beat squares, see Chapter 4 and the pattern catalog.

PRACTICE

1. Implement driver-location storage with sorted sets. Write the exact commands for position update, nearby query, and stale expiry — then work out what happens to a driver crossing a cell boundary.
2. Choose the dispatch batch window. Model the trade-off between match quality and cancellation risk, and say what you would measure to tune it.
3. You have 40 services and no mesh. Make the case for Envoy to a skeptical engineering leader, in terms of what will go wrong without it.
4. Shared rides break bipartiteness. Formulate the shared-ride matching problem, and propose a tractable approximation.
5. Your map-matching output is a distribution. Design the ETA that consumes it. What do you show the rider — the mean, a percentile, or a range?

SOURCES

1. Neo Kim, "How Lyft Support Rides to 21 Million Users", The System Design Newsletter #43, Apr 2024 (partly paywalled)
2. Daniel Hochman, "Geospatial at Scale" (RedisConf 2017) and "Redis at Lyft: 2,000 Instances and Beyond" (RedisConf 2018)
3. Oussama Hanguir, "Solving Dispatch in a Ridesharing Problem Space", eng.lyft.com, Jul 2025
4. Matt Klein, "Announcing Envoy: C++ L7 proxy and communication bus", eng.lyft.com, Sep 2016; envoyproxy.io
5. Douriez, Murphy & Staley, "A New Real-Time Map-Matching Algorithm at Lyft", eng.lyft.com, Aug 2020
6. "How Lyft Discovered OpenStreetMap Is the Freshest Map for Rideshare", eng.lyft.com; Lyft FY2025 results

Bluesky

What does a social network look like if you refuse to own the data — and what does that cost?

AT A GLANCE

Protocol	Authenticated Transfer Protocol (ATProto) — Bluesky is one app on it
Architecture	Federated: client–server, but different people run the servers
Per-user storage	One repository per user, one SQLite database each, records in CBOR
Density (see flag below)	6 million repositories on one data server at \$150/month
Index server	Go, backed by ScyllaDB, cached with Redis
Custom feeds	50,000+
Identity	Immutable DID for storage; mutable DNS-based handle for display

Chapter 1 is a centralized system that precomputes a view. This is the same product built by a team that refused to hold the data — and the interesting part is not the ideology. It is that every aggregate a centralized system reads from a column becomes, here, a crawl-and-count problem.

1 • The problem it was built to solve

Twitter's content moderation was easy when there were a few million users and every interaction went through central servers. Growth forced automation and human reviewers, and that produced three problems that scale could not fix:¹

- The risk of **error** increases when a single authority decides moderation policy.
- There is a risk of **bias** when a single authority makes the decisions.
- Managing moderation at scale needs enormous effort and becomes a **bottleneck**.

So Bluesky was set up as a research initiative to build a decentralized social network. Three decentralized architectures were available:

Architecture	Shape
Federated	Client–server model, but different people run parts of the system and the parts communicate with each other. Chosen — for the familiar client–server model, reliability and convenience. Servers talk over HTTP.

Architecture	Shape
Peer-to-peer	No distinction between client and server; every device acts as both.
Blockchain	A distributed ledger providing consensus and trustless interaction between servers.

The source's analogy for federation is the right one: **email**. Someone on Gmail can communicate with someone on Protonmail. And for the relationship between protocol and app: ATProto is the theme park, Bluesky is one attraction — a single ticket admits you to all of them, and if you dislike one you try another. Concretely, a user's data is shared across apps built on ATProto, so switching from Bluesky to a photo-sharing or blogging app on the same protocol **does not lose their followers**.

2 • Architecture

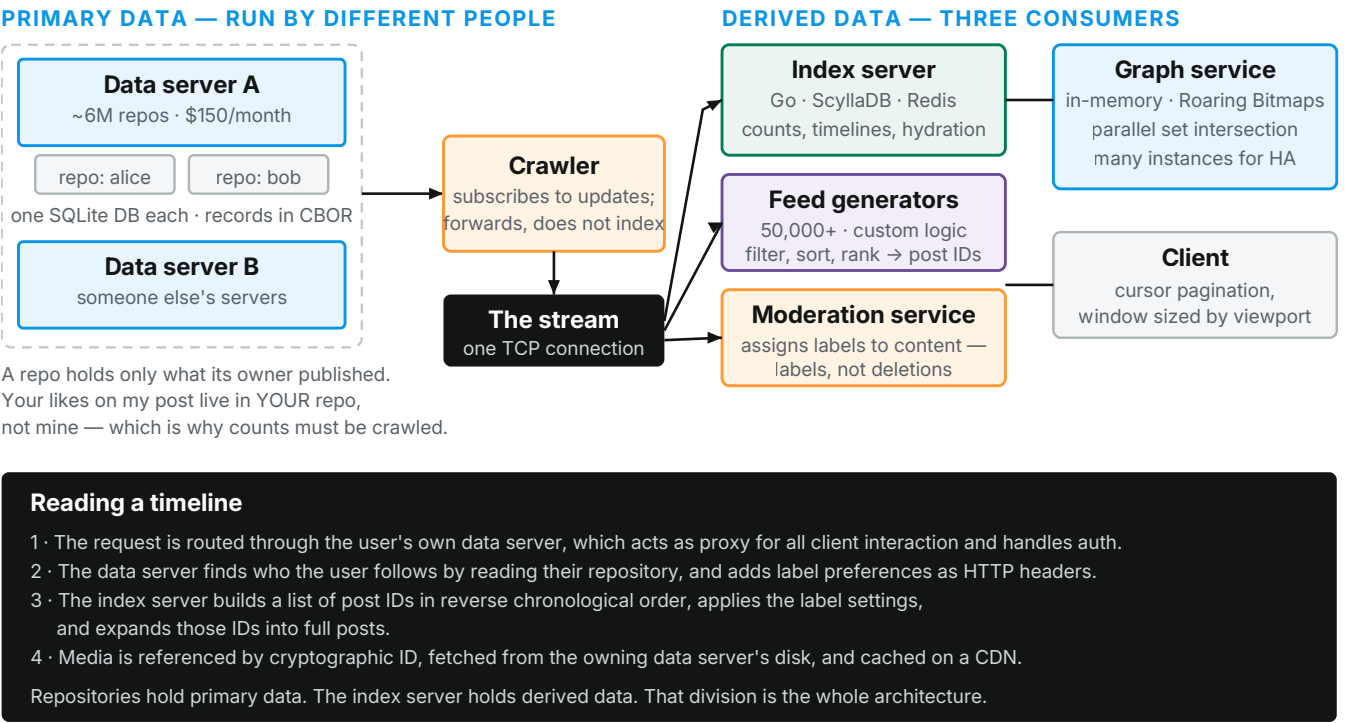


Figure 15.1 — One stream, three independent consumers. Because the crawler forwards rather than indexes, anyone can build a fourth consumer — which is how there are 50,000 custom feeds and a moderation layer that is not run by the people who run the servers.

Repositories: the unit of ownership

A **repository** is the collection of data published by a single user. Each gets its own **SQLite database**, chosen for simplicity, and records are encoded in **CBOR** — a compact binary format — before storage, to keep costs down.¹

Repositories live on **data servers**, and there are many for scale. A data server exposes HTTP and acts as a *proxy for all client interactions*, managing authentication and authorization, with tooling to apply updates across the federation.

The density figure usually quoted is **6 million user repositories on a single data server at \$150 a month** — and it is only conceivable because a repository holds so little: just what its owner published, in a compact binary encoding, in a file-based database.

FLAG: TREAT THAT FIGURE AS A CAPACITY CLAIM, NOT A DEPLOYMENT FACT

It appears in the secondary write-up this chapter is built from, and I could not trace it to a Bluesky primary source. Note also the timing problem: Bluesky's own engineering posts from April 2024 put the whole network at roughly **5.5 million users** — so "6 million repositories on one server" cannot have been a description of production at that point.

What is well sourced is the same order of magnitude from the other direction: Bluesky's graph service work reports fitting the entire network — **5.5M users and 164M+ follows** — into a roughly **1.6 GB** SQLite database on disk.² That is the number to quote, and it makes the same point more defensibly.

THE GIT ANALOGY IS LOAD-BEARING, NOT DECORATIVE

A user repository is like a Git repo and a data server is like GitHub. Moving a Git repo from GitHub to GitLab is easy — and likewise a user repository is **movable from one data server to another**.¹

That portability is the actual product claim of federation. It is not "the data is distributed"; it is "you can leave and take your identity and followers with you". Everything else in the architecture exists to make that possible.

The decision everything else follows from

A user's repository **does not store information about actions performed by their followers** — comments or likes on their post. That data lives only in the repository of the follower who took the action.¹

This is correct for ownership — your like is your data — and it is the source of every hard problem in the rest of the chapter. In a centralized system a post's like count is a column. Here, it does not exist anywhere until someone computes it.

The crawler and the stream

Pushing updates into each follower's repository would be expensive — that is fanout-on-write, and in a federated system you do not even control the destinations. So information is collected *from* every data server instead:¹

1. The crawler subscribes for updates on each data server — new posts, likes, comments.
2. Data servers notify the crawler in real time over websockets.
3. The crawler combines it into a **stream** — a log over websockets, merging every user's actions into a single TCP connection.

The crawler does not index data; it forwards it. That sentence is worth pausing on, because it is what makes the architecture extensible: the crawler has no opinion about what the data means, so anyone can consume the stream and build their own derived view.

The index server

The stream alone is not enough to render a post, because a post is displayed only after counting its likes, comments and reposts. The **index server** aggregates the stream — transforming raw data into a consumable form. Think of it as the data presentation layer.¹

It is built in **Go** for concurrency, with **ScyllaDB** for horizontal scalability, and because it is the most read-heavy service, its results are **cached in Redis**. Authentication between Bluesky services uses JWT.

And the mechanics of a count: a reference to a post's ID is added to the follower's repository when they like or repost. So the total number of likes and reposts is calculated by **crawling every user repository and adding the numbers up**.

THE COST OF NOT OWNING THE DATA

Follower count works the same way — found by indexing every repository, which the source compares to how **Google finds inbound links to a web page: all documents on the web are crawled**.¹

That comparison is exactly right, and it is the price of federation. In Chapter 1, a follower count is a column read. Here it is a web-scale crawl-and-aggregate problem, and it is eventually consistent by construction. **Decentralization converts every aggregate into a search-engine problem.**

3 • Identity: immutable for storage, mutable for humans

Following someone updates *only your own repository* — adding a reference to that user's **decentralized identifier (DID)**.¹

The identity split is elegant and worth copying:

- **Posts are stored by DID**, which is immutable.
- **The handle is displayed** alongside posts, and is mutable — reassignable to a custom domain name.
- Therefore **changing your handle does not affect your previous posts**. Every user gets a handle on the `bsky.social` subdomain at account creation.

Custom-domain verification uses DNS, which is a neat reuse of existing infrastructure: the user enters their domain in settings; Bluesky generates a unique text value (a public key); the user places it in the domain's DNS TXT record; the index server periodically checks the record and validates the handle.

SEPARATE THE IDENTIFIER FROM THE DISPLAY NAME — ALWAYS

This is one of the most broadly applicable patterns in the book, and it has nothing to do with decentralization. **Store by an immutable internal identifier; display a mutable human-facing one.**

Every system that stored a username as a foreign key eventually discovers that people change their names, and that rewriting history to match is either impossible or a data-integrity incident. The cost of the indirection is one join. The cost of not having it is permanent.

4 • Timelines, feeds, and 50,000 algorithms

A **timeline** is a user's own posts in reverse chronological order — but even that requires the index server, because the repository has no like or comment counts. The timelines of popular users are cached for performance.¹

A **feed** is posts from people you follow — and here the architecture pays off unexpectedly. Bluesky supports feeds with custom logic, and there are **50,000+ custom feeds**. A custom feed is essentially a filter for a list of keywords or users:

1. The crawler generates the stream from data servers.
2. A **feed generator** — a separate service, which anyone can run — consumes the stream.
3. It filters, sorts and ranks content by its own logic.
4. It produces a list of post IDs.
5. The index server populates the content on user request.

COMPARE THIS DIRECTLY WITH CHAPTER 1

Twitter's ranking is one pipeline, owned by Twitter, running 5 billion times a day with a 48-million-parameter model. Bluesky's is **50,000 pipelines owned by whoever wrote them**, each producing nothing more than a list of post IDs that the index server then hydrates.

The architectural enabler is the same as the extensibility point noted earlier: because the crawler forwards rather than indexes, and because a feed's output contract is just "a list of IDs", the ranking layer is pluggable. **Pick your output contract small enough and third parties can implement it.**

Pagination is **cursor-based**: a sequential unique column such as the post timestamp is the cursor; requests include it to indicate the result offset; responses return a new cursor. The **client decides the window size based on its viewport** — a nice touch — and the scheme scales because it prevents a full table scan in the index server. Chapter 1 makes the same argument for the same reasons.

5 • Moderation as labeling, not deletion

This is where the original motivation cashes out. Content is analyzed by a **moderation service** that assigns a **label** — metadata categorizing the content. Labels can also be applied manually. A user then

chooses to hide, or show a warning for, posts carrying a particular label, and that preference is stored on *their* data server.¹

The flow:

1. The moderation service consumes the stream from the crawler.
2. It analyzes content and assigns labels.
3. The index server stores content along with its labels.
4. User requests route via their data server to the index server.
5. **The data server includes label IDs in the HTTP request headers** before forwarding.
6. The index server applies the label setting in its response.

Separately, a data server operator does basic moderation, and the data server filters out muted users before responding. The summary: a user can post about anything, but content is moderated before being shown to the public.

WHY "LABEL, DON'T DELETE" IS THE STRUCTURAL ANSWER TO THE OPENING PROBLEM

The three problems that motivated Bluesky were single-authority error, single-authority bias, and moderation as a bottleneck. Labeling addresses all three at once, because **labeling is a separable, pluggable, non-destructive layer**: multiple labellers can disagree, users choose which to trust, and no single party has to be right.

The cost is equally real and should be named: content that a centralized platform would remove outright remains on the network, visible to anyone who has not subscribed to a labeller that flags it. This is a genuine trade, not a free win.

6 • The hard part: social proof, and four attempts at it

Bluesky shows you how many of the people *you* follow also follow a given user. It is a small feature with a brutal data shape, and the escalation is the best thing in the chapter.¹

Attempt 1 — query the database

Find the people I follow. Then run a separate parallel query for each to find who *they* follow. Check whether any of them follow the target.

This does not scale, because the number of parallel queries grows with the number of people I follow. Someone following 5,000 accounts triggers 5,000 queries for one profile view.

Attempt 2 — reframe as set intersection

Set 1: the people I follow. Set 2: the people who follow the target. The intersection is the answer. This is the correct reframing, and everything afterwards is about making intersection fast.

Attempt 3 — an in-memory graph service

An in-memory service avoids expensive database queries and performs intersections quickly. But **Redis sets do not use multiple CPU cores at once** — the single-threaded property that Chapter 7 celebrates is here a ceiling. So they built their own, and the MVP was:

- Each user has a 32-character DID.
- DIDs are converted to `uint64` to reduce memory.
- Each user maintains two sets: people they follow, and people who follow them.

Still too much memory, and slow to start.

Attempt 4 — Roaring Bitmaps

The source builds up to this properly, and the progression is worth following because it teaches the underlying trade-off:

Representation	How it works	Why it fails
Bitmap	Binary states in a bit array. Set the index of each person you follow to 1; walk the array and record non-zero bits. Lookup is constant time and bitwise operations are fast.	Inefficient for sparse data. If Bluesky had 100 users, you need a 100-bit array even if you follow exactly one person. At real scale this is fatal.
Run-length encoding	Turn consecutive runs of 0s and 1s into counts. Following the last 10 of 100 users stores as "90 zeros, 10 ones".	Fails for randomly distributed data, and a lookup must walk the entire bitset to find one index.
Roaring Bitmaps	Compressed bitmaps, "but 100 times faster". Data is split into containers , and each container uses a different storage mechanism : dense data as a fixed-size bit array; large contiguous integer ranges as run-length encoding; sparse data as a sorted array of integers.	Nothing of consequence here. Intersection is done in parallel, with containers converted to a common format as needed.

THE IDEA WORTH STEALING: CHOOSE THE ENCODING PER REGION, NOT PER STRUCTURE

Bitmaps are optimal for dense data and catastrophic for sparse. Sorted arrays are the reverse. Run-length encoding is optimal for runs and useless for randomness. The usual response is to pick whichever is least bad on average.

Roaring Bitmaps refuse the choice: **partition the key space into containers and let each container pick its own representation based on its own density.** The cost is that operations must handle mixed representations; the benefit is near-optimal encoding everywhere.

This generalizes well past bitmaps. Chapter 1's Manhattan does the same thing at a different layer, with three storage engines — LSM for write-heavy, B-tree for read-heavy, read-only for batch output — chosen per access pattern rather than globally. **When one representation cannot fit all your data, stop looking for a better single representation.**

Graph data is also *stored and transferred over the network* in Roaring Bitmap serialization format — so there is no encode/decode step at the boundary. And many instances of the graph service run for high availability and rolling updates.

7 • Short video, with a nice tracking trick

Bluesky supports videos up to 90 seconds, streamed over **HLS**:¹

- A video is encoded into quality levels — 480p, 720p, 1080p — and each is split into segments of about five seconds.
- A **master playlist** lists all available quality levels; a **media playlist** lists all segments for one quality level.
- The client downloads the master playlist, selects a media playlist based on network conditions, and downloads segments in sequence.
- Videos and playlists are cached in a CDN, with stream requests routed there via HTTP 302 redirects.
- Subtitles are stored as text with their own media playlist, referenced from the master.

ANALYTICS FOR FREE, FROM THE PROTOCOL

Video views are tracked by **counting requests for the master playlist file.** The master playlist response includes a **Session ID**, which is included in subsequent media-playlist requests to track the session. And seconds watched is derived by **checking the last fetched segment.**¹

No analytics beacon, no separate telemetry path, no client SDK to keep in sync. The delivery protocol already emits exactly the signal needed. **Before building a telemetry pipeline, check whether your existing request pattern already contains the answer.**

8 • Trade-offs

The design bought	And paid
Users can move hosts and keep their identity and social graph	Every aggregate — likes, reposts, follower counts — becomes a crawl-and-count problem rather than a column read
Moderation that no single authority controls	Content a centralized platform would delete stays on the network, visible to anyone not subscribed to a labeller that flags it
50,000 ranking algorithms instead of one	No ability to reason globally about what users see, and an index server that must serve any feed's ID list
6 million repositories for \$150/month	Repositories hold almost nothing, so the derived layer carries all the cost and complexity
Social proof at interactive latency	A bespoke in-memory graph service with a custom compressed set representation — genuinely specialist engineering for one feature
A protocol other apps can build on	Schemas and endpoints must be standardized and then kept stable, which constrains how fast Bluesky itself can change

WHAT TRANSFERS

1. **Separate primary data from derived data explicitly.** Repositories hold what users wrote; index servers hold everything computed from it. Knowing which of your tables is which is clarifying even in a centralized system.
2. **Forward, don't index, at the integration point.** Because the crawler has no opinion about meaning, three independent consumers — and 50,000 feed generators — can be built on one stream.
3. **Pick an output contract small enough for third parties to implement.** "A list of post IDs" is why the ranking layer is pluggable.
4. **Store by immutable identifier; display a mutable name.** The indirection costs one join. Not having it costs you history.
5. **Reuse existing infrastructure for verification.** A DNS TXT record proves domain control with no new system to build.
6. **When one representation cannot fit all your data, partition and let each region choose.** Roaring Bitmaps for sets; multiple storage engines for a database.
7. **Check whether your request pattern already contains your analytics.** Master-playlist requests are the view count.
8. **Decentralization is a set of specific costs, not a vibe.** Name them: aggregates become crawls, deletion becomes labeling, and global reasoning becomes impossible.

PRACTICE

1. A post goes viral and gains a million likes in an hour, spread across a thousand data servers. Trace how the count becomes correct, and say how stale it is at each stage.
 2. Design the social-proof service. What is in memory, how much, how is it sharded, and what happens on a cold start?
 3. You must implement "delete my account and all my data" in a federated system where other people's repositories contain references to your posts. What can you actually guarantee?
 4. Write the feed-generator contract. What does a third party receive, what must they return, and how do you stop a badly-written generator from hurting the index server?
 5. Compare Chapters 1 and 15 on five axes: who ranks, who moderates, where aggregates live, what a user can take with them, and what the operator can change unilaterally.
-

SOURCES

1. Neo Kim, "How Bluesky Works", The System Design Newsletter #65, Jan 2025 — written from Bluesky's engineering blog and ATProto documentation (final section paywalled)
2. Jaz Volpert, "In-Memory Graphs" (Apr 2024) and "Roaring Bitmaps" (Apr 2024), jazco.dev — the primary account of Bluesky's GraphD service and its move to Roaring Bitmaps
3. atproto.com — protocol specification, repository and lexicon documentation
4. Chambi, Lemire, Kaser & Godin, "Better bitmap performance with Roaring bitmaps", *Software: Practice and Experience*, 2016

PART THREE

Reference

The cross-reference index, a catalog of what the received wisdom gets wrong, the numbers worth committing to memory, and a way to work through all of it without giving up in week three.

A Pattern Index — every pattern, and where to see it working

B Stale Facts and Confident Errors

C Numbers Worth Knowing

D A Fifteen-Week Study Plan

Pattern Index

The inverse of Chapter 4: every pattern, and every place in Part II where you can watch it doing real work.

Use this when you have a problem rather than a curiosity. Find the shape of your problem, then read the two or three chapters where somebody has already solved it.

Read/write asymmetry — precompute or compute on read

[Twitter 1](#) · [Bluesky 15](#) · [WhatsApp 2 \(counter-case\)](#) · [Airbnb 6](#)

Measure the ratio first. Twitter's 50:1 licensed O(followers) writes; WhatsApp's ~1:1 licensed nothing. When the requirement changed from ordering to ranking, Twitter deleted the precomputation entirely.

Store references, hydrate on read

[Twitter 1](#) · [Bluesky 15](#) · [YouTube 3](#) · [Spotify 8](#)

Twenty bytes per timeline entry instead of a kilobyte, then parallel multi-get against independently cached services. A manifest and a feed-generator's ID list are the same idea at different layers.

Cache invalidation, leases, and the thundering herd

[Instagram 5](#) · [Redis 7](#) · [Cloudflare 12](#) · [Figma 11](#)

Invalidate from the source of truth, not the application. Use a lease so one request does the work. Offer stale data where staleness is harmless — and know which of your reads those are. Note that a deploy is also a herd trigger.

Sharding, hot shards, and the celebrity problem

[Twitter 1](#) · [Tinder 9](#) · [Figma 11](#) · [Lyft 14](#) · [Cloudflare 12](#)

Shard on the dimension load varies along, not the human-legible label. Then: decouple logical partition from physical placement, cache the read hotspots, and throttle the write ones — you cannot cache a write.

Geospatial indexing

[Tinder 9](#) · [Lyft 14](#) · [Uber 4](#)

Hierarchical cells so granularity follows density; a space-filling curve so 2D proximity becomes a 1D sort key; and the same key serving as both index and shard key. Uber's published case for hexagons is one uniform center-to-neighbor distance.

Idempotency and exactly-once

[Airbnb 6](#) · [YouTube 3](#) · [PayPal 13](#) · [Spotify 8](#)

Pre-RPC / RPC / Post-RPC, with no network calls in the transaction phases and no database work in the RPC phase. A row lock on the idempotency key granting a lease longer than the RPC timeout. Errors classified, defaulting to non-retryable.

Concurrency control — locks, versions, constraints

[Airbnb 6](#) · [Redis 7](#) · [Figma 11](#)

Make the invariant a database constraint, not application logic. Use a distributed lock for admission control, never as the guarantee. And never hold a lock across a network call.

Serialize by routing, not by locking

[PayPal 13](#) · [Tinder 9](#) · [WhatsApp 2](#)

The most under-taught idea in the book. An actor mailbox, a Kafka partition keyed by entity, or hashing a record onto one process on one node — three technologies, one result: concurrency control becomes unnecessary rather than difficult.

Backpressure, head-of-line blocking, circuit breaking

[WhatsApp 2](#) · [Cloudflare 12](#) · [Lyft 14](#) · [Airbnb 6](#) · [Instagram 5](#)

More queues, keyed by the thing that can fail. Then decide deliberately whether to propagate backpressure or drop it. Put a deadline on anything holding a pool slot, and question FIFO — order by cost or by age instead.

Chunking for parallelism

[YouTube 3](#) · [Uber 4](#) · [Spotify 8](#)

Find the boundary at which work becomes independently processable — a closed GOP, a graph partition, an idempotent event — and parallelism is free. Then amortize the expensive shared prefix across all outputs.

Adaptive delivery — move the decision to the edge

[YouTube 3](#) · [Spotify 8](#) · [Bluesky 15](#) · [Zoom 10](#)

Encode a ladder, segment it, publish a manifest, let the client choose — the manifest is what lets a dumb HTTP cache serve an adaptive stream. In steady state the client needs buffer occupancy, not a capacity estimate.

Move the work to the client

[Zoom 10](#) · [WhatsApp 2](#) · [Lyft 14](#) · [Twitter 1](#)

Put the decision where the information is. Quality adaptation belongs where packets arrive; the road graph belongs where the driver is. And sometimes refuse the expensive operation entirely rather than optimizing it.

Message passing and isolated state — the actor model

[PayPal 13](#) · [WhatsApp 2](#) · [Cloudflare 12](#) (inverted)

Match the concurrency model to the workload's unit of concurrency. The unit of isolation must be cheap enough to have millions — which is exactly why an OS process per connection is Cloudflare's problem and an actor is PayPal's solution.

Decentralization and federation

[Bluesky 15](#) · [Twitter 1](#) (counter-case)

Separate primary from derived data. Forward rather than index at the integration point, so third parties can build consumers. Then price the costs honestly: aggregates become crawls, deletion becomes labeling, global reasoning becomes impossible.

Two-stage retrieval, then ranking

[Twitter 1](#) · [Airbnb 6](#) · [Spotify 8](#)

Cheap recall-oriented candidate generation to a fixed budget, then expensive precision-oriented ranking. The budget is the contract between stages, and stripping provenance before ranking is what makes heterogeneous sources comparable.

Data placement as a latency strategy

[Instagram 5](#) · [Lyft 14](#) · [YouTube 3](#) · [Cloudflare 12](#)

Replica count need not equal data-center count. Put data where the user is, keep one replica elsewhere for survival, pay a small lookup to find it — and make placement dynamic so data follows the user.

Change data capture as the integration point

[Tinder 9](#) · [Airbnb 6](#) · [Instagram 5](#) · [Bluesky 15](#)

The database's change log is more reliable than application code remembering to publish an event. There is always one write path that forgets.

Residual prediction — model your error, not the answer

[Uber 4](#) · [Spotify 8](#)

Keep a physically interpretable baseline that works on day one in a new market, and have the model learn only the correction. You degrade to the baseline rather than to nothing.

Choose the representation per region, not per structure

[Bluesky 15](#) · [Twitter 1](#) · [Spotify 8](#)

Roaring Bitmaps pick an encoding per container by density; Manhattan picks a storage engine per access pattern; Cassandra needs TTL'd and non-TTL'd rows in separate column families. When one representation cannot fit all your data, stop looking for a better single one.

Insert a protocol-compatible proxy

[YouTube 3](#) · [Figma 11](#) · [Cloudflare 12](#) · [Lyft 14](#)

Sharding logic belongs in a proxy, not in application code — and once the proxy exists it becomes the place to put pooling, query rewriting, load shedding, priority queueing and congestion control. A sidecar does the same for cross-cutting network concerns.

Separate identifier from display name

[Bluesky 15](#) · [Twitter 1](#)

Store by an immutable internal identifier; display a mutable human-facing one. The indirection costs a join; not having it costs you history.

Tier treatment by popularity, reversibly

[YouTube 3](#) · [Spotify 8](#) · [Bluesky 15](#)

Spend compute on the head to save bandwidth; spend nothing on the tail beyond correctness; re-evaluate when popularity moves. One policy governs both processing depth and edge caching.

Instrument the runtime, not just the application

[WhatsApp 2](#) · [Cloudflare 12](#) · [Instagram 5](#)

"Optimize the hot path" is wrong advice when the hot path is 16% of the budget. Watch for misleading health signals — 45% CPU alongside 95% scheduler utilization — and get metrics at a resolution that can see the event.

Make production observable and safely mutable

[WhatsApp 2](#) · [Twitter 1](#) · [Spotify 8](#) · [Airbnb 6](#)

When the test harness costs more than the system, stop simulating. Tee'd traffic, hot code loading, runtime flags, and the ability to turn an optimization off so you can measure it. Decoupling launch from deploy is the general capability.

Stale Facts and Confident Errors

Thirty claims that are taught everywhere and are wrong, out of date, or unsourced.

This appendix exists because system design has a citation problem. Numbers get repeated for a decade after the system changed, misquotes calcify, and one secondary blog post becomes forty.

Read it for two reasons. The obvious one is to avoid saying something wrong with confidence. The better one is that **knowing which parts of the received wisdom are soft is itself a senior skill** — and being able to say "that number is from 2014 and here is what changed" is a much stronger signal than reciting it.

Four verdicts are used:

Verdict	Means
WRONG	The claim is factually incorrect, often inverted from the truth.
STALE	It was true. The system changed and the teaching material did not.
UNSOURCED	Repeated everywhere; no primary source exists. Sometimes falsely attributed.
CONFLATED	Two real things merged into one wrong thing.

Twitter / X

"Twitter builds your timeline with fanout-on-write into Redis."

STALE — AND THIS IS THE BIG ONE

Twitter's 2023 recommendation release states: "We recently stopped using Fanout Service, a 12-year-old service that was previously used to provide In-Network Tweets from a cache of Tweets for each user." In-network posts now come from the search index. Nearly every course teaches the 2012 architecture in the present tense. Teach it as history, then teach the retirement as the lesson. (Chapter 1)

"Accounts above 10,000 followers switch to fanout-on-read."

UNSOURCED — AND OFTEN FALSELY ATTRIBUTED

No numeric threshold appears in the original talk or its canonical write-up, and several secondary sources attribute the number to them anyway. The hybrid is described qualitatively and as *future work*, not shipped state. If you use a threshold, present it as your own design parameter and justify the number.

"Nighthawk is Twitter's timeline cache."

CONFLATED

Nighthawk is Twitter's general sharded-Redis cache tier. The tweet-timeline cache specifically is **Haplo**, a customized Redis implementing a HybridList type, written by Timeline Service and Fanout Service and read only by Timeline Service. Only the 2017 infrastructure post distinguishes them.

"Manhattan stores the timelines."

CONFLATED

Manhattan backs tweets, direct messages and accounts. Timelines lived in Redis with durable backing in Gizzard/MySQL. Manhattan also launched in 2014 — *after* the timeline architecture usually described alongside it.

"Each timeline caches 800 tweets."

STALE

Real, and a genuinely useful number — but a 2012 figure from a system that no longer exists in that form. There is no published modern equivalent.

"Earlybird is a fork of Lucene."

CONFLATED

Popular glosses say "a modified version of Lucene"; the ICDE paper describes index structures deliberately *unlike* standard retrieval engines — an uncompressed active segment, postings stored chronologically but traversed backwards. "Lucene-derived, heavily rewritten" is the defensible phrasing.

"Twitter handles 500 million tweets a day. / 5,700 TPS. / 143,199 TPS. / 4,000 QPS."

CONFLATED ACROSS A DECADE

These are four different quantities from three different eras: fanout pipeline work (2012), daily average (2013), a one-second peak (2013), and posts per day (2023). Note the trap: "500 million tweets per day" appears in both the 2013 and 2023 posts and almost certainly counts different things. Never mix strata.

WhatsApp

"WhatsApp runs 2 million connections per server."

STALE — AND IT WAS NEVER THE STEADY STATE

Two million was a *tuning milestone*; the peak achieved was 2.8M; three million failed outright. By 2014 they had **deliberately reverted to about one million** per server for headroom. Teach it as "what they proved possible", never as "what they ran". (Chapter 2)

"WhatsApp handles 70 million messages per second."

WRONG — MISQUOTED

The figure is >70 million *Erlang* messages per second — internal VM message passing between processes. The user-message peak from the same talk is 342K inbound and 712K outbound per second. This misquote is extremely widespread.

"Fifty engineers served 900 million users."

UNSOURCED

The attributable figures are **32 engineers at 450M users** (Feb 2014) and about ten people on the Erlang team (Mar 2014). The 50/900M and 50/2B pairings circulate entirely through secondary posts, and they conflate total engineering with the server team.

"WhatsApp uses client-side fanout."

WRONG FOR GROUPS

True for one-to-one, where the client encrypts N times for N devices. For groups, the encryption whitepaper says the sender sends one ciphertext "to the server, which does server-side fan-out to all group participants". The correct framing: client fanout across devices for pairwise chats; sender keys plus server fanout for groups.

"WhatsApp stores no data at all."

OVERSTATED

Undelivered messages are held with a 30-day window and a write-back cache at a 98% hit rate; multimedia metadata occupied roughly 2 TB of in-memory Mnesia across ~18 billion records in 2014; prekey bundles and end-to-end-encrypted app-state blobs are server-stored. The accurate claim is narrower: *message content is not retained after delivery*.

"WhatsApp is XMPP / ejabberd."

STALE

It started from ejabberd and then "switched from XMPP to an internally developed protocol". "XMPP-derived, ejabberd-descended, since replaced" is right. Note also that [yowsup](#), the reverse-engineered client library often cited for protocol details, was taken down — so claims sourced from it are both unofficial and stale.

"Group size is capped at 1,024 because of sender-key rotation cost."

THE CAP IS REAL; THE REASON IS INFERENCE

1,024 is current, raised from 512 in late 2022, but documented only in consumer help pages and tech press. **There is no published statement that it is a technical limit.** The rotation-cost argument is sound engineering reasoning — $O(\text{members} \times \text{devices})$ on every departure — but presenting it as WhatsApp's stated rationale would be invention.

"WhatsApp's architecture is FreeBSD, Erlang R16, Mnesia and 550 servers."

STALE — EVERYTHING HERE IS 2014

All of it describes a company that has since grown roughly 4× in users and spent a decade inside Meta's infrastructure. I found no primary post-acquisition disclosure of the serving architecture. Label any "how WhatsApp works today" narrative built on these numbers as of-2014.

YouTube

"500 hours of video are uploaded every minute."

UNSOURCED, AND FROZEN SINCE ~2019

Universally repeated and attributed to YouTube, but it traces to marketing aggregators. Google's own peer-reviewed phrasing, from 2021, is the vaguer "multiple hundreds of hours of video every minute". Use that. (Chapter 3)

"One billion hours of YouTube are watched daily."

REAL, OFFICIAL — AND FROM FEBRUARY 2017

Nine years old, and still the most recent official figure. Cite it with the date attached.

"Google's Argos chip gives a 20–33× speedup."

WRONG TWICE

It is **performance per total cost of ownership**, not throughput, and the 20–33× band is **VP9-specific**. H.264 perf/TCO is 4.4–7.0×. The 33× figure also requires the dense 20-card configuration. Raw VP9 throughput at 20 VCUs is about 99× the CPU baseline — a different claim again.

"YouTube runs on Vitess."

TRUE HISTORICALLY; UNCONFIRMED TODAY

True per Vitess's own documentation, but no current Google statement describes YouTube's database topology. Vitess is now maintained largely outside Google, and its named production users are Slack, GitHub, Etsy and others. The widely-quoted "70,000 MySQL nodes across 20 data centers" is undated vendor marketing with no primary corroboration.

"YouTube was rewritten off MySQL."

BACKWARDS

Vitess exists precisely so that it did not have to be. That is the whole point of the case study.

"YouTube uses DASH."

INCOMPLETE RATHER THAN WRONG

YouTube has moved substantially toward server-side adaptive bitrate over a proprietary format, combining audio and video in a single request and requesting by player time. This is documented only through reverse-engineered clients, not by Google — so it is *unofficial*, and DASH/HLS remains the right thing to design with. But note the direction of travel.

"YouTube's CDN is a three-tier cache hierarchy."

SOURCED TO A 2012 EXTERNAL MEASUREMENT STUDY

Google confirms only that Global Cache "is a hierarchical CDN" with dynamic cache-fill source selection. It does not publish a tier count. Google *does* state that caches are never pre-filled, and that some high-volume content is not popular enough to cache at the edge and is served from core or proxied through the edge node.

Spotify

"Spotify starts playback in under 250 milliseconds."

NOT A PUBLISHED FIGURE

The only primary number is **265 ms median, 515 ms p75, 1,047 ms p90** — from 2010, on desktop, in the P2P era, and explicitly dominated by DRM and local processing rather than network. No newer Spotify-published startup latency exists. (Chapter 8)

"80% of Spotify's traffic was peer-to-peer."

CONTRADICTED BY THE ONLY MEASUREMENT

Measured: 8.8% servers, **35.8% peers**, 55.4% local cache. The 55.4% cache share is what this is being conflated with.

"Spotify killed P2P because ISPs were blocking it."

SPOTIFY NEVER PUBLISHED A RATIONALE

Every reason in circulation traces to two 2014 press articles. The ISP-blocking explanation in particular is speculation. One structural reason *is* solid, because it is in the 2010 paper: mobile clients never had P2P, so the tier covered a shrinking share of traffic as listening moved to phones.

"Spotify replaced Kafka with Google Cloud Pub/Sub."

MISLEADING SUMMARY

They evaluated *Kafka 0.8* for *one new subsystem*, hit specific bugs — several of which the post itself notes were fixed in 0.9 — and chose Pub/Sub. Not a verdict on modern Kafka, and not a company-wide replacement. The disqualifying bug is worth remembering though: silent data loss behind a successful acknowledgment.

"Spotify runs on Cassandra. / Spotify runs 1,600 microservices."

BOTH STALE OR UNSOURCED

The Cassandra post is January 2015, and the 2017 Scio post lists **Cassandra → Bigtable** as part of a managed-services consolidation. Storm, Crunch, Scalding, Luigi-on-Hadoop and Hive are likewise superseded. As for service counts: the figures of 1,600, 2,000 and 4,000 circulate with no first-party source. The only published count is **810, from 2015**.

"Discover Weekly combines NLP crawlers over music blogs with CNNs on audio spectrograms."

FROM A THIRD-PARTY EXPLAINER, NOT SPOTIFY

This three-model story originates in a widely-shared 2017 Medium post. Spotify's own published material supports implicit-feedback matrix factorization (logistic matrix factorization, Annoy for retrieval) and Scalding/Scio batch processing. The crawler and CNN components are unverified as production.

Airbnb

"Airbnb prevents double-booking with row locks on calendar_day rows."

AIRBNB HAS PUBLISHED NOTHING ON BOOKING CONCURRENCY

Every variant of this claim — row locks, Redis holds, optimistic versioning — traces to interview-preparation content, frequently written in Airbnb's voice. Chapter 6's five-mechanism analysis is industry practice with my reasoning, not Airbnb's documented design, and it says so.

"Airbnb uses HNSW for vector search. / Airbnb's search runs on Elasticsearch."

THE FIRST IS BACKWARDS; THE SECOND IS UNCONFIRMED

They evaluated HNSW and **rejected** it for IVF, specifically because availability and pricing churn inflated the HNSW memory footprint. And the 2025 retrieval post says **Lucene**, not Elasticsearch — which may well be true underneath, but is not what they published.

"Airbnb completed its migration to microservices."

OVERSTATED

Airbnb consistently says *service-oriented architecture*, not microservices — explicitly contrasting the two and citing component reuse. And the payments account describes migration still winding down, with two orchestration systems co-resident for years.

Uber and Lyft

"Uber replaced Dijkstra with contraction hierarchies."

BACKWARDS

Uber evaluated contraction hierarchies and **rejected** them for the production engine, because ~12-hour full preprocessing and ~10-minute incremental updates both block live traffic. They also rejected geographic sharding, for load-shape reasons. What shipped is a layered-cell partition scheme. (Chapter 4)

"Uber uses H3 to find nearby drivers."

EXTRAPOLATION

Uber's H3 post describes it as the grid for marketplace analysis, surge pricing and dispatch optimization — not as the index behind real-time proximity lookup. Also note that "all hexagon neighbors are equidistant" has documented exceptions: H3 contains 12 pentagons, because hexagons cannot tile an icosahedron.

"Lyft shards driver locations at geohash level 5, handling 125,000 GPS writes per second."

MECHANISM SOURCED; NUMBERS NOT

The cell-based sharding and sorted-set liveness pattern are well sourced from Lyft's conference talks. The specific figures appear only in secondary blogs and interview-prep sites. Treat them as plausible illustration.

"Lyft does 15 million QPS on Redis, fronted by Twemproxy, using HMM/Viterbi map matching."

ALL THREE ARE OUT OF DATE

15M QPS comes from a 2017 talk title; their own 2018 follow-up said ~25M. The Redis proxy layer moved from Twemproxy to Envoy between those two talks. And HMM plus Viterbi is the baseline Lyft *replaced* in 2020 with a marginalized particle filter over Unscented Kalman filters — citing $O(N^2)$ cost in candidates per state and poor behavior at higher observation frequencies. (Chapter 14)

"Lyft's dispatch batch window is 5 seconds, and it does route swapping for shared rides."

BOTH UNDERSPECIFIED

Lyft says only "every few seconds" / "a few seconds" — no specific window is published. Route swapping is attributed to a 2016 series, but the 2025 dispatch post explicitly declines to cover shared rides beyond noting that they are non-bipartite. Treat the shared-ride solver specifics as unverified.

Bluesky

"Bluesky runs 6 million user repositories on one data server for \$150 a month."

A CAPACITY CLAIM, NOT A DEPLOYMENT FACT

It appears in secondary write-ups and I could not trace it to a Bluesky primary source. There is also a timing problem: Bluesky's own April 2024 engineering posts put the entire network at roughly **5.5 million users**, so it cannot have described production at that point.

The well-sourced number that makes the same point: Bluesky's graph service fits the whole network — 5.5M users and 164M+ follows — into roughly **1.6 GB** of SQLite on disk. Quote that instead. (Chapter 15)

And one of my own

"Spotify's P2P overlay had a fixed maximum node degree of 60 and no tit-for-tat."

CLOSE, BUT IMPRECISE — THIS GUIDE SAID IT FIRST

The paper specifies a **soft limit of 50 and a hard limit of 60**: the client stops opening connections at 50 and prunes back below it; 60 is only the never-exceed ceiling. And while the paper never uses the phrase "tit-for-tat", its peer-utility heuristic *does* include bytes sent and received — so "no reciprocity" overstates it.

Included here because it is the kind of small drift that turns into a myth after three retellings, and because a corrections appendix that exempts itself is not worth much.

HOW TO USE THIS IN AN INTERVIEW

Do not lead with corrections — it reads as pedantry. Lead with the reasoning, then date the architecture:

"The classic answer here is fanout-on-write, and the read/write ratio that justified it was about 50 to 1.

Worth noting Twitter retired that service in 2023 once ranking replaced chronology — so let me design for the requirement you actually have."

That sentence demonstrates the pattern knowledge, the quantitative reasoning, and the awareness that architectures have dates. All three in one breath.

Numbers Worth Knowing

Latency, capacity, and every scale figure in this guide — with its date attached.

Latency

Operation	Order of magnitude
L1 cache reference	~1 ns
Branch mispredict	~3 ns
L2 cache reference	~4 ns
Mutex lock / unlock	~17 ns
Main memory reference	~100 ns
Compress 1 KB	~2 μ s
Read 1 MB sequentially from memory	~10 μ s
SSD random read	~100 μ s
Round trip within the same data center	~500 μ s
Read 1 MB sequentially from SSD	~1 ms
Disk seek (spinning)	~10 ms
Read 1 MB sequentially from disk	~20 ms
Round trip California → Netherlands	~150 ms

Two ratios are worth carrying: memory is roughly **1,000× faster** than SSD for random access, and a cross-continent round trip costs roughly **300×** a same-datacentre one. Nearly every caching and data-placement decision in Part II falls out of one of those two facts.

Capacity and cost approximations

Quantity	Approximation
Seconds in a day	86,400 — call it 10^5
1 million QPS sustained	≈ 86 billion requests/day
1,000 QPS sustained	≈ 86 million requests/day
Peak multiplier over average	3–5× typically; up to 25× for event-coupled products

Quantity	Approximation
UUID	16 bytes binary, 36 characters as text
Timestamp	8 bytes
A short text post with metadata	~1 KB
A timeline entry as ID + user ID + flags	~20 bytes
A 3.5-minute audio track at 128 kbps	~3 MB
One minute of 1080p video, pre-transcode	~50–100 MB
Full rendition ladder below 1080p	~0.85× the pixel cost of 1080p alone
Postgres connection	One OS process — expensive enough to need a pooler

Availability arithmetic

Target	Downtime per year	Per month
99%	3.65 days	7.3 hours
99.9% — "three nines"	8.77 hours	43.8 minutes
99.99%	52.6 minutes	4.4 minutes
99.999% — "five nines"	5.26 minutes	26 seconds

Dependencies at 99.9%	Composite availability	Downtime per month
1	99.90%	43.8 minutes
5	99.50%	3.6 hours
10	99.00%	7.3 hours
20	98.02%	14.5 hours
50	95.12%	35.6 hours

This is the arithmetic that justifies circuit breakers, bulkheads, deadline propagation and graceful degradation. Airbnb's property page calls 20 services — run the numbers once and the resilience patterns stop looking like over-engineering. (Chapters 2 and 6)

Every scale figure in this guide, dated

System	Figure	As of
Twitter / X	300K timeline reads/s vs ~6K writes/s	2012

System	Figure	As of
	143,199 tweets in one second (peak)	3 Aug 2013
	~500M posts/day; 150B posts served/day	2023
	Ranking: ~5B runs/day, <1.5 s mean, 220 CPU-s each	2023
WhatsApp	~100 billion messages/day	Oct 2020
	147M peak concurrent connections; 342K in / 712K out per second	Mar 2014
	~550 servers, >11,000 cores, 32 engineers at 450M users	Feb–Mar 2014
YouTube	"Multiple hundreds of hours" uploaded per minute	2021
	1 billion hours watched per day	Feb 2017
	Argos VCU: 4.4–7.0× perf/TCO H.264, 20.8–33.3× VP9	2021
Uber	~500K ETA requests/s; ~1,000 per trip; >18M trips/day	—
	CH preprocessing ~12 h world graph; shipped engine single-digit ms	2015
Instagram	~2.5 billion users; 12-second request timeout	2024
Airbnb	9M+ active listings; 148.3M nights booked in the quarter	2026
	~17 bookings/s against ~1,000 reads/s (sector model)	2024–26
	99.999% payment consistency; 20-service page fan-out	2018–19
Spotify	777M MAU, 300M Premium, 100M+ tracks	2026
	265 ms median playback; 8.8% / 35.8% / 55.4% byte split	Mar 2010
	>8M events/s peak, >350 TB/day raw; 810 services	2019 / 2015
Tinder	1.6B swipes/day; ~26M matches/day; 3 of 55 geoshards per query	2024
Zoom	300 million video calls/day	2020

System	Figure	As of
Figma	4M users at the time of writing; 40M+ later	2024
Cloudflare	55M requests/s, ~20% of internet traffic, 15 Postgres clusters	2023
PayPal	1 billion transactions/day on 8 VMs, down from >1,000	2023
Lyft	945.5M rides FY2025; 51.3M annual riders	2026
	Envoy at open-sourcing: >100 services, >2M req/s; Redis ~25M QPS	2016 / 2018
Bluesky	5.5M users and 164M+ follows in ~1.6 GB of SQLite	Apr 2024
	50,000+ custom feeds	2025

Where a figure has a date, use the date. Where it does not, say so. Appendix B covers the ones that are commonly quoted without either.

A Fifteen-Week Study Plan

One case a week, two hours each. Designed so you actually finish.

Reading this guide front to back takes a weekend and will leave you with very little. Working one case a week for fifteen weeks takes thirty hours and changes how you think. The difference is entirely in whether you do the exercises.

The weekly format

Time	Activity	Why
15 min	Read the requirements and "at a glance" only. Then stop and design it yourself on paper.	This is the whole exercise. Reading the architecture first means you learn a fact; designing first means you learn where your instincts are wrong.
10 min	Do the back-of-the-envelope arithmetic before looking at it.	Getting a number an order of magnitude wrong is the most useful feedback available.
40 min	Read the chapter properly. Mark every place your design differed, and why.	The differences are your curriculum. The agreements are confirmation, which teaches nothing.
25 min	Write answers to two or three of the Practice questions. In prose, not bullets.	Prose forces you to notice where the reasoning does not connect. Bullets let you hide it.
20 min	Read one primary source from the chapter's Sources list.	Over fifteen weeks that is fifteen engineering papers and talks — a substantial amount of primary literature.
10 min	Add one line to a running document: the single most transferable idea from this case.	By week fifteen you have written your own pattern catalog, which will be worth more to you than Chapter 4.

The schedule

Week	Case	The one thing to take away	Also do
0	Part I — Foundations	The ladder, the arithmetic, the framework, the fourteen patterns	Identify which rung your own system is on, and which wall it hits next

Week	Case	The one thing to take away	Also do
1	7 · Redis at Scale	One datastore, eight jobs — and which two it should not be doing	Audit what your team uses Redis for against the table in §9
2	11 · Figma	Five techniques in order; only the last raises the write ceiling	Work out which technique your system needs next
3	12 · Cloudflare	Put the control point where requests are still schedulable	Read the TCP Vegas paper. It is short and it will change how you think about limits.
4	1 · Twitter / X	Measure the asymmetry, then put the work on the cheap side — and revisit when the requirement changes	Watch the 2012 "Timelines at Scale" talk, then read the 2023 blog post. In that order.
5	5 · Instagram	Data placement is a latency strategy	Classify your own reads as staleness-tolerant or not
6	15 · Bluesky	Separate primary from derived data; forward rather than index	Try to design "delete my account" in a federated system. Notice what you cannot promise.
7	2 · WhatsApp	Choosing not to store is a scalability strategy	Read the 2014 Erlang Factory talk in full. It is the best performance-investigation story in the corpus.
8	10 · Zoom	Refuse the expensive operation rather than optimizing it	Compare against Chapter 3 on five axes and find where each wins
9	13 · PayPal	Route all operations on an entity to one place, and locks become unnecessary	Construct a deadlock with no shared memory, then fix it
10	3 · YouTube	Find the boundary that makes work independently processable	Read the ASPLOS 2021 Argos paper. It is the best hardware/software co-design case study available.
11	8 · Spotify	Split the object into a latency-critical head and a bulk tail	Read the P2P'10 paper for the measured byte split and the prefetch counterfactual
12	4 · Uber ETA	Design precomputation for incremental invalidation, not just fast reads	Trace a road closure through contraction hierarchies and through cell partitioning
13	14 · Lyft	Cross-cutting reliability belongs in a sidecar	Read the Envoy announcement. The problem statement is the best short argument about microservices in print.
14	9 · Tinder	Never use a uniform grid over a non-uniform distribution	Implement the sorted-set liveness pattern for real, in code

Week	Case	The one thing to take away	Also do
15	6 · Airbnb	Make the invariant a database constraint, not application logic	Actually write the exclusion-constraint DDL and try to break it with concurrent inserts

WHY AIRBNB IS LAST

It is the chapter most candidates handle worst, and it is the one that benefits most from having read everything else. By week fifteen you will have seen locks, leases, queues, idempotency and replica lag from five other angles — and the double-booking problem will read as a synthesis rather than a puzzle.

After week fifteen

1. **Read Appendix B in one sitting.** It will be much more interesting once you know all fifteen systems, and it is the fastest way to calibrate how much of what you read online to trust.
2. **Re-read your own running document** — the fifteen transferable ideas you wrote down. That is your pattern catalog, in your own words, and it is the artefact of this exercise.
3. **Design something not in this book.** A rate limiter, a notification system, a distributed job scheduler, a metrics pipeline. Use the framework from Chapter 3 and the catalog from Chapter 4, and notice how much of it is recombination.
4. **Then design the system you actually work on**, from scratch, as though you had never seen it. This is the most uncomfortable and most valuable exercise in the appendix, and it is the one that occasionally changes what a team does next quarter.

ONE LAST THING

The fifteen systems in this guide are all impressive, and all of them have been substantially rebuilt at least once. Twitter deleted the design it was famous for. WhatsApp halved the number it was celebrated for. Uber rejected the algorithm everyone assumes it uses. Spotify removed the P2P network that defined its early architecture.

None of that means the engineering was wrong. It means architecture is a response to constraints, and constraints move. The reasoning is what survives — which is why every chapter here ends with what transfers rather than with a diagram.